

The Quantum Random Oracle Toolbox

A Tutorial[★]

Hans Heum  and Kathrin Hövelmanns 

¹ Simula UiB, Bergen, Norway

`hansh@simula.no`

² Eindhoven University of Technology, The Netherlands

`k.hovelmans@tue.nl`

Abstract. The Random Oracle Model (ROM) has been highly successful in bringing to life cryptographic schemes that are 1) efficient and 2) tightly secure, with 3) conceptually simple security proofs. However, a quantum attacker would clearly be able to run the primitives our random oracles are modelling (such as SHA3) on a *quantum superposition* of inputs. We can not exclude the existence of quantum attacks that employ such a strategy; hence, if we want to claim post-quantum security relative to a random oracle, we better ensure that our random oracle is *quantum-accessible*. This is the Quantum Random Oracle Model (QROM). Unlike their classical counterparts, proofs of security in the QROM are often 1) conceptually challenging and 2) lossy. These issues boil down in large part to the non-applicability of three simple yet powerful techniques that sit at the heart of the classical random oracle model:

1. Programming: Altering the random oracle on certain inputs
2. Simulation: Lazy-sampling a random oracle
3. Extraction: Learning the inputs to the random oracle

In this gentle introduction to provable security in the QROM, we will survey how tools developed over the past decade recover these capabilities, and how to apply them:

1. The one-way to hiding lemma (for reprogramming)
2. The compressed-oracle technique (for simulation)
3. Online-extractability of quantum random oracles (for extraction)

Along the way, we will meet popular construction paradigms like the Fujisaki–Okamoto transformation and the Fiat–Shamir transformation, underlying the recent post-quantum standards ML-KEM (Kyber) and ML-DSA (Dilithium): Through examples lifted from their proofs of post-quantum security, we will show how and when to apply each tool, and the impact each will have on the overall tightness.

We only assume knowledge of basic linear algebra; hence, this tutorial also serves to introduce both provable security and quantum physics, with ample examples and exercises throughout.

Roadmap.

- v0.1 Primers on provable security, the random oracle model, and quantum computing; the One-way To Hiding (O2H) lemma; OW-security to IND-security via the U-transform
- v0.2 Quantum pseudocode notation; the compressed oracle technique
- v0.3 Online extractability; from CPA to CCA security via the U-transform
- v0.4 The O2H Zoo: SC-O2H, DS-O2H, MRM, MRE, and how each trades between tightness and applicability
- v1.0 Adaptive reprogrammability via the GHM resampling lemma, and the fixed-permutation O2H lemma; from NMA-secure to CMA-secure digital signatures
- v2.0 Measure-and-reprogram; the Fiat-Shamir transformation

Keywords: Post-Quantum Cryptography · The Quantum Random Oracle Model · Tutorial

[★] v0.0 (August 26, 2026)

Table of Contents

1	Preface	4
I Classical Security Is Easy with Random Oracles		
2	Foundations of Provable Security	7
3	The Random Oracle Model (ROM)	17
4	Example 1a: Classical Indistinguishability from the U-transform	20
II Enter Quantum		
5	A Brief Quantum Primer	28
6	The Quantum Random Oracle Model (QROM)	49
III Non-Adaptive Programming		
7	The One-Way to Hiding (O2H) Lemmas	54
8	Interlude: Understanding Tightness Losses	58
9	Example 1b: Post-Quantum Indistinguishability from the U-transform	63
10	Interlude: A Quantum Register Pseudocode	64
11	The Extractors of O2H	68
IV On-the-fly Simulation		
12	Lazy-Sampling QROs with the Compressed-Oracle Technique	69
V Online Extraction		
13	Example 2a: Classical CCA-Security from the U-transform	77
14	The Online-Extractability Technique	78
15	Example 2b: Post-Quantum CCA-Security from the U-transform	79
VI Adaptive Programming		
16	Example 3a: Classical CMA-Security from NIZKs	80
17	The Resampling Lemma and Fixed-Permutation O2H	81
18	Example 3b: Post-Quantum CMA-Security from NIZKs	82
VII Rewinding		
19	Example 4a: Classical NIZKs from the Fiat-Shamir Transformation	83
20	Rewinding Quantum Adversaries with Measure-and-Reprogram	84
21	Example 4b: Post-Quantum NIZKs from the Fiat-Shamir Transformation	85
	References	86
Appendix		
A	The Toolbox	88
B	Quantum Pseudocode: Additional Definitions	89
C	Thinking in Dual Alphabets	90
D	The T-transform	101
E	A Brief Quantum Information Theory Primer	102

F	Deriving the Tools	103
G	Select Exercise Solutions	105

1 Preface

So you want to prove post-quantum security in the Random Oracle Model (ROM).

Security proofs in the ROM enjoy a lot of advantages. First, they usually support strong security claims. Second, the results often achieve near-tightness, thereby enabling highly efficient constructions. Lastly, they often are conceptually simple. This is as close as one gets to a holy trinity for provable security. In contrast, things are less simple in a world with quantum attackers – if we want our security proofs to have merit in a post-quantum scenario, we have to revisit the ROM itself. This is because random oracles model hash functions, that is, publicly available algorithms that can be run by anyone locally on their machine. Nothing is stopping someone with a quantum computer to boot it up and run it on a superposition of inputs. If we want to accurately model quantum attackers, our model must thus reflect such strategies. This led to the development of the Quantum Random Oracle Model (QROM), and it seems that our choice for arguing post-quantum security is between the Standard Model and the QROM. Proofs in the QROM, however, can be notoriously hard to parse without specialized knowledge in quantum information theory, and they often are infamously lossy.

In the standard model, on the other hand, superposition queries are no problem at all, since the standard model provides hash functions to attackers as-is. But even with a lot of the QROM proofs being lossy (so far), we note that when it comes to efficiency, Standard Model constructions nonetheless pale in comparison with their (Q)ROM counterparts. (And, as every cryptographer knows, efficiency tends to trump security in the real world).

As we inch ever closer to a point in time where cryptographically relevant quantum computers might become more than a looming threat, we view it important to lower the barrier for security proofs in the QROM as much as possible, to help the community enable the post-quantum transition in a manner that is safe, efficient, and (provably) secure. One of the most important questions will be if and how far we can draw from the decades of experience with classical ROM proofs, and how to adapt classical proof strategies to quantum attackers.

ROM Techniques Don't Transfer ... As we will see through the proceedings of the tutorial, one of the most useful facts about the random oracle model is that anyone (including a security reduction) can efficiently *simulate* a random oracle, simply by drawing its outputs upon request, one-at-a-time, and then storing them in an appropriate list for book-keeping. This technique, known as *lazy sampling*, allows the reduction to implement the oracle efficiently. It also allows the reduction to see what input values the adversary is interested in. This property sometimes is called *preimage awareness*. As we will see, preimage awareness will be very useful in security reductions because it gives the reduction valuable insights into the attacker's strategies. What's more, the random oracle allows the reduction to *adaptively reprogram* it on inputs of the reduction's choice, allowing the reduction to trick the attacker into behaviour that helps with relating security to the underlying hardness assumption. Indeed, much of the strength of the random oracle model lies in exactly these capabilities.

None of this transfers directly to the case of quantum-accessible oracles. As just one example, think of lazy sampling, and consider an adversary that queries the oracle on a superposition of all possible inputs all at once: to sample, respond on-the-fly and book-keep, the simulator would afterwards have to store an exponentially large list of input values! Besides the challenges surrounding storage, there's more – if the simulator already committed to all oracle output values just to be able to respond to the first query, then how would there be any room left for programming certain oracle outputs according to the reduction's needs?

... But the Concepts Do. And yet, during the past decade the community has developed a plethora of techniques that recover most of these capabilities even in the presence of quantum attackers (albeit at a loss). Prime among these techniques are

- the various *One-Way to Hiding* (O2H) lemmas, which allow for careful reprogramming of random oracle outputs as well as a somewhat limited awareness of query inputs;
- the *Compressed Random Oracle* (CRO) technique, a procedure with which one can efficiently simulate a quantum random oracle, in direct analogy to classical “lazy sampling”, and
- *Quantum Online-Extractability* (QOE), which – as the name suggests – allows for on-the-fly extraction of oracle inputs, at least in some settings.

Outline. We begin by laying some of the foundations, with definitions for basic cryptographic concepts in Sect. 2, a brief recap of the ROM in Sect. 3 and a primer on quantum computing concepts in Sect. 5. In Sect. 10, we introduce a novel pseudocode notation, which will be helpful in reasoning about oracles and reductions. Then in Sect. 3, we finally introduce our main objects of study, the quantum random oracles. Through illustrative examples we will be building intuition, aiming to get a “feel” for how they work.

Hans: outdated below; update once the v0.1 structure is settled

Part I is all about the classical case. Here, we show several examples of security proofs in the random oracle model, all of which are conceptually simple and yield (relatively) tight security. The examples are lifted from the security proofs of two important transformations, the Fujisaki-Okamoto (FO) transformation (which transforms a weakly-secure public-key encryption scheme into a strongly-secure key encapsulation mechanism), and the Fiat-Shamir (FS) transformation (which transforms an interactive zero-knowledge protocol into a non-interactive signature scheme).

We meet these examples again in ??, where we use them to showcase the tools as we introduce them by showing that security holds also in the QROM, though usually with a security loss compared to the classical case, with the particular security loss depending on the tool used. Sect. 7 is all about the One-way To Hiding (O2H) lemma, and its many variants: how to use it to reprogram quantum random oracles, when to employ which version, and the impact of each version on the tightness of the proof. Our running example here will be the second half of the FO transformation, known as the U-transform, and showing that if we start with a one-way secure (OW-CPA) deterministic PKE, we get an indistinguishable KEM secure against passive attacks (IND-CPA). In Sect. 14, we introduce the technique of Quantum Online-Extraction (QOE), and show how it can be used to prove that the U-transform *also* upgrades to security against active attacks (IND-CCA). In Sect. 17, we show how reprogramming can be made *adaptive*, meaning that the reprogramming may now happen on-the-fly, on points that may (in full or in part) depend on adversarially-chosen input values. We use this to show how to go from a passively-secure Non-Interactive Zero-Knowledge (NIZK) protocol (such as constructed from a sigma protocol via the FS transformation) to an actively-secure signature scheme (EUF-CMA). Then, in Sect. 12, we delve into the details of Compressed Random Oracles (CROs), and show how one might implement the technique to provide an efficient yet exact simulation of a quantum random oracle. This incredibly powerful technique takes some effort to wrap one’s head around, and so we dedicate significant space to give gentle and intuitive explanations for all its moving parts, with several examples showing how queries are processed. Finally, in Sect. 20, we show how to lift rewinding arguments from the ROM to the QROM, with the central example being how the FS transformation transforms a kind of interactive zero-knowledge protocol known as a sigma protocol, to a NIZK.

Hans: leave for version 2

?? is all about *constructing the tools themselves*, which requires a deeper dive into the world of quantum information theory.

Hans: and so on and so forth; leave for version 3

The appendices contain extra material, including some more formal definitions in Sect. 10, and solutions to select exercises in App. G.

Hans: fix “Sect.” vs “App.”

K: Undefined

Part I

Classical Security Is Easy with Random Oracles

2 Foundations of Provable Security

Hans: The first two subsections read much more like the preliminaries to a research article than part of a tutorial. Rework into the text when the topics naturally appear? Could also keep a list of notation in an appendix rather than up-front.

Hans: Commenting out for now; here’s a list of things that should be introduced/mentioned.

Hans: Follow Mike Rosulek and use the font $\mathbf{0}, \mathbf{1}, \mathbf{10}, \dots$ for bitstrings and reserve the standard font for natural numbers $0, 1, 2, \dots$? Could extend to variables: $\mathbf{x} \in \{\mathbf{0}, \mathbf{1}\}^{\log_2 N}$ is the bitstring representation of $x \in \mathbb{Z}_N$. Being clear about which one we mean at any point in time could be very helpful at the level of a tutorial.

Notation and conventions

1. The natural numbers start at 1.
2. $[n] = \{1, \dots, n\}$
3. Functions are over bitstrings
4. Set of all functions from $\{\mathbf{0}, \mathbf{1}\}^n$ to $\{\mathbf{0}, \mathbf{1}\}^m$: Use $\{f : \{\mathbf{0}, \mathbf{1}\}^n \rightarrow \{\mathbf{0}, \mathbf{1}\}^m\}$?
5. $n \times n$ identity matrix: $\mathbb{1}_n$ (or $\mathbb{1}$ when the context is clear).
6. $|x|$ is bitstring length, $|\mathcal{X}|$ is set cardinality; likewise for lists
7. The i th list element is written $\mathbf{X}(i)$
8. Lists may be combined by concatenation: $\mathbf{X} \parallel \mathbf{Y}$
9. It will at times be useful to treat bitstrings as lists of binary values
10. Assignment and sampling:
 - $x := y$ for deterministic assignment of a value or from a deterministic algorithm
 - $x \leftarrow X(\cdot)$ for probabilistic assignment from algorithm or distribution
 - $x \leftarrow \$\mathcal{X}$ for uniform sampling from set
11. Conditional probability when running **Code** is denoted $\Pr[\mathbf{Code} : \mathbf{Event} \mid \mathbf{Condition}]$
12. and the same for expectations
13. $\llbracket \mathbf{Condition} \rrbracket$: the predicate that returns 1 if **Condition** is true and 0 otherwise. (Let’s use bit values for **true** and **false**.)
14. Algorithms are classical or quantum Turing machines (although we use the circuit model for quantum algorithms).
15. Oracle algorithms exist. Oracles often have unbounded resources.
16. Stateless, deterministic oracles implement a function, in which case we may refer to it by $\mathcal{O}_f$
17. In pseudocode, functions that return no output instead output $\checkmark$
18. States and statefulness (such as for multi-stage adversaries and random oracle simulations) can probably be left implicit by the notation

2.1 Expressing Security as Games

This subsection revisits the game-based approach to provable security. Feel free to skip it if you are familiar with how game-based security proofs work.

Hans: Let’s just skip using the word “experiment” entirely.

Security Games. Security is defined in terms of a *security game*. You can envision these as a game in which a player (or *adversary*) is challenged to break security of a cryptographic construction (or *primitive*), where the win conditions of the game captures what “breaking” means in the specific context. More formally, we will model the “player”/“adversary” as an algorithm $\mathcal{A}$.³ A security game **Game** is a program, usually written in pseudocode, that interacts with $\mathcal{A}$ by running it as a black-box subroutine; see Fig. 1 for an example. **Game** will use $\mathcal{A}$ as a *black-box*, meaning it can only provide inputs to and receive outputs from $\mathcal{A}$, because we do not want to make any assumptions about $\mathcal{A}$ ’s potential attack strategies. This way, we can confidently say that our security experiment considered *any* kind of attack that aimed to break the formalized security goal. At the end, **Game** will finish by outputting a bit that depends on the interaction with $\mathcal{A}$, to indicate if $\mathcal{A}$ managed to break security.

³ In technical terms, $\mathcal{A}$ is assumed to be a probabilistic (or quantum) Turing machine.

Any security game is defined to express security of some primitive P , for example a particular public-key encryption (PKE) scheme. Thus, part of the experiment's code uses the algorithms of the primitive P whose security we want to express. E.g., we might want to write an experiment to express that encryptions of a PKE scheme PKE are “one-way” in the sense that they are hard to invert without the secret key. To express this, the “**One-Wayness** under **Chosen-Plaintext Attacks**” game OW-CPA would encrypt a random plaintext message m^* into a ciphertext c^* , hand c^* to $\mathcal{A}$, have $\mathcal{A}$ output a guess $\hat{m}$, and return 1 if $\mathcal{A}$ guessed the correct encrypted message (so if $\hat{m} = m^*$). This is the example formalized in Fig. 1.

Like all common definitions of security, the OW-CPA definition is intended to be *generic*, meaning it can capture this type of security for *any* kind of PKE scheme. To account for that, the game's template uses abstract placeholders for the specifics of PKE, like **KeyGen** for the key generation, **Enc** for the encryption algorithm, and $\mathcal{M}$ for the space of messages that PKE can encrypt. If we want to express security of, say, ElGamal encryption, we would fill in the specifics of ElGamal at the respective places. For any security game filled in with the specific algorithms of a concrete primitive P , we will write $\text{Game}_P(\mathcal{A})$. We will say that adversary $\mathcal{A}$ wins the game **Game** for a specific construction of primitive P if $\text{Game}_P(\mathcal{A})$ outputs 1. We will denote this event by $\text{Game}_P(\mathcal{A}) \Rightarrow 1$. In that case, we will also say that adversary $\mathcal{A}$ broke the **Game** security of primitive P .

Security games are typically named by the convention that the letters preceding the hyphen defines a security *goal* that we would like our primitive P to achieve, and the letters after the hyphen lists the attack *power* of the adversaries with respect to which it will achieve that security goal: so GOAL-POWER. In the example of OW-CPA, the *goal* is the one-wayness of encryption, while the adversary's *power* is chosen plaintext attacks (CPA), i.e. the power to generate encryptions at will of any message it likes. Since this example deals with public-key encryption, this power is given for free, and so this is the minimal power that the adversary can have for a well-defined notion of PKE security. For other primitives the adversary could have less attack power: consider for example symmetric encryption, where you need to know the secret key to encrypt. In other words, if attackers are not handed the secret key, then they cannot produce encryptions on their own. It would thus be plausible to formalize the idea that a symmetric scheme is one-way, restricted to the requirement that the attacker never got to see an encryption. This weakest notion for symmetric encryption might then be called OW-NMA, for “no message attack”.

The Strength of a Security Notion. What do we mean when we say that a notion such as OW-NMA would be “weak”? Well, in general the *strength* of the notion scales with how “easy” it would be for a sufficiently strong adversary to win the game. As an example, look at a security notion for PKE with a simpler *goal*, such as guessing which of two known messages are encrypted (this is known as *indistinguishability*, or IND). This is a stronger definition than one with a more complex goal, such as recovering the full message from scratch like in the above example. (You can convince yourself that any attacker who is capable of recovering the full message can easily determine which of two known messages was encrypted, by inverting the received ciphertext and seeing which of the two messages was recovered.) We have lowered the bar for what counts as “breaking” the scheme. If no adversary can win at the easier game, then there certainly aren't any adversaries that can win at the harder one either. Hence, claiming security under an easier-to-break goal gives a *stronger* statement.

The other lever with which we can control the strength of a security notion is of course the attack power given to the adversary: more power means a stronger security notion. For example, consider again the case of symmetric encryption: we could give the adversary the power of chosen-plaintext attacks by simply *providing* it with encryptions of its choosing whenever it asks. The claim that the game *still* can not be won by any adversary is stronger than claiming that it can not be won without this ability.

So: OW-NMA is a weaker notion of security than OW-CPA, and IND-CPA is stronger than OW-CPA (and thus also stronger than OW-NMA). Just for fun, you could also add IND-NMA to the zoo. As it so happens, for *symmetric* schemes, OW-CPA and IND-NMA would in general be *incomparable* in terms of strength: A (symmetric) encryption scheme might be secure with respect to one notion while being completely broken in terms of the other!⁴

Hans: Changed to this style, except with the adversary still as “input”. Open to changing my mind about that (I think usually the input slot would be reserved for the security parameter in asymptotic notions?), but I feel like it might be useful to have the superscript available for e.g. bit values in game hops.

⁴ It is a fun puzzle to try to come up with encryption schemes that exemplify this behaviour (secure under one notion, insecure under the other); a little creative thinking should allow you to find some. (Hint: As a starting point, assume that you have an encryption scheme that is secure under both notions, then think about how you could alter it so that it “breaks” under one notion but not under the other.)

K: use xspace in macro for P

K: I noted that we're mixing "Game" and a lot. I would pick either/or and commit to that. Personally, I prefer the notation "**Game** OW-CPA_{PKE}^A", but I won't mope as long as everything is consistent.

Game $\text{OW-CPA}_{\text{PKE}}(\mathcal{A})$	
01	$(\text{pk}, \text{sk}) \leftarrow \text{KeyGen}$
02	$m^* \leftarrow \$ \mathcal{M}$
03	$c^* \leftarrow \text{Enc}_{\text{pk}}(m^*)$
04	$\hat{m} \leftarrow \mathcal{A}(\text{pk}, c^*)$
05	return $\llbracket \hat{m} = m^* \rrbracket$

Fig. 1. Example for a security game: “**O**ne-**W**ayness under **C**hosen-**P**laintext **A**ttacks” (OW-CPA) game for a PKE scheme with key generation KeyGen , encryption algorithm Enc , and supported message space $\mathcal{M}$.

Advantages. When designing cryptographic constructions, we will sometimes find a design that is quite nice, but vulnerable “in principle”. By that we mean that in principle, there exists some theoretical attack (i.e., an attack algorithm) that would break the construction, but it will never be useful in practice, because that attack

- is so slow that no-one can execute it in a reasonable amount of time, or
- has so little chance at actually succeeding that no-one can execute it and reasonably hope to achieve anything.

Would we want to throw away that design just because there exists a theoretical attack that will never matter in practice? That would be quite a waste. When studying the security of a primitive, we thus want to be able to draw a line between attacks that are

- feasible or “efficient” (which we want to protect against); and ones that are
- infeasible or “inefficient” (which we don’t need to care about),

so that we can express security against “just the feasible attacks”. Similarly, we want to be able to draw a line between attacks whose chances at success are

- dangerously high or “significant”; and ones that are
- not credibly threatening or “insignificant”,

so that we can express security against “just the credibly threatening attacks”.

Intuitively, a primitive P is considered “broken” if there exists an “efficient” adversary that wins the game with a “significant” probability. Conversely, P is “secure” if no such adversary exists. But what does it mean to be “efficient” or “significant”? Intuitively, you can think of “efficient” attacks as attacks whose running time doesn’t blow up catastrophically if we increase the construction’s parameters, like, e.g., the key length. Similarly, you can think of “significant” probabilities as probabilities that do not speed-run towards 0 upon parameter increase.

Under the paradigm of *asymptotic security*, one would identify “efficient” with “scales as a polynomial in the security parameter”, and “significant” as “decays faster than any inverse polynomial”. As an example, if we have security parameter λ and the attack has running time λ^2 , it is polynomial, meaning it does not blow up catastrophically and we classify it as “efficient”. If it is exponential, such as 2^λ , then it is super-polynomial and we discard it as too slow. Similarly, we view a success probability $1/2^\lambda$ as insignificant because it speed-runs towards 0, whereas we view $1/\lambda^2$ as “significant”.

In practice, however, cryptographic designs need to make concrete parameter choices, and such asymptotic notions do not necessarily help with assessing how secure the concrete design is in practice. We will thus follow the *concrete security* approach, expressing security via explicit bounds that relate the adversary’s resources to its probability of successfully breaking our scheme. This success probability is captured by the notion of an *advantage*. In our prior example, this would be the probability that an attacker can find the original message m^* belonging to a ciphertext c^* .

How the advantages are defined, concretely, depends on what kind of security notion we are talking about. The security notions we care about all fall into one of two categories:

- *Finding*, in which case the adversary is tasked with finding some secret value (like our ciphertext inversion example). Here, advantages will usually be taken to simply be the win probability:

$$\text{Adv}_P^{\text{FIND}}(\mathcal{A}) := \Pr[\text{FIND}_P(\mathcal{A}) \Rightarrow 1].$$

- *Distinguishing*, in which case the adversary is tasked with distinguishing between two scenarios. (E.g., the adversary might need to distinguish a real encryption from a ciphertext that was chosen uniformly at random.) Here, the expected output of $\mathcal{A}$ is a bit, to indicate its guess about which scenario (or “world”) it interacted with.

In “distinguishing” notions, the security game often chooses one of the two scenarios, with equal probability. Consequently, any adversary could win with probability $1/2$ by simply making a random guess. We would like the adversary’s *advantage* to capture how much *better* the adversary can do than randomly guessing. There are several ways to do this. While they all are equivalent, it will sometimes be useful to switch between them.

One standard way of defining distinguishing advantages is the following:

$$\text{Adv}_{\mathcal{P}}^{\text{DIST}}(\mathcal{A}) := |2 \cdot \Pr[\text{DIST}_{\mathcal{P}}(\mathcal{A}) \Rightarrow 1] - 1|.$$

Here, the game internally tosses a coin to determine which of the two to-be-distinguished worlds the adversary will be interacting with, and the game outputs 1 if the adversary guessed the correct world. Note how scaling by a factor 2 and subtracting 1 makes the advantage a value that runs between 0 (for strategies that are no better than guessing) to 1 (for strategies that always win), matching the “finding” case.

Alternatively yet equivalently, you could split the experiment into the two to-be-distinguished scenarios and quantify how much more (un)likely it is that $\mathcal{A}$ outputs 1 in the one setting than the other. To do so, you can capture the outcome of the coin toss as a “challenge bit” b , condition the experiment on b , and set

$$\text{Adv}_{\mathcal{P}}^{\text{DIST}}(\mathcal{A}) := |\Pr[\text{DIST}_{\mathcal{P}}(\mathcal{A}) : 1 \leftarrow \mathcal{A} \mid b = 0] - \Pr[\text{DIST}_{\mathcal{P}}(\mathcal{A}) : 1 \leftarrow \mathcal{A} \mid b = 1]|.$$

Hans: Instead define $\text{Adv}_{\mathcal{P}}^{\text{DIST}}(\mathcal{A}) := |\Pr[\text{DIST}_{\mathcal{P}}^0(\mathcal{A}) : 1 \leftarrow \mathcal{A}] - \Pr[\text{DIST}_{\mathcal{P}}^1(\mathcal{A}) : 1 \leftarrow \mathcal{A}]|$?

We again get a formula that scales from 0 to 1: If $\mathcal{A}$ always makes a random guess, then the two probabilities are equal, and the advantage is 0; while if it always guesses correctly, then one of the two probabilities is 0 and the other one is 1, making for an advantage of 1.

Hans: Duly noted. The (perhaps unstated) convention I had been using was that distinguishing games output a truth value `true/false`, so that the games could be treated as a predicate; but I agree that talking about bit values is simpler, so moving to this convention.

K: I noticed that you abbreviated the prob by writing $\Pr[\mathbf{X}_{\mathcal{P}}(\mathcal{A})]$, but $\mathbf{X}_{\mathcal{P}}(\mathcal{A})$ is not really an event. I’m more used to denoting the prob of the event that the experiment *outputs 1*, by writing $\Pr[\mathbf{X}_{\mathcal{P}}(\mathcal{A}) \Rightarrow 1]$. I changed it above, but not in the remainder of the file.

2.2 Cryptographic Primitives

We next introduce the main cryptographic schemes we will be concerned with: public-key encryption, key-encapsulation mechanisms, digital signatures, and, of course, hash functions.

Our definitions are slightly simplified in that they leave parameter generation implicit.⁵ Also for simplicity, we demand our schemes satisfy perfect correctness, whereas many post-quantum schemes actually exhibit a small correctness error, something the security proofs will have to account for [HHK17].

Public-Key Encryption. The quest for asymmetric cryptography began with Diffie and Hellman’s idea of public-key encryption [DH76], so fittingly this is also where we begin.

Definition 1 (Public-Key Encryption). A public-key encryption scheme $\text{PKE} = (\text{KeyGen}, \text{Enc}, \text{Dec})$ is a triple of algorithms defined with respect to public-key space $\mathcal{PK}$, secret-key space $\mathcal{SK}$, message (plaintext) space $\mathcal{M}$, ciphertext space $\mathcal{C}$, and randomness space $\mathcal{R}$. More precisely, the triple of algorithms is defined as follows.

- $\text{KeyGen}() : (\text{pk}, \text{sk}) \in \mathcal{PK} \times \mathcal{SK}$. Probabilistic algorithm that takes no input and outputs a “keypair” (pk, sk) .
- $\text{Enc}(\text{pk} \in \mathcal{PK}, m \in \mathcal{M}) : c \in \mathcal{C}$. Probabilistic algorithm that, given a public key pk and message m , produces an “encryption” (“ciphertext”) c of m under pk . We may explicitly specify the randomness used in encryption by writing $\text{Enc}(\text{pk}, m; r)$, where $r \in \mathcal{R}$.

⁵ To include them, enhance the definitions below with a parameter generation algorithm PmGen , which takes as input a security parameter encoded in unary 1^λ , and outputs a set of public, shared parameters pm , to be provided as input to KeyGen . Then, add parameter generation to the top of every security game.

- $\text{Dec}(c \in \mathcal{C}, \text{sk} \in \mathcal{SK}) : m \in \mathcal{M} \cup \{\perp\}$. *Deterministic algorithm that, given a secret key sk and ciphertext c , yields either a message $m \in \mathcal{M}$ or an explicit failure symbol $\perp \notin \mathcal{M}$, the latter indicating that c is not a valid ciphertext (under sk).*

To ease readability, we will use the convention that the “key” part of the input is provided as a subscript, so $\text{Enc}_{\text{pk}}(m) := \text{Enc}(\text{pk}, m)$ and $\text{Dec}_{\text{sk}}(c) := \text{Dec}(\text{sk}, c)$. We will also use the convention that if you know a secret key sk belonging to a keypair (pk, sk) , then you also know pk . This is w.l.o.g. in the sense that after a keypair (pk, sk) is generated, you can always re-define the keypair to instead be $(\text{pk}, (\text{pk}, \text{sk}))$ (though it enforces a unique-public-key-per-secret-key relation, and there exist PKE schemes that violate this [BHY09]).

As mentioned, we require that PKE exhibits perfect correctness:

$$\forall m \in \mathcal{M} : \Pr[(\text{pk}, \text{sk}) \leftarrow \$ \text{KeyGen}, c \leftarrow \$ \text{Enc}_{\text{pk}}(m) : \text{Dec}_{\text{sk}}(c) = m] = 1.$$

We also require perfect *rigidity* [BP18, HHMS25], which can be thought of as the reverse direction of perfect correctness, and which says that there are no ciphertexts in $\mathcal{C}$ that successfully decrypt under sk , yet can not be reproduced as an encryption of the decrypted message under pk :

$$\forall c \in \mathcal{C} : \Pr[(\text{pk}, \text{sk}) \leftarrow \$ \text{KeyGen}, m \leftarrow \text{Dec}_{\text{sk}}(c) : m \neq \perp \wedge \forall_{r \in \mathcal{R}} \text{Enc}_{\text{pk}}(m; r) \neq c] = 0.$$

Key Encapsulation Mechanisms. A key encapsulation mechanism is a public-key primitive that samples a symmetric key, encrypts (or *encapsulates*) it, and returns both.

Definition 2 (Key Encapsulation Mechanism). A key encapsulation mechanism $\text{KEM} = (\text{KeyGen}, \text{Encaps}, \text{Decaps})$ is a triple of algorithms defined with respect to public-key space $\mathcal{PK}$, secret-key space $\mathcal{SK}$, symmetric key space $\mathcal{K}$, and encapsulation space $\mathcal{C}$. More precisely, the triple of algorithms is defined as follows.

- $\text{KeyGen}() : (\text{pk}, \text{sk}) \in \mathcal{PK} \times \mathcal{SK}$. *Probabilistic algorithm that takes no input and outputs a “keypair” (pk, sk) .*
- $\text{Encaps}(\text{pk} \in \mathcal{PK}) : (K, c) \in \mathcal{K} \times \mathcal{C}$. *Probabilistic algorithm that, given a public key pk , produces a tuple (c, k) , where c is said to be an “encapsulation” of the key k .*
- $\text{Decaps}(\text{sk} \in \mathcal{SK}, c \in \mathcal{C}) : k \in \mathcal{K} \cup \{\perp\}$. *Deterministic algorithm that, given a secret key sk and ciphertext c , yields either a key $k \in \mathcal{K}$ or an explicit failure symbol $\perp \notin \mathcal{K}$, the latter indicating that c is not a valid ciphertext (under sk).*

It is easy to see that at least syntactically, a KEM can be built from any PKE, by first sampling a symmetric key and then encrypting it. In the other direction, a PKE can be built from a KEM by combining it with any symmetric encryption scheme (such as AES), so that the KEM encapsulates a symmetric key, and the symmetric scheme encrypts the message. In this context, the symmetric scheme is referred to as a “DEM”, for “Data Encapsulation Mechanism”, and the resulting PKE is called a “KEM-DEM hybrid encryption scheme”. Since symmetric keys are typically much shorter than messages, such PKEs are usually a lot more efficient than using the underlying PKE scheme to encrypt the message directly, owing to the superior efficiency of symmetric encryption.

K: Sure, syntactically.
But is it secure? ;P

Digital Signature Algorithms.

Hans: Add if/when relevant. Should I add sigma protocols and NIZKs too?

Hash Functions. A hash function $\mathcal{H} : \{0, 1\}^* \rightarrow \{0, 1\}^m$ is a deterministic but random-looking function that takes inputs of any size, and returns a comparably small “fingerprint”, or *digest*, of that input.⁶ Hash functions are ubiquitous in computing: As just one example out of many, they can be used to efficiently check if two large files are equal, by simply comparing their much smaller “fingerprints” instead.

Cryptography puts strict requirements on hash functions, and we demand that any cryptographic hash function should satisfy *one-wayness* and *collision resistance*. We will define these shortly, though it is perhaps easy to see that a uniformly random function should satisfy both (for large enough m). This is relevant, as hash functions are precisely the objects that we will model as random oracles in Sect. 3.

PKE Security. There are many ways that one could define security for public-key encryption, but a few have emerged as standard.

K: Suggestion to move the security notions to the corresponding primitives

One-Wayness. At minimum, a secure PKE scheme must hide the message being encrypted. We can formalize this as “One-Wayness under Chosen-Plaintext Attacks”, or OW-CPA, where “under Chosen-Plaintext Attacks” is simply to say that anyone with access to a public key can use it to generate ciphertexts for any message they wish. (In other words, the CPA capability comes for free.)

The one-wayness game is shown to the left in Fig. 2. As you can see, the game is defined in pseudocode. With experience, you will learn to skim pseudocode-games like these to quickly grasp the main concepts, but in case you are new to security games, here are a few things to notice:

- The encrypted message m is sampled uniformly at random at the start of the game, before the adversary $\mathcal{A}$ is ever invoked, and so $\mathcal{A}$ can not influence the distribution that the message was sampled from at all. This should be considered a weakness of the notion, since less adversarial power $\rightarrow$ weaker security guarantees: What if $\mathcal{A}$ can influence the choice of message somehow and some messages are known to give insecure ciphertexts?
- The adversary $\mathcal{A}$ (who is a Turing machine) is called as a subroutine exactly once in a black-box manner, meaning that once $\mathcal{A}$ is given an input, the game only continues once $\mathcal{A}$ halts with an output.
- The adversary is only given a single challenge ciphertext, in addition to the public key. More general security notions might allow $\mathcal{A}$ to receive many challenge ciphertexts, and maybe even multiple public keys, and allow $\mathcal{A}$ to win if it reverses any one of them.
- Also notice that $\mathcal{A}$ is not given any information beyond the public key and the challenge ciphertext it is tasked with breaking. In particular, it is not given access to any additional oracles that provide extra capabilities, such as, e.g., an oracle that the attacker can invoke to obtain decryptions.

Definition 3 (OW-CPA). The one-wayness advantage $\text{Adv}_{\text{PKE}}^{\text{OW-CPA}}(\mathcal{A})$ of an adversary $\mathcal{A}$ against a public-key encryption scheme PKE is the “find” advantage (??) of $\mathcal{A}$ in the OW-CPA game (Fig. 2, left) when instantiated with the scheme PKE.

You might wonder if it would not be a more natural notion for PKE that is is hard to compute the secret key from the public key? Well, this is a perfectly valid notion of security—let’s call it SKU, for “**S**ecret-**K**ey **U**nrecoverability”. As before, since anyone can encrypt messages, the adversary has the power of Chosen-Plaintext Attacks, this leads to security notion SKU-CPA.

Notice, though, that anyone who can compute the secret key from the public key, can also decrypt any ciphertext that were created using that public key. This means that if an adversary $\mathcal{A}$ can recover secret keys, then they can also win the OW-CPA game, whereas the opposite does not seem to hold in general (since there might be ways to attack a ciphertext that does not rely on knowing the secret key). Hence, OW-CPA gives a *stronger* security guarantee than SKU-CPA – if one-wayness is satisfied, then secret-key unrecoverability must also be satisfied. We say that OW-CPA *implies* SKU-CPA.

Indistinguishability. Another standard notion of security for public-key encryption is **I**ndistinguishability under **C**hosen-**P**laintext **A**ttacks” (IND-CPA). Again, the CPA capability comes for free since with public-key encryption, any adversary can generate as many encryptions as it wants, for whatever message it chooses. (For symmetric schemes, this would have to be provided via an encryption oracle.)

K: missing ref

On the left of ?? is the experiment defining our (single-user) IND-CPA security notion. The game starts by sampling parameters pm , a keypair (pk, sk) , and a challenge bit b , before handing the public key to the adversary $\mathcal{A}$. If the construction in question uses a random oracle $\mathcal{H}$, then $\mathcal{A}$ also gets oracle access to $\mathcal{H}$.

K: why suddenly parameters?

K: We never mentioned ROs, in particular not in the OW notion, why suddenly here? I’d completely gloss over ROs for now.

At some point, the adversary decides on two equal-length messages m_0 and m_1 , and call its *challenge encryption* oracle $\mathcal{E}$ (??, middle). This oracle then provides an honest encryption of the message m_b and returns a challenge ciphertext c^* . If the adversary ever calls the challenge oracle on messages of differing lengths, then it returns the special “fail” symbol $\perp$ instead; this is due to the assertion that a scheme should be considered secure even if ciphertexts leak information about message lengths. In fact, you may notice from ?? that our BRE scheme leaks the message length exactly, since c_2 must necessarily have the same length as m .

K: missing ref

K: undefined?

We are in the *multi-challenge* setting, and so the adversary may continue to make as many calls to $\mathcal{E}$ as it likes, in addition to arbitrary calls to the random oracle $\mathcal{H}$. (If the adversary is restricted to a single

⁶ Technically, then, a hash function is a *family* of functions, one for each input length.

Game OW-CPA _{PKE} ($\mathcal{A}$)	Game IND-CCA _{PKE} ($\mathcal{A}$)	$\mathcal{E}(m_0, m_1)$	$\mathcal{D}(c)$
01 $(pk, sk) \leftarrow \text{KeyGen}$	06 $(pk, sk) \leftarrow \text{KeyGen}$	10 if $ m_0 \neq m_1 $: return `	14 if $c \in \mathcal{C}$: return `
02 $m^* \leftarrow \mathcal{M}$	07 $b \leftarrow \mathcal{S} \{0, 1\}$	11 $c^* \leftarrow \text{Enc}_{pk}(m_b)$	15 $m \leftarrow \text{Dec}_{sk}(c)$
03 $c^* \leftarrow \text{Enc}_{pk}(m^*)$	08 $\hat{b} \leftarrow \mathcal{A}^{\mathcal{E}, \mathcal{D}}(pk)$	12 $\mathcal{C} \leftarrow \bigcup c^*$	16 return m
04 $\hat{m} \leftarrow \mathcal{A}(pk, c^*)$	09 return $\llbracket \hat{b} = b \rrbracket$	13 return c^*	
05 return $\llbracket \hat{m} = m^* \rrbracket$			

Fig. 2. The single-challenge OW-CPA game and multi-challenge IND-CCA game for $\text{PKE} = (\text{KeyGen}, \text{Enc}, \text{Dec})$.

call to $\mathcal{E}$, we call this the *single-challenge setting*.) Once it is satisfied, $\mathcal{A}$ halts with a guess $\hat{b} \in \{0, 1\}$, and wins if it guessed b correctly.

We measure security in terms of $\mathcal{A}$'s *distinguishing advantage*, i.e. how much better it can do than random guessing.

As is easiest seen from the first of the above three equations, the advantage is just the probability that $\mathcal{A}$ wins the game, minus the probability of winning based on a random guess, scaled so that the advantage is a value defined on the interval from 0 to 1, where 0 means $\mathcal{A}$ has no advantage (e.g. if the messages are perfectly hidden), and 1 means that $\mathcal{A}$ is a perfect distinguisher (the scheme is completely broken, or $\mathcal{A}$ is unbounded).

KEM Security. We also recall the standard security definition for KEMs, Indistinguishability under Chosen Ciphertext Attacks (IND-CCA).

Definition 4 (IND-CCA). We define the IND-CCA game as in Figure 3 and the IND-CCA advantage function of an adversary $\mathcal{A}$ (with binary output) against KEM as

$$\text{Adv}_{\text{KEM}}^{\text{IND-CCA}}(\mathcal{A}) = \left| \Pr \left[\text{IND-CCA}_{\text{KEM}}^{\mathcal{A}} \Rightarrow 1 \right] - \frac{1}{2} \right|.$$

Game IND-CCA _{KEM} :	DEC($c \neq c^*$):
01 $(pk, sk) \leftarrow \text{KeyGen}$	07 $K \leftarrow \text{Decaps}(sk, c)$
02 $b \leftarrow \mathcal{S} \{0, 1\}$	08 return K
03 $(K_0, c^*) \leftarrow \text{Encaps}(pk)$	
04 $K_1 \leftarrow \mathcal{K}$	
05 $b' \leftarrow \mathcal{A}^{\text{DEC}}(pk, c^*, K_b)$	
06 return $\llbracket b' = b^* \rrbracket$	

Fig. 3. IND-CCA game for $\text{KEM} = (\text{KeyGen}, \text{Encaps}, \text{Decaps})$. IND-CPA is similarly defined, except without the decryption oracle.

Hash Functions.

Hans: NB: copy-pasted from Sect. 3, which leads to some duplicated text; rewrite.

A hash function can be thought of as an algorithm that takes an input of any length and outputs a short, fixed-length “fingerprint” of the input, such that re-evaluating the hash function on the same input produces the same fingerprint. They are, in other words, *deterministic*.

For a hash function to be suitable for cryptographic purposes, the fingerprints it produces should be random-looking, and it should be hard to find two inputs with the same fingerprint.

2.3 Reductions

While looking at security notions for PKE, we saw that some notions can be related in a way. As one example, we argued that OW-CPA *implies* SKU-CPA – if one-wayness is satisfied, then secret-key

unrecoverability must also be satisfied. As another example, we argued that OW-CPA *implies* IND-CPA – if indistinguishability is satisfied, then one-wayness must also be satisfied.

Proving implications like the above is one of the core activities of provable security. So though the statement may seem trivial, let's try our hand at proving it.

The main tool we have in proving security is that of a *security reduction* (...)

Hans: skip the following?

We say that a reduction is *simple* if it only runs the adversary once, and *fully black-box* if it only accesses the adversary and cryptographic primitives in black-box ways, running the adversary from start to finish without tampering with its internal workings, answering oracle queries along the way, and otherwise only providing its input and reading its output. In contrast, a reduction might for example *rewind* an adversary, running it several times to figure out how it *would* behave given different inputs; such a reduction would not be considered simple. Generally speaking, simple, fully black-box reductions are preferred, as they are more general and easier to work with, but such reductions are not always possible to construct.

Since simple, fully black-box reductions only access the adversary in a black-box manner, they do not depend on the specific workings of the adversary, and may therefore be quantified before it.

Hans: TRIVIAL REDUCTION EXAMPLE TO SHOW THE FRAMEWORK: IND $\rightarrow$ OW

If, given a ciphertext, you have the ability to break encryption and recover the underlying plaintext, then you can also distinguish which of two ciphertexts were encrypted. When talking about reductions between security notions, we usually state the contrapositive: that “*indistinguishability* (the inability to distinguish) implies *one-wayness* (the inability to recover plaintexts)”.

Let us prove this simple fact as a demonstration of how we usually construct security reductions.

Theorem 1 (IND $\Rightarrow$ OW). *Let PKE be any public-key encryption scheme. Then there exists a simple, fully black-box reduction $\mathcal{B}$, such that for every adversary $\mathcal{A}$,*

$$\text{Adv}_{\text{PKE}}^{\text{OW-CPA}}(\mathcal{A}) \leq \text{Adv}_{\text{PKE}}^{\text{IND-CPA}}(\mathcal{B}^{\mathcal{A}}),$$

where $\mathcal{B}$ runs in the same time as $\mathcal{A}$.

Parsing the Statement.

Hans: cut-and-pasted from the next section

If you are new to security reductions, it may be hard to see at a glance how the theorem statement connects to the intuition that “if dPKE achieves one-wayness security, then KEM achieves distinguishing security”, so let us take a moment to parse the different parts of the statement.

On the left-hand side of the main inequality, we have the distinguishing advantage of our given adversary $\mathcal{A}$. The theorem quantifies over all possible $\mathcal{A}$, so you may think of this as the maximum possible distinguishing advantage against the construction. On the right-hand side, we have an upper-bound on this largest-possible distinguishing advantage. Hence, if the right-hand side is small, then the largest-possible distinguishing advantage against KEM must also be small. This is just a different way of saying that KEM is indistinguishable. The term on the right-hand side is the one-wayness advantage of our *constructed* adversary $\mathcal{B}$. Note the quantifiers: the theorem statement says that “for every (given) $\mathcal{A}$, there exists a (constructed) $\mathcal{B}$ ”. Hence, while we have no way of knowing the inner workings of the given adversary $\mathcal{A}$, we are free to construct $\mathcal{B}$ to our liking.

The trick to understand how this “proves security” is to think contrapositively. Recall that what we want to prove is that “if dPKE is secure, then KEM is secure”. The contrapositive statement that we are actually proving is “if KEM is *insecure*, then dPKE must *also* be insecure”. Hence, instead of thinking of $\mathcal{B}$'s advantage as an upper-bound on the best-possible $\mathcal{A}$ advantage, think of this best-possible $\mathcal{A}$ advantage as a *lower-bound* on the advantage an adversary $\mathcal{B}$ could achieve against dPKE: If the former is large (KEM is broken), then the latter must also be large (dPKE is broken). Since we started out assuming that dPKE is secure, this gives us our proof-by-contradiction.

Hans: might be helpful to the SFBB framework to show explicitly that $\mathcal{B}$ gets oracle access to $\mathcal{A}$ in the theorem statement itself

Hans: the way I wrote the reduction now, it has 50% chance of winning in the event that $\mathcal{A}$ failed to recover the correct message (by returning 0 rather than $\perp$), so that the inequality, which is useful to keep around for a sense of “direction”, is actually warranted.

K: I still wanna add motivation here

Reduction $\mathcal{B}^{\mathcal{E}, \mathcal{A}}(\text{pk})$	
01	$m_0, m_1 \leftarrow \{0, 1\}^n$
02	$c^* \leftarrow \mathcal{E}(m_0, m_1)$
03	$\hat{m} \leftarrow \mathcal{A}(\text{pk}, c^*)$
04	if $\hat{m} \notin \{m_0, m_1\}$: return 0
05	else return $\llbracket \hat{m} = m_1 \rrbracket$

Fig. 4. The reduction $\mathcal{B}$, simulating the OW-CPA game for $\mathcal{A}$ in such a way that if $\mathcal{A}$ wins, then $\mathcal{B}$ wins.

Proof. The reduction is given in Fig. 4. It simulates ...

□

Your turn.

Exercise 1. In this exercise, you will prove that OW-CPA is at least as strong as SKU-CPA by showing the implication OW-CPA $\implies$ SKU-CPA.

- Write down the pseudocode for the SKU-CPA game.
- Assume that there is an adversary $\mathcal{A}$ that wins the SKU-CPA game with some advantage. Write down the pseudocode for a new adversary $\mathcal{B}$ (called a *reduction*), which, given black-box access to $\mathcal{A}$, is able to win the OW-CPA game with the same advantage.
- Use the above to prove the following statement^a: *For every adversary $\mathcal{A}$, there is a reduction $\mathcal{B}$ such that for any public-key encryption scheme PKE,*

$$\text{Adv}_{\text{PKE}}^{\text{SKU-CPA}}(\mathcal{A}) \leq \text{Adv}_{\text{PKE}}^{\text{OW-CPA}}(\mathcal{B}).$$

- How does this statement capture the implication?
- What would suffice to prove that the opposite implication does not hold, i.e. that SKU-CPA $\not\Rightarrow$ OW-CPA?

Hint: In the reduction, it is imperative that $\mathcal{A}$ does not *notice* that they are not in the real SKU-CPA game. In other words, $\mathcal{B}$ should *perfectly simulate* the OW-CPA environment for $\mathcal{A}$, lest $\mathcal{A}$ notices that something is off and refuses to play, ruining both advantages in the process. (We will later see how to relax this.)

^a You will actually get an equality here rather than an inequality. However, since reductions have to be proven one direction at a time anyway, writing the statement as an inequality is helpful for identifying at a glance which direction the reduction is going: simply parse the $\leq$ as an arrow!

Hans: todo: write solution for a) and b)

Game Hopping.

Hans: copy-pasted from Sect. 2

For our security proofs, we will be working in the game-hopping paradigm of Shoup [Sho04]. The fundamental lemma of game hopping states that the distinguishing advantage of an adversary can be bounded in terms of their distinguishing advantage over several intermediary game “hops”.

Lemma 1. *For any adversary $\mathcal{A}$ tasked with distinguishing G_0 from G_n , and for any games $G_1, \dots, G_{n-1}$, there exist adversaries $(\mathcal{A}_1, \dots, \mathcal{A}_n)$ such that*

$$|\Pr[G_0(\mathcal{A}) : 1 \leftarrow \$ \mathcal{A}] - \Pr[G_n(\mathcal{A}) : 1 \leftarrow \$ \mathcal{A}]| \leq \sum_{i=1}^n |\Pr[G_{i-1}(\mathcal{A}) : 1 \leftarrow \$ \mathcal{A}_i] - \Pr[G_i(\mathcal{A}) : 1 \leftarrow \$ \mathcal{A}_i]|.$$

The lemma states that, if $\mathcal{A}$ were able to distinguish, then at least one of $\mathcal{A}_1, \dots, \mathcal{A}_n$ must be able to distinguish too—and vice versa, that if none of the $\mathcal{A}_i$ are able to distinguish, then neither is $\mathcal{A}$. (In fact, the $\mathcal{A}_i$ that achieve this will in essence be just copies of $\mathcal{A}$.)

The proof is remarkably simple, reproduced here for those who haven’t seen it.

Proof. For all i from 1 to $n-1$, insert $0 = \Pr[G_i(\mathcal{A}_i) : 1 \leftarrow \$ \mathcal{A}_i] - \Pr[G_i(\mathcal{A}_i) : 1 \leftarrow \$ \mathcal{A}_i]$ into the absolute value on the left hand side, then apply the union bound. $\square$

Hans: Add: **parse** keyword. (Allow **if parse** too? By having **parse** return a boolean value? Hmm, tricky.) And: **bad** abort convention.

Hans: Add: Adversaries are always assumed to halt in due time, even when the environment fails to match their expectation.

Identical-Until-Bad.

Hans: to-be-written; below are cut-and-pasted snippets from the old version of the U-proof.

Hans: Nice way to phrase the puncturing oracle later as just quantumly checking for bad events

Lemma 2 (Equivalent-Until-Bad). *xyz*

Let us use the law of total probability (and the triangle inequality again) to split into two distinguishing advantages, one for the case that **bad** is set to **true** and one for the case that **bad** remains false.

$$\begin{aligned} \text{Adv}_{\text{KEM}}^{\text{IND-CPA}}(\mathcal{A}) &\leq |\Pr[G_2(\mathcal{A}) : 1 \leftarrow \mathcal{A} \wedge \neg \text{bad}] - \Pr[G_3(\mathcal{A}) : 1 \leftarrow \mathcal{A} \wedge \neg \text{bad}]| \\ &\quad + |\Pr[G_2(\mathcal{A}) : 1 \leftarrow \mathcal{A} \wedge \text{bad}] - \Pr[G_3(\mathcal{A}) : 1 \leftarrow \mathcal{A} \wedge \text{bad}]|. \end{aligned}$$

These distinguishing advantages represent the probability that the adversary notices that we are exchanging the real key for a random key. Notice that **dPKE** being deterministic and correct imposes a one-to-one correspondence between messages and ciphertexts. This means that if **bad** is never set to **true**, then re-encryption never yielded c^* , which in turn implies that the random oracle was never called on x^* . Thanks to the uniform nature of random oracles, this means that both real and random K^* will look completely random from the point of view of $\mathcal{A}$. Hence, the two probabilities $\Pr[G_2(\mathcal{A}) : 1 \leftarrow \mathcal{A} \wedge \neg \text{bad}]$ and $\Pr[G_3(\mathcal{A}) : 1 \leftarrow \mathcal{A} \wedge \neg \text{bad}]$ are equal, and the $\neg \text{bad}$ part of the distinguishing advantage is zero:

$$\text{Adv}_{\text{KEM}}^{\text{IND-CPA}}(\mathcal{A}) \leq |\Pr[G_2(\mathcal{A}) : 1 \leftarrow \mathcal{A} \wedge \text{bad}] - \Pr[G_3(\mathcal{A}) : 1 \leftarrow \mathcal{A} \wedge \text{bad}]|.$$

We have reached the crux of the matter: The distinguishing advantage of $\mathcal{A}$ can be characterized as the difference in its probability of outputting 1 *and* in calling the random oracle on the correct x^* . But notice that 1) up until the point that **bad** is set to **true**, K^* will look completely random to $\mathcal{A}$, and 2) *after* **bad** is set to **true**, an adversary would easily be able to distinguish, e.g. by using **dPKE** to re-encrypt x^* and confirm that it gives the correct c^* , then comparing K^* against $\mathcal{H}(x^*)$. We therefore lose nothing of significance if we simply bound this distinguishing advantage by the probability that **bad** is set to **true**.

$$\text{Adv}_{\text{KEM}}^{\text{IND-CPA}}(\mathcal{A}) \leq \Pr[G_2(\mathcal{A}) : \text{bad}] = \Pr[G_3(\mathcal{A}) : \text{bad}],$$

where, again, the latter equality holds due to the view of $\mathcal{A}$ being equal in both games up until the point that **bad** is set to **true**, since random oracles are perfectly preimage-hiding.

2.4 Bit Security and Tightness

Hans: should probably briefly explain the t/ϵ convention for bit security, so that we can later use it to illustrate tightness losses

3 The Random Oracle Model (ROM)

This chapter introduces the concept of the ROM, by first explaining how the ROM models hash functions as random functions that are only available in oracle form. In the next chapter, we will then encounter a first application of the ROM: we will meet the Fujisaki–Okamoto (FO) transformation, a PKE-to-KEM recipe that became quite popular in the post-quantum-context.

3.1 Modelling Hash Functions as Random

As explained in Sect. 2.2, a hash function is a deterministic algorithm that takes arbitrary-length inputs and produces a fixed-length “fingerprint” of that input, such that re-evaluating the hash function on the same input produces the same fingerprint.

But is a hash function really a *function*? Well, since hash functions can take arbitrary-length inputs, they can at least not be *finite-domain* functions. Still, it is helpful to think of hash functions as an infinite *collection* of finite-domain functions $\{f_n\}$, with one f_n for each possible input length n . Let us for the sake of simplicity specialize to one such f_n . If the hash function produces fingerprints of (fixed) length m , then this is a function from the set $\{0, 1\}^n$ to $\{0, 1\}^m$, which is much simpler to reason about. If needed, we can then always go back to the full picture by considering the collection $\{f_n\}$ instead.

We mentioned that cryptographic hash function should produce random-looking fingerprints, and that it should further be hard to find two inputs with the same fingerprint, even though such collisions will definitely *exist* whenever n is larger than m (which it usually is), by the pigeonhole principle.

Indeed, in typical applications there must be a huge number of collisions. Consider the case of hashing a 10MB picture: then n is 80 million (since there are 8 bits in a byte), whereas a typical choice for the output length m might be 256. This leads to a truly astronomical number of colliding input values: the pigeonhole principle tells us that at least one of the hash function’s 2^m possible output values will map to at least

$$\frac{2^n}{2^m} = \frac{2^{80,000,000}}{2^{256}} = 2^{79,999,744}$$

different input values. For comparison, the number of atoms in the observable universe is estimated to be around $10^{80} \approx 2^{266}$.

And yet, actually *finding* even a single such collision can be very hard. To see this, assume that the hash function is a function f drawn uniformly at random from the set of all functions from $\{0, 1\}^n$ to $\{0, 1\}^m$; since f is uniform, we are guaranteed that there is no internal structure that can be exploited to find collisions, and no special inputs that have an a priori higher probability of being part of a collision than others. Hence, an attacker has no choice but to evaluate the function on different inputs until it finds two colliding outputs.

How many times will the attacker have to evaluate f before it is expected to find a collision? Let us calculate. First, let $\llbracket f(x_1) = f(x_2) \rrbracket$ be the predicate that takes the value 1 if $f(x_1) = f(x_2)$, and 0 otherwise. Let x_1 and x_2 two arbitrary but distinct input values. Since there are 2^m possible outputs, and each input were assigned an output uniformly at random (that’s what it means to draw a function at random), the probability that $\llbracket f(x_1) = f(x_2) \rrbracket = 1$ is exactly 2^{-m} . Hence, for any choice of distinct x_i and x_j , the expectation value is also

$$\mathbb{E}[\llbracket f(x_i) = f(x_j) \rrbracket] = 2^{-m},$$

with the expectation taken over the choice of f .

Now, let coll_q be the expected number of collisions observed after q evaluations of f on different inputs.

$$\text{coll}_q = \mathbb{E} \left[\sum_{i=1}^{q-1} \sum_{j=i+1}^q \llbracket f(x_i) = f(x_j) \rrbracket \right].$$

Here, the sum is taken over $j > i$ to avoid double-counting the pairs. A useful fact about expectation values is that, unlike (correlated) probabilities, they are linear; this may therefore be written

$$\text{coll}_q = \sum_{i=1}^{q-1} \sum_{j=i+1}^q \mathbb{E}[\llbracket f(x_i) = f(x_j) \rrbracket] = \sum_{i=1}^{q-1} \sum_{j=i+1}^q \frac{1}{2^m} = \frac{q(q-1)}{2} \cdot \frac{1}{2^m} \approx \frac{q^2}{2^{m+1}}.$$

<u>Initialize $\mathcal{H}$</u>	<u>Oracle $\mathcal{H}(x)$</u>	<u>Initialize $\mathcal{H}$</u>	<u>Oracle $\mathcal{H}(x)$</u>
01 $f \leftarrow \{f : \{0,1\}^n \rightarrow \{0,1\}^m\}$	03 return $f(x)$	01 $D := (\varepsilon, \varepsilon)$	03 if $D[x] = \perp : D[x] \leftarrow \{0,1\}^m$
02 return $\checkmark$		02 return $\checkmark$	04 return $D[x]$

Fig. 5. Left: A monolithic random oracle, sampling and storing a function f from n to m bits uniformly at random, then answering oracle calls by evaluating the f on the input. Since f is uniformly random, it is generally incompressible, so we have no choice but to store it as a truth table of size (length of input-output pair) $\times$ (number of inputs), giving a memory requirement of $(n + m) \cdot 2^n$ bits. Even if we limit the input length n to 128 bits, storing f would be well beyond the capability of any realistic computer.

Right: A lazy-sampled random oracle, maintaining a query database D of input/output tuples $(x, \mathcal{H}(x))$. Note how the size of the database grows as $(n + m) \cdot q$ bits, where q is the number of calls to $\mathcal{H}$ with differing inputs.

We wanted to know how large q must be for $\text{coll}_q \approx 1$. With an explicit formula in hand, we can find our answer by solving for q , then inserting $m = 256$:

$$\text{coll}_q \approx 1 \iff q \approx \sqrt{2^{m+1}} \iff q \approx \sqrt{2^{257}} \approx 2^{128}.$$

So despite the astronomical number of collisions, we would still have to evaluate a random function with length-256 outputs on roughly 2^{128} input values before we would expect to ever find one. This is comfortably beyond the capabilities of even the largest supercomputers.

Incidentally, the somewhat unintuitive fact that the expected number of collisions grows as the *square* of the number of potentially-colliding items is sometimes known as the “birthday paradox”, so named because of the surprising fact that, assuming everyone are assigned birthdays at random, it only takes a room of 23 people before the probability that two of them shares a birthday exceeds 50%.

3.2 Monolithic Random Oracles

The idealization that we just did when we studied a uniformly random function as a placeholder for a hash function, turned out to be very useful in analyzing the effort required to find a collision: the no-structure guarantee is what enabled us to explicitly calculate how many evaluations one would expect to need to find one.

However, this same no-structure guarantee also means that there is no way to actually *store* f more efficiently than to simply keep a truth table with all the input and output values. For a function from n to m bits, one such input-output pair will take up $n + m$ bits of memory, and there will be one such pairs for every possible input, so that the total memory requirement for storing f is $2^n \cdot (n + m)$. If the input length n is greater than 40, this would already be in the terabytes range, and n greater than 60 is beyond the capabilities of even the world’s largest supercomputers.

As we will see, hash function are very useful primitives when building cryptography. However, we need our cryptography to be *efficiently implementable*. This means that we can’t just put in a to be truly random f and expect that systems would actually be able to run it. Indeed, even just assuming that the hash function behaves like a random function for the sake of the security proof does not work, since we also need to be able to argue that any hypothetical attackers are themselves efficient (otherwise they could just break all our crypto via brute-force search). If we assume the hash function uniform, an efficient attacker would not even be able to *run* our algorithms, much less break them.

A common solution in theoretical computer science is to instead provide evaluations of the inefficient function as an *oracle*. Imagine a giant computer that lives in the sky, which is given as much resources as it needs to store and evaluate random functions. Everyone in the world are given input/output access to this *random oracle*, so that they can evaluate f by sending x to the oracle and receiving $f(x)$ in return. Crucially, given the same input x , everyone will see the same value for $f(x)$, which means that the random oracle can be used in place of a publicly known hash function such as SHA3.

The inner workings of such an oracle is shown in pseudocode in Fig. 5 (left): the “Initialize” method initializes our random oracle $\mathcal{H}$ by sampling f uniformly at random from the set of all functions from n to m bits. Thereafter, the “Oracle” interface takes inputs x and returns $f(x)$.

Note how $\mathcal{H}$ is implicitly taken to be *stateful*, in the sense that it has a persistent memory that stores f after initialization.

3.3 Simulation via Lazy-Sampling

The statefulness of $\mathcal{H}$ opens up for a very simple and efficient way to *simulate* a random function, known as lazy-sampling. The idea is simple: start with an empty function table; then, every time you receive an input x , check if your table already defines an output value for x ; if so, return that value, and otherwise draw an output value uniformly at random and store it together with x . The result is an oracle that behaves identically to a monolithic random oracle, i.e. to a uniformly random function. And yet, if this “lazy-sampled” random oracle receives at most q queries, then at most $q \cdot (n + m)$ bits is required to store the table – an exponential improvement over the memory requirements of the monolithic oracle.

The lazy-sampling procedure is shown in pseudocode in Fig. 5 (right): First, the database D is initialized with an empty tuple (recall that databases are lists of tuples); then the oracle interface, on input x , checks if D already contains x , and if not, samples uniformly from the set of output strings $\{0, 1\}^m$ and stores it in $D[x]$.

4 Example 1a: Classical Indistinguishability from the U-transform

We will now meet the Fujisaki–Okamoto (FO) transformation [FO99], a PKE-to-KEM recipe that became quite popular in the post-quantum-context – it is used as part of many post-quantum KEMs, including the now-standardized ML-KEM (Kyber) [NIS24].

We first focus on the part of FO that turns a deterministic PKE scheme into a KEM. Following the literature [HHK17], this part of FO will be called the U-transform. To have a simple ROM example, we will – for now – only prove that the obtained KEM achieves key indistinguishability against *passive* adversaries. This proof will already teach us several useful tricks you can play in the ROM. These tricks foreshadow and motivate the QROM tools that will be introduced in Sect. 7.

At a later point, we will also prove key indistinguishability against *active* adversaries. There also exists a second part of FO that turns probabilistic PKE schemes into ones that fit the U-transform’s requirements. (This part is called the T-transform, in case you’re curious, see App. D.)

In short, the U-transformation builds a KEM from a PKE scheme by sampling a random message, encrypting it, and hashing it (possibly together with the public key and ciphertext) to derive a symmetric key K . This is the transformation that will serve as our running example.⁷

Definition 5 (U-transform). *Let $\text{dPKE} = (\text{KeyGen}, \text{Enc}, \text{Dec})$ be a deterministic public-key encryption scheme, and let $H : \{0, 1\}^* \rightarrow \{0, 1\}^m$ be a hash function. Then the U-transform is the key-encapsulation mechanism $\text{U}[\text{PKE}, H] =: (\text{KeyGen}, \text{Encaps}^{\text{U}}, \text{Decaps}^{\text{U}})$, as defined in Fig. 6.*

$\text{Encaps}_{\text{pk}}^{\text{U}}()$	$\text{Decaps}_{\text{sk}}^{\text{U}}(c)$
01 $x \leftarrow \{0, 1\}^n$	05 $x' \leftarrow \text{Dec}_{\text{sk}}(c)$
02 $c \leftarrow \text{Enc}(\text{pk}, x)$	06 if $x' = \perp$:
03 $K \leftarrow H(x)$	07 return $\perp$
04 return (K, c)	08 return $H(x')$

Fig. 6. Algorithms of $\text{U}[\text{PKE}, H]$.

As explained in Sect. 2, One-way security under Chosen Plaintext Attacks, or OW-CPA, is really a minimal security requirement for PKE schemes, and should be considered too weak a security goal for most use-cases. Much preferred would be the stronger security requirement of *indistinguishability* under chosen-plaintext attacks, or IND-CPA.⁸

However, stronger security assumptions usually mean less efficient schemes, as the parameters must be increased to compensate for the increased attack surface. What’s worse, we are taking as our starting point a *deterministic* encryption scheme. Such a PKE scheme could *never* achieve indistinguishability, since the adversary can simply re-encrypt the two candidate messages, and check which of them encrypts to the challenge ciphertext.

And yet, taking our deterministic PKE scheme through the U-transform does in fact give us a KEM that achieves indistinguishability ... and the object that allows us to cross that chasm is precisely the *random oracle*.

The proof strategy will rely on the fact that our reduction can fully control the random oracle for the adversary, inspecting the queries before answering them via the lazy-sampling procedure from Sect. 3.3. Intuitively speaking, in order to successfully distinguish, the adversary will have to have called the random oracle on one of the messages used to derive the ephemeral key. The reduction can recognize this the moment it happens, by re-encrypting each message and checking if the encryption matches the challenge it was issued; if so, it wins at the one-wayness game by outputting that message.

To summarize, if dPKE satisfies one-wayness (Def. 3), then KEM satisfies indistinguishability (Def. 4), as captured in the following theorem.

⁷ When Hofheinz et al. [HHK17] first defined the T- and U-transforms, they actually defined several U-variants; Def. 5 corresponds to $\text{U}_m^{\perp}$ in their notation (message-only hashes, explicit decryption failures).

⁸ The standard security requirement for deployable KEMs is an even stronger one, namely indistinguishability under chosen-ciphertext attacks, or IND-CCA. As it turns out, the U-transform also achieves this, provided the re-encryption check from the T-transform is carried forward; this will be covered in later chapters (to appear).

Sect. 7 vs Sect. 4.2

add pointer to where

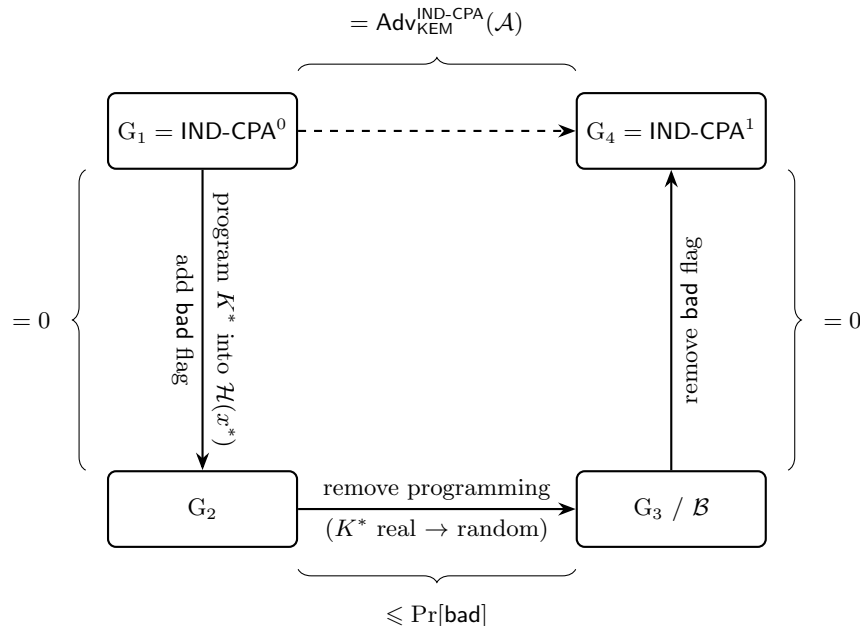


Fig. 7. Proof outline for Thm. 2. The arrows show the changes incurred by each game hop, and the braces show the difference in win probabilities between each game.

Theorem 2 (dPKE Is OW-CPA $\Rightarrow$ KEM Is IND-CPA). *Let dPKE be a deterministic PKE scheme, let $\mathcal{H}$ be a random oracle, and let the KEM = U[dPKE, $\mathcal{H}$]. Then there is a simple, fully black-box reduction $\mathcal{B}$, such that for every adversary $\mathcal{A}$,*

$$\text{Adv}_{\text{KEM}}^{\text{IND-CPA}}(\mathcal{A}^{\mathcal{H}}) \leq \text{Adv}_{\text{dPKE}}^{\text{OW-CPA}}(\mathcal{B}^{\mathcal{A}}).$$

The adversary $\mathcal{B}$ runs in the same time as $\mathcal{A}$, plus the overhead of lazy-sampling a random oracle.

Proof. We want to construct a $\mathcal{B}$ that exploits any winning strategy of $\mathcal{A}$ in such a way that if $\mathcal{A}$ would win at the indistinguishability game, then $\mathcal{B}$ actually wins at the one-wayness game. However, the constructed adversary $\mathcal{B}$ will not be able to simulate the real ($b = 0$) world of the KEM indistinguishability game, since it does not know which message the challenge ciphertext encrypts – that’s what it is trying to figure out! It can, however, easily simulate the random ($b = 1$) world, since here the symmetric key given to the adversary is simply uniformly sampled.

But in order to argue that $\mathcal{A}$ can not distinguish $\mathcal{B}$ ’s simulation from the real-world game, we need to alter the game in such a way that the move from real to random keys is completely hidden from the view of $\mathcal{A}$. We do this by inserting two intermediary games between the real-world and the random-world games, as illustrated in Fig. 7. These two middle games formalize the intuition that, thanks to the U-transform, the adversary will have to call the random oracle on the correct message in order to notice that it is not in the real world: If it does, then a boolean flag **bad** is set to true. Assuming the **bad** flag is never set to true, the game proceeds exactly as the real-world game, and thanks to ??, we know that the noticeable difference between these games is bounded exactly by the probability that **bad** is set to true. Then, *conditioned* on **bad** remaining false, we can exchange the real symmetric key for a randomly sampled one, and be sure that the adversary will never be able to notice.

The four games are given in pseudocode in Fig. 8, in a layout matching Fig. 7, and with changes between games highlighted. To make the argument formal, start by recalling the definition of a distinguishing advantage; we will use the “difference” version of distinguishing advantages, as it fits more cleanly with this “game hopping” style of proof. (For the sake of simplicity, we suppress the superscript dependencies $\mathcal{H}$ and $\mathcal{A}$ in the following.)

$$\text{Adv}_{\text{KEM}}^{\text{IND-CPA}}(\mathcal{A}) := |\Pr[\text{IND-CPA}_{\text{KEM}}(\mathcal{A}) : 1 \leftarrow \mathcal{A} \mid b = 0] - \Pr[\text{IND-CPA}_{\text{KEM}}(\mathcal{A}) : 1 \leftarrow \mathcal{A} \mid b = 1]|.$$

The game G_1 is just the real-world ($b = 0$) IND-CPA game, with the construction of KEM written out explicitly, together with a lazy-sampled random oracle. Similarly, G_4 is just the random-world ($b = 1$) IND-CPA game with the KEM construction explicit. We can therefore write

$$\text{Adv}_{\text{KEM}}^{\text{IND-CPA}}(\mathcal{A}) = |\Pr[G_1(\mathcal{A}) : 1 \leftarrow \mathcal{A}] - \Pr[G_4(\mathcal{A}) : 1 \leftarrow \mathcal{A}]|.$$

We are of course always free to “add zero” by adding and subtracting the same term, so this is equal to:

$$\begin{aligned} \text{Adv}_{\text{KEM}}^{\text{IND-CPA}}(\mathcal{A}) &= |\Pr[G_1(\mathcal{A}) : 1 \leftarrow \mathcal{A}] - \Pr[G_2(\mathcal{A}) : 1 \leftarrow \mathcal{A}] \\ &\quad + \Pr[G_2(\mathcal{A}) : 1 \leftarrow \mathcal{A}] - \Pr[G_3(\mathcal{A}) : 1 \leftarrow \mathcal{A}] \\ &\quad + \Pr[G_3(\mathcal{A}) : 1 \leftarrow \mathcal{A}] - \Pr[G_4(\mathcal{A}) : 1 \leftarrow \mathcal{A}]|. \end{aligned}$$

Applying the triangle inequality then gives us an upper-bound on the original distinguishing advantage as the sum of the distinguishing advantages between successive games:

$$\begin{aligned} \text{Adv}_{\text{KEM}}^{\text{IND-CPA}}(\mathcal{A}) &\leq |\Pr[G_1(\mathcal{A}) : 1 \leftarrow \mathcal{A}] - \Pr[G_2(\mathcal{A}) : 1 \leftarrow \mathcal{A}]| \\ &\quad + |\Pr[G_2(\mathcal{A}) : 1 \leftarrow \mathcal{A}] - \Pr[G_3(\mathcal{A}) : 1 \leftarrow \mathcal{A}]| \\ &\quad + |\Pr[G_3(\mathcal{A}) : 1 \leftarrow \mathcal{A}] - \Pr[G_4(\mathcal{A}) : 1 \leftarrow \mathcal{A}]|. \end{aligned}$$

Let us analyze each distinguishing advantage in turn. First, instead of deriving the key from the hash of the message as usual, G_2 instead samples K^* uniformly at random at the start of the game and gives it to $\mathcal{A}$. Then, if $\mathcal{H}$ is ever called with the encrypted message x^* , which *should* have been hashed to derive K^* at the start of the game, the oracle database is updated such that $D[x^*] = K^*$. Since the random oracle was lazy-sampled anyway, there is no way for $\mathcal{A}$ to tell if the internal oracle database was updated right away, or only when it is needed. From the point of view of $\mathcal{A}$, these games therefore behave identically, and

$$|\Pr[G_1(\mathcal{A}) : 1 \leftarrow \mathcal{A}] - \Pr[G_2(\mathcal{A}) : 1 \leftarrow \mathcal{A}]| = 0.$$

Two more things to notice about G_2 : First, the oracle interface could have simply done the above check by checking whether the input x matches the sampled message x^* ; instead, it takes the roundabout route of re-encrypting the input and checking whether the resulting ciphertext is equal to c^* . Since dPKE is deterministic (and by assumption perfectly correct), these checks lead to identical outcomes; but of course, only the latter check will be available to $\mathcal{B}$, who does not know x^* . Second, in addition to the “delayed programming” of the random oracle, the game introduces the **bad** flag in line 10. Initialized to **false**, this flag is only set to **true** in the case that the database is actually updated to hold the sampled K^* , or in other words, if the random oracle is called on the encrypted message x^* . Other than this, the flag has no effect on the behaviour of the game.

Compared to the changes introduced in G_2 , the change from G_2 to G_3 is deceptively simple: When the random oracle is called on x^* , after **bad** is set to **true**, do ... nothing. This is where G_2 would program the random oracle with K^* , thus ensuring that the random oracle remained consistent with the real world. Now we are removing this programming, which means that once **bad** is set to **true**, the game is *noticeably* inconsistent with the real-world game: If $\mathcal{A}$ was able to recover x^* from c^* , then they will quickly notice that $\mathcal{H}(x^*)$ does not evaluate to K^* (except if, by freak accident, the random value drawn by the lazy-sampling procedure happens to be exactly K^*).

Observe how G_2 and G_3 are by design exactly identical up until the point that **bad** is set to **true**. This means that we can use ?? to bound their difference as the probability that **bad** is set to **true** at the end of either game:

$$|\Pr[G_1(\mathcal{A}) : 1 \leftarrow \mathcal{A}] - \Pr[G_2(\mathcal{A}) : 1 \leftarrow \mathcal{A}]| \leq \Pr[G_2(\mathcal{A}) : \text{bad}] = \Pr[G_3(\mathcal{A}) : \text{bad}].$$

So, G_3 will be noticeably inconsistent with the real-world game if $\mathcal{A}$ ever calls $\mathcal{H}$ on x^* ; but what G_3 *will* look consistent with, is the case that $\mathcal{A}$ was given a uniformly random K , chosen completely independently of the ciphertext c^* . That’s the random-world game! The final hop to G_4 makes this manifest by removing the **bad** flag again, as well as the re-encryption check in the oracle interface. Since the flag itself had no affect on the operation of the game, this change is once again for free, and the result is exactly the random-world IND-CPA game instantiated with our construction.

$$|\Pr[G_3(\mathcal{A}) : 1 \leftarrow \mathcal{A}] - \Pr[G_4(\mathcal{A}) : 1 \leftarrow \mathcal{A}]| = 0.$$

Collecting, we see that we have found the bound

$$\text{Adv}_{\text{KEM}}^{\text{IND-CPA}}(\mathcal{A}) \leq 0 + \Pr[G_3(\mathcal{A}) : \text{bad}] + 0 = \Pr[G_3(\mathcal{A}) : \text{bad}].$$

All that remains is to bound $\Pr[G_3(\mathcal{A}) : \text{bad}]$, and this is where our reduction $\mathcal{B}$ enters the stage. The reduction is playing its own OW-CPA game, and is given black-box access to the adversary $\mathcal{A}$ (presumably

Game $G_1(\mathcal{A})$ 01 $(pk, sk) \leftarrow \text{KeyGen}()$ 02 $x^* \leftarrow \mathbb{S}\{0, 1\}^n$ 03 $c^* := \text{Enc}_{pk}(x^*)$ 04 $K^* \leftarrow \mathcal{H}(x^*)$ 05 $\hat{b} \leftarrow \mathcal{A}^{\mathcal{H}}(pk, K^*, c^*)$ 06 return $\hat{b}$	Oracle $\mathcal{H}(x)$ 07 if $D[x] = \perp$: 08 $D[x] \leftarrow \mathbb{S}\{0, 1\}^m$ 09 return $D[x]$	Game $G_4(\mathcal{A})$ 01 $(pk, sk) \leftarrow \text{KeyGen}()$ 02 $x^* \leftarrow \mathbb{S}\{0, 1\}^n$ 03 $c^* := \text{Enc}_{pk}(x^*)$ 04 $\boxed{K^* \leftarrow \mathbb{S}\{0, 1\}^m}$ 05 $\hat{b} \leftarrow \mathcal{A}^{\mathcal{H}}(pk, K^*, c^*)$ 06 output $\hat{b}$	Oracle $\mathcal{H}(x)$ 07 $e := \text{Enc}_{pk}(x)$ 08 if $e = c^*$: 09 $c := c^*$ 09 bad $:= \text{true}$ 10 if $D[x] = \perp$: 11 $D[x] \leftarrow \mathbb{S}\{0, 1\}^m$ 12 return $D[x]$
Game $G_2(\mathcal{A})$ 01 $\boxed{\text{bad} := \text{false}}$ 02 $(pk, sk) \leftarrow \text{KeyGen}()$ 03 $x^* \leftarrow \mathbb{S}\{0, 1\}^n$ 04 $c^* := \text{Enc}_{pk}(x^*)$ 05 $\boxed{K^* \leftarrow \mathcal{H}(x^*)}$ $\boxed{K^* \leftarrow \mathbb{S}\{0, 1\}^m}$ 06 $\hat{b} \leftarrow \mathcal{A}^{\mathcal{H}}(pk, K^*, c^*)$ 07 output $\hat{b}$	Oracle $\mathcal{H}(x)$ 08 $\boxed{c := \text{Enc}_{pk}(x)}$ 09 if $c = c^*$: 10 $\boxed{\text{bad} := \text{true}}$ 11 $\boxed{D[x] := K^*}$ 12 if $D[x] = \perp$: 13 $D[x] \leftarrow \mathbb{S}\{0, 1\}^m$ 14 return $D[x]$	Game $G_3(\mathcal{A})$ 01 bad $:= \text{false}$ 02 $(pk, sk) \leftarrow \text{KeyGen}()$ 03 $x^* \leftarrow \mathbb{S}\{0, 1\}^n$ 04 $c^* := \text{Enc}_{pk}(x^*)$ 05 $K^* \leftarrow \mathbb{S}\{0, 1\}^m$ 06 $\hat{b} \leftarrow \mathcal{A}^{\mathcal{H}}(pk, K^*, c^*)$ 07 output $\hat{b}$	Oracle $\mathcal{H}(x)$ 08 $c := \text{Enc}_{pk}(x)$ 09 if $c = c^*$: 10 $c := c^*$ 10 bad $:= \text{true}$ 11 $D[x] := K^*$ 12 if $D[x] = \perp$: 13 $D[x] \leftarrow \mathbb{S}\{0, 1\}^m$ 14 return $D[x]$

Fig. 8. The game hops in the proof of Thm. 2, following the arrows of Fig. 7. The dashed box highlights the difference between the real-world and the random-world indistinguishability games, the solid boxes highlight lines of code that were added between successive games, and strike-outs highlight lines of codes that were removed between games. Highlighted in red is the “bad event”, represented by the flag `bad` being set to `true`.

together with high praises of its code-breaking abilities). It wants to leverage the abilities of $\mathcal{A}$ to win at its own game, by simulating an environment for $\mathcal{A}$ and extracting its outputs. However, $\mathcal{B}$ will have to provide an *exact* simulation of the environment that $\mathcal{A}$ expects: if $\mathcal{A}$ notices anything is off, it can refuse to participate, for example by deliberately outputting wrong values, or halting prematurely.⁹

The problem is, $\mathcal{A}$ expects to be placed in the real- or random-world IND-CPA game with 50/50 probability, but $\mathcal{B}$ only knows how to simulate the random-world game. If $\mathcal{B}$ just simulated the random-world game for $\mathcal{A}$, then it would look to $\mathcal{A}$ like b were equal to 1 with 100% probability. Hence, it would seem that an adversary that simply ignores its input and outputs 1 would break $\mathcal{B}$ ’s strategy, since it would win at the simulated game 100% of the time while leaving $\mathcal{B}$ empty-handed.

But of course, $\mathcal{B}$ will not be simulating the random-world game G_4 , but rather the equivalent G_3 . Notice how the `bad` flag is first set to `true` the first time the encrypted value x^* is input to the random oracle. Notice further that, thanks to the re-encryption trick, this check can actually be implemented by $\mathcal{B}$, who knows pk and c^* but not x^* . A strategy reveals itself: Use the inputs pk and c^* given by the OW-CPA environment to simulate G_3 for $\mathcal{A}$, except that when G_3 would set `bad` to `true`, instead halt the simulation and output the value x^* that was just entered into the random oracle.

The pseudocode for our reduction $\mathcal{B}$ is given in Fig. 9. Since the event that $\mathcal{B}$ halts and outputs the correct value corresponds exactly to the event that G_3 would set `bad` to `true`, we have

$$\Pr[G_3(\mathcal{A}) : \text{bad}] = \text{Adv}_{\text{dPKE}}^{\text{OW-CPA}}(\mathcal{B}),$$

from which we may finally conclude that

$$\text{Adv}_{\text{KEM}}^{\text{IND-CPA}}(\mathcal{A}) \leq \Pr[G_3(\mathcal{A}) : \text{bad}] = \text{Adv}_{\text{dPKE}}^{\text{OW-CPA}}(\mathcal{B}).$$

In summary: *Either* $\mathcal{A}$ never calls the random oracle on the encrypted value x^* (`bad` = `false`), in which case the view of $\mathcal{A}$ is perfectly consistent with being in either of the real- or random-world games (advantage = 0), *or* $\mathcal{A}$ calls $\mathcal{H}$ on x^* and therefore *would* be able to distinguish (`bad` = `true`), in which case $\mathcal{B}$ aborts the simulation and outputs x^* , winning the one-wayness game. $\square$

⁹ To see how this admittedly anthropomorphizing description arises from the mathematical statement, recall that the theorem quantizes over the distinguishing advantage of *any* adversary that is placed in the IND-CPA environment. It does not (and can not) say anything about how these same adversaries behave in any *other* environments, and the “for all” quantifier means that we might as well assume the worst.

Reduction $\mathcal{B}(\text{pk}, c^*)$	if $\mathcal{A}$ calls $\mathcal{H}(x)$
01 $K^* \leftarrow \mathcal{K}$	04 $c := \text{Enc}_{\text{pk}}(x)$
02 $\hat{b} \leftarrow \mathcal{A}^{\mathcal{H}}(\text{pk}, K^*, c^*)$	05 if $c = c^*$:
03 output $\perp$	06 abort and output x
	07 if $D[x] = \perp$:
	08 $D[x] \leftarrow \{0, 1\}^m$
	09 return $D[x]$

Fig. 9. Pseudocode for our reduction $\mathcal{B}$; highlighted in green is the point at which $\mathcal{B}$ will abort the simulation and win the one-wayness game. If the simulation runs to completion without the correct value ever having been called to the random oracle, $\mathcal{B}$ returns $\perp$ to indicate “I give up”.

4.1 When You Have to Guess

What would happen if we started from a probabilistic rather than a deterministic PKE? Well, assuming the probability of getting the same ciphertext twice is small (as is usually the case), then the re-encryption check that the reduction $\mathcal{B}$ relied on to find the correct pre-image, would in all likelihood fail to identify x^* even if it were input. In other words, the reduction no longer has the ability to *recognize* x^* , and so it must resort to *guessing*.

The argument goes: If $\mathcal{A}$ is able to distinguish, then it must have queried the random oracle on input x^* at *some* point. Therefore, with $\mathcal{B}$ keeping track of the oracle database D as part of its simulation, x^* must be *in* D , even if $\mathcal{B}$ is unable to tell which one it is. So it does the simplest thing it can think of: After $\mathcal{A}$ halts and outputs a guess $\hat{b}$, $\mathcal{B}$ chooses one of the x -values from D uniformly at random, and returns that one as its guess. Then, if $\mathcal{A}$ made q queries to $\mathcal{H}$, and provided **bad** = **true** ($\mathcal{A}$ called $\mathcal{H}$ on x^*), $\mathcal{B}$ will win the one-wayness game with probability $1/q$. After a little manipulation of probabilities, we end up with the following bound.

Theorem 3 (PKE Is OW-CPA $\Rightarrow$ KEM Is IND-CPA). *Let PKE be a probabilistic PKE scheme, let $\mathcal{H}$ be a random oracle, and let the KEM = $\text{U}[\text{PKE}, \mathcal{H}]$. Then there is a simple, fully black-box reduction $\mathcal{B}$, such that for every adversary $\mathcal{A}$,*

$$\text{Adv}_{\text{KEM}}^{\text{IND-CPA}}(\mathcal{A}^{\mathcal{H}}) \leq q \cdot \text{Adv}_{\text{PKE}}^{\text{OW-CPA}}(\mathcal{B}^{\mathcal{A}}) .$$

The adversary $\mathcal{B}$ runs in the same time as $\mathcal{A}$, plus the overhead of lazy-sampling a random oracle.

In Thm. 2, the advantage of $\mathcal{A}$ was *tightly* bounded by the advantage of $\mathcal{B}$, in the sense that the advantage of $\mathcal{B}$ had a factor 1 in front of it. Now there is a factor q . What is the consequence?

Well, say we would like the advantage of $\mathcal{A}$ to be upper-bounded by 2^{-128} , a common target for security. If we start from a deterministic PKE scheme, we can rely on Thm. 2, and choose parameters such that the best-known attack against the one-wayness of dPKE is expected to succeed with probability 2^{-128} per time step – or equivalently, that the best attacks require at least 2^{128} time steps in expectation to succeed. However, with q counting the total number of hash function evaluations, q can realistically get quite large: It is for example commonly estimated that the hash function SHA3 has been evaluated more than 2^{80} times in the life-time operation the Bitcoin block-chain.

Let $2^{-\lambda}$ denote the new one-wayness advantage that our parameters should target, after compensating for the loss in tightness. Inserting $q \leq 2^{80}$ gives us:

$$\text{Adv}_{\text{KEM}}^{\text{IND-CPA}}(\mathcal{A}^{\mathcal{H}}) \leq q \cdot 2^{-\lambda} \leq 2^{80} \cdot 2^{-\lambda} = 2^{80-\lambda} \leq 2^{-128} \implies \lambda \geq 208 .$$

Therefore, if we want the distinguishing advantage to be bounded by 2^{-128} , we will need parameters such that the best attacks only achieve a one-way advantage of 2^{-208} . Perhaps not a dramatic increase – increasing the sizes of keys and ciphertexts accordingly should fairly easily be able to accomodate this – but experience shows that even a modest loss in efficiency may make our scheme look a lot less competitive to implementers than it just did, whether or not its competitors come bundled with tightness-respecting security proofs of their own!

We will repeatedly tackle the question of tightness over the course of this tutorial, as one of the central challenges of the QROM is to pin down exactly what level of tightness is achievable, and when.

Extractor $\mathcal{E}^{\mathcal{D}, \mathbb{1}^*}(z)$	if $\mathcal{D}$ calls $\mathcal{H}(x)$
01 $\hat{b} \leftarrow \mathcal{D}^{\mathcal{H}}(z)$	03 if $\mathbb{1}^*(x) = 1$:
02 output $\perp$	04 abort and output x
	05 if $\mathcal{D}[x] = \perp$:
	06 $\mathcal{D}[x] \leftarrow \{0, 1\}^m$
	07 return $\mathcal{D}[x]$

Fig. 10. The extractor $\mathcal{E}$ of Lemma 3. Here, the extractor $\mathcal{E}$ simulates $\mathcal{H}$ for the distinguisher $\mathcal{D}$; the extractor might of course just as well provide a simulation of $\mathcal{G}$.

Exercise 2. Prove Thm. 3.

Hint: Let $G_1 - G_4$ be as before; you only need to change how the bad check is performed. Write down pseudocode for a reduction $\mathcal{B}$ that implements the guessing strategy described above, and lower-bound its one-wayness advantage in terms of $\Pr[\text{bad}]$ and q .

4.2 One-Way to Hiding

Find-O2H. Thm. 2 may be about a specific construction (the U-transform), but the take-away message is quite general: If a reduction has control over the random oracle, and is able to recognize the correct answer when it sees it, we can use the random oracle to tightly reduce security from relying on a stronger distinguishing-style assumption, to instead rely on a weaker one-wayness assumption. We can generalize this observation in the following “one-way to hiding” lemma.¹⁰

Lemma 3 (Find-O2H, Classical). *Let $\mathcal{H}, \mathcal{G} : \{0, 1\}^n \rightarrow \{0, 1\}^m$ be random oracles such that for every $x \neq x^*$, $\mathcal{H}(x) = \mathcal{G}(x)$, whereas $\mathcal{H}(x^*)$ and $\mathcal{G}(x^*)$ are independently uniform. Let z be a value that depends arbitrarily on $\mathcal{H}$, $\mathcal{G}$, and x^* . Let $\mathbb{1}^*$ be an indicator oracle that on input x returns 1 if $x = x^*$, and otherwise returns 0.*

Then there is a simple, fully black-box extractor $\mathcal{E}$, such that for every distinguisher $\mathcal{D}$,

$$|\Pr[1 \leftarrow \mathcal{D}^{\mathcal{H}}(z)] - \Pr[1 \leftarrow \mathcal{D}^{\mathcal{G}}(z)]| \leq \Pr[x \leftarrow \mathcal{E}^{\mathcal{D}, \mathbb{1}^*}(z) : x = x^*].$$

It may not be clear at a glance how this lemma relates to the proof of Thm. 2. After all, there there was only one random oracle in play, while Lemma 3 has two. Also, Thm. 2 speaks of an adversary and a reduction, while Lemma 3 has a distinguisher and an extractors. How do they connect?

To see the connection, note how the difference between G_2 and G_3 is contained to the oracle interface only. In fact, the random oracle in G_2 and the random oracle in G_3 behave identically relative to $\mathcal{A}$ ’s input, *except* on the input value x^* , where it returns an independently uniform value instead of the expected K^* . If we let the random oracle in G_2 be identified with $\mathcal{H}$, and the random oracle in G_3 be $\mathcal{G}$, and further let $z = (\text{pk}, K^*, c^*)$, then we can identify $\mathcal{D}$ with $\mathcal{A}$, and things start to line up:

$$|\Pr[G_2(\mathcal{A}) : 1 \leftarrow \mathcal{A}] - \Pr[G_3(\mathcal{A}) : 1 \leftarrow \mathcal{A}]| = |\Pr[1 \leftarrow \mathcal{D}^{\mathcal{H}}(z)] - \Pr[1 \leftarrow \mathcal{D}^{\mathcal{G}}(z)]|.$$

As for the extractor, it is given in Fig. 10. It should look familiar: aside from the use of the “indicator oracle” $\mathbb{1}^*$, it is an almost exact adaptation of $\mathcal{B}$ from Fig. 9. The indicator oracle is simply a formalization of the statement that “the extractor is able to recognize the correct value”; we provide this ability in oracle form to make sure that our formalization only gives the extractor the ability to recognize correct values, such that it can be applied to any situation where this is the case. For example, in our case the $\mathbb{1}^*$ oracle can be locally computed from the input z , via the same re-encryption trick employed by $\mathcal{B}$.

Taken together, we can now provide simpler proof of the crucial hop from G_2 to G_3 in Thm. 2, by identifying $\mathcal{A}$ with $\mathcal{D}$, applying Lemma 3 to get the bound

$$|\Pr[G_2(\mathcal{A}) : 1 \leftarrow \mathcal{A}] - \Pr[G_3(\mathcal{A}) : 1 \leftarrow \mathcal{A}]| \leq \Pr[x \leftarrow \mathcal{E}^{\mathcal{D}, \mathbb{1}^*}(z) : x = x^*],$$

¹⁰ “Hiding” is another name for indistinguishability-style security, most often seen in the context of commitment schemes, which are usually required to be both “hiding” and “binding”.

then let our reduction $\mathcal{B}$ instead simulate the environment for $\mathcal{E}$ by sampling K^* , setting $z = (\text{pk}, K^*, c^*)$, use the re-encryption trick to respond to $\mathbb{1}^*$ queries, and output whatever value $\mathcal{E}$ outputs once it halts. Thus,

$$\Pr[x \leftarrow \mathcal{E}^{\mathcal{D}, \mathbb{1}^*}(z) : x = x^*] = \Pr[x \leftarrow \mathcal{B}^{\mathcal{A}, \mathcal{E}}(\text{pk}, c^*) : x = x^*] = \text{Adv}_{\text{KEM}}^{\text{OW-CPA}}(\mathcal{B}),$$

and the theorem is proven.

Guess-O2H. We can likewise define a “Guess-O2H” lemma, in correspondence with the guessing strategy of Thm. 3. Here the extractor is not given access to an indicator oracle, making the lemma applicable to situations where the reduction does not have the ability to recognize the correct answer. Hence, Guess-O2H has a higher level of applicability than Find-O2H, at the cost of sacrificing tightness.

Lemma 4 (Guess-O2H, Classical). *Let $\mathcal{H}, \mathcal{G} : \{0, 1\}^n \rightarrow \{0, 1\}^m$ be random oracles such that for every $x \neq x^*$, $\mathcal{H}(x) = \mathcal{G}(x)$, whereas $\mathcal{H}(x^*)$ and $\mathcal{G}(x^*)$ are independently uniform. Let z be a value that depends arbitrarily on $\mathcal{H}, \mathcal{G}$, and x^* .*

Then there is a simple, fully black-box extractor $\mathcal{E}$, such that for every distinguisher $\mathcal{D}$ that makes at most q calls to its random oracle,

$$|\Pr[1 \leftarrow \mathcal{D}^{\mathcal{H}}(z)] - \Pr[1 \leftarrow \mathcal{D}^{\mathcal{G}}(z)]| \leq q \cdot \Pr[x \leftarrow \mathcal{E}^{\mathcal{D}}(z) : x = x^*].$$

Exercise 3. Give a simpler proof of Thm. 3 by referencing Lemma 4.

Hint: Let $\mathcal{E}$ do the heavy lifting – no need to even specify how $\mathcal{E}$ works; it is enough that the lemma states that it exists. Then, all that is left for $\mathcal{B}$ to do is to provide $\mathcal{E}$ with its expected environment.

4.3 The Post-Quantum Setting – What Breaks?

Having seen the proofs of Thm. 2 and Thm. 3, the above “O2H lemmas” may seem almost too trivial to bother writing down. After all, the extractors in the lemmas do little more than lazy-sample a random oracle, then either return the correct value when input (if the extractor is able to recognize it), or output one of the queries at random (if not). With the analysis being a near-trivial manipulation of conditional probabilities, it is not clear that the modularity gained outweighs the cost of switching mental models from the behaviour of adversaries between games, to distinguishers trying to tell oracles apart.

However, the same can certainly not be said for *quantum* random oracles, since now, the input to the oracle can be a quantum superposition of any number of values at once. How does one recognize the correct input value in such a case? Is the “bad” event we used in our analysis even well-defined? Can you even *guess*, given that the no-cloning theorem bars you from copying down the input states?

This is where the very same O2H formalism that we introduced above earns its keep. In fact, the O2H lemmas turns out to be one the most useful toolkits we have, and they show up everywhere in the QROM literature. This is because they successfully shove the tricky quantum analysis “under the rug”, allowing non-quantum cryptographers to design their post-quantum game hops without having to worry that they are making a quantum fool of themselves. (Or at least, worry *less*.)

We will meet our first quantum O2H lemmas in Sect. 7, where we will apply them to lift Thm. 2 and Thm. 3 to the post-quantum setting. But before then, it is about time we brushed up on our quantum.

Part II

Enter Quantum

5 A Brief Quantum Primer

Quantum theory arose as a branch of physics, and so (naturally) much of the standard notational conventions are hand-wavy and inconsistently implemented across the literature. For the sake of conceptual clarity and self-containedness, we will therefore make an effort to specify all our working conventions, introducing the building blocks that make up quantum computing theory along the way. For more details, see the rightly celebrated introductory text by Nielsen and Chuang [NC10], or the equally-excellent book on quantum information theory by Mark Wilde [Wil17].

This primer is written with two very different audiences in mind: The beginner student, coming from a computer science and/or engineering background; and the experienced cryptographer, coming from a pure-maths background or similar. As a result, prose that may at times seem overly trivial or pedantic to one group of readers may serve to bring understanding to the other group of readers, so should you find a particular paragraph overly trivial or overly dense, feel free to skip ahead.

Regardless of your background, the best way to get a *feel* for the at-times esoteric rules of quantum logic is to play around with it. This chapter contains several exercises meant to serve this purpose; solutions for some (but not all) of the exercises are provided in App. G.

5.1 Quantum Physics and the Dirac Notation

In brief: Quantum states are complex-valued vectors in Hilbert space, which is to say, a vector space defined over functions. This Hilbert space may or may not be infinite-dimensional, though thankfully, we will only have to contend with the finite-dimensional case.¹¹ Quantum states are represented using the Dirac notation, in which a quantum state carrying the *label* ψ is written $|\psi\rangle$, its conjugate transpose is written $\langle\psi| := |\psi\rangle^\dagger := (|\psi\rangle^*)^T$, and its inner product with itself is written $\langle\psi|\psi\rangle := |\psi\rangle^\dagger |\psi\rangle = \|\psi\|^2$. Quantum states are unit-norm, and so we have that $\langle\psi|\psi\rangle = 1$. For two not-necessarily-equal quantum states $|\psi\rangle$ and $|\varphi\rangle$, their inner product $\langle\psi|\varphi\rangle$ is some complex number α whose norm $|\alpha|$ sits between zero and one, where “zero” means that the states are orthogonal, and “one” means that the states are identical (modulo an unobservable global phase factor $e^{i\theta}$).

This calculus is fine, but one may wonder, what *is* a quantum state? Meaning, what kind of mathematical object is it? The answer can be found in the statement that they live in a possibly-infinite-dimensional *Hilbert space*, which are vector spaces defined over *functions*. Here, the inner product between two functions f and g on the interval $[a, b]$ is defined as $\int_a^b f(x)^* g(x) dx$; this inner product defines a norm, as well as a notion of orthogonality, from which a vector space may be constructed. Quantum states, then, are the functions of this Hilbert space, arising as solutions to some differential equation describing the dynamics of the system. One such differential equation is the famous Schrödinger equation, which covers the non-relativistic, non-spin case. For fermions like electrons (which do have spin), we need the *Pauli equation*, and making quantum physics consistent with Einstein’s theory of special relativity further requires the *Dirac equation*.¹² These differential equations happen to describe the dynamics of waves, and so their solutions are often referred to as the “wave function” of the system. They take some input value corresponding to a possible observation, such as a particular position x , and returns an *amplitude* corresponding to that position; squaring this amplitude will then give you the probability that the particle is observed to be in position x .

If all of this sounds terribly complicated, then you may appreciate why physicists grew so fond of Dirac’s notation, which neatly sweeps the heavy machinery under a rug of neat, algebraic clarity.

Thankfully, things are much simpler in the discrete case: the Hilbert space is now finite, and so it can be expressed in terms of a finite set of basis “vectors” (i.e. finite-domain functions), and inner products are now defined in terms of sums rather than integrals. Each element of a “state vector” corresponds to one of the possible inputs to the function (i.e., one of the possible observable outcomes), enabling us to employ our toolbox from linear algebra to study them, leveraging the isomorphism between the set of functions from $\{0, 1\}^n$ to $\mathbb{C}$ and $\mathbb{C}^{2^n}$. This is the approach adopted in the quantum information sciences.

¹¹ For an example of an infinite-dimensional Hilbert space, consider the state $|p(x)\rangle$ describing the physical position $x \in [0, 1]$ of some particle p . Since space is not (or not known to be) discretized, observing this state may have infinitely many possible outcomes: one for every real number between 0 and 1. Hence, $|p(x)\rangle$ lives in the infinite-dimensional Hilbert space spanned by the infinite set of basis states $\{|p(x)\rangle\}_{x \in [0, 1]}$.

¹² Of course, combining quantum physics with *general* relativity constitutes *the* open problem of theoretical physics, and has given us a half-century of stagnant debate over which as-yet unsuccessful approach (string theory, loop quantum gravity, ...) is bound to one day grant us the coveted Theory of Everything. (For more on this, see the popular science section of your local book store.)

You may have noticed that if the input to our system is an n -length bitstring, then the corresponding quantum state vector will be of length 2^n , since vector elements correspond to the possible inputs to the function. From one point of view, this is no more mysterious than the fact that a discrete probability distribution can be expressed as an exponential-length vector, listing one probability for each element in its domain; in fact, you may find it helpful to think of quantum state vectors as a kind of generalized probability distribution.

Another view is that it is exactly in this exponentiality of state vectors that the power of quantum computation lies. This is because, unlike classical probabilities, amplitudes may *interfere* with each other, cancelling and reinforcing exactly like water waves meeting in a pond, opening a whole new world of calculation possibilities. And yet, at the end of the day, what we actually observe will still be a probability distribution, derived from the squares of the amplitudes. It is like nature gave us *some* limited access to the exponential nature of quantum physics, yet stopping short of letting us observe it directly, lest our power grow too great.¹³

5.2 Basis States

Single-Qubit Basis States. We are interested in modelling quantum computers. A quantum computer must, of course, have some quantum memory, which will in our case be a collection of qubits; let's say n of them. This memory can then at any point in time be said to be in some quantum state defined over n qubits.

A *qubit* is a physical system which, when observed, can be observed to take one of two possible values. In physical implementations, these observables may for example correspond to the spin of an electron (“up” and “down”), or to the two lowest energy states of an atom; in general they may be any two orthogonal states of an observable, where two quantum states are orthogonal if and only if there exists a measurement¹⁴ procedure that perfectly distinguishes them.

With exactly two possible outcomes, the qubit *state* lives in the two-dimensional Hilbert space $\mathbb{C}^2$, which we characterize by the basis states $|0\rangle$ and $|1\rangle$. It is worth emphasizing that in these states, the numbers “0” and “1” are *labels*, and do not say anything about the physical system in question other than that it supports (at least) two orthogonal quantum states, which we have the ability to distinguish, and which we *choose* to interpret as the symbols 0 and 1. In particular, the $|0\rangle$ state could certainly not be the zero *vector*, since all quantum states must be unit-length for probabilities to sum to one, and the zero vector has zero length.

Let $|b_1\rangle$ and $|b_2\rangle$ be two qubit basis states, where $b_1, b_2 \in \{0, 1\}$. Since these must be either identical or orthogonal, we have that

$$\langle b_1 | b_2 \rangle = \delta_{b_1 b_2} ,$$

where δ_{ij} is the Kronecker delta (equal to 1 if $i = j$ and 0 otherwise). Expressing the qubit states (which, remember, are functions from a binary set of observable values to a complex number with magnitude at most 1) as vectors, we see that we can express our qubit basis states as the column vectors

$$|0\rangle = \begin{bmatrix} 1 \\ 0 \end{bmatrix} , \quad |1\rangle = \begin{bmatrix} 0 \\ 1 \end{bmatrix} .$$

Then, their conjugate transposes become row vectors:

$$\begin{aligned} \langle 0| &= |0\rangle^\dagger = (|0\rangle^*)^T = [1 \ 0] , \\ \langle 1| &= |1\rangle^\dagger = (|1\rangle^*)^T = [0 \ 1] . \end{aligned}$$

As a sanity check, the inner product of two single-qubit basis states is then

$$\langle b_1 | b_2 \rangle = [b_1 \ 1 - b_1] \begin{bmatrix} b_2 \\ 1 - b_2 \end{bmatrix} = b_1 b_2 + (1 - b_1)(1 - b_2) = \delta_{b_1 b_2} ,$$

¹³ If we could directly access the ever-splitting branches of quantum states, and pick and choose the outcomes we would most like to see rather than hope that they appear by chance upon measurement, then we could easily solve NP-complete problems [Aar05].

¹⁴ The words “observation” and “measurement” describe the same procedure. We prefer “observation” when speaking informally, as the term captures the fact that you don’t need fancy lab equipment to collapse a quantum state – coming in contact with it in any way is enough – but we will follow the standard nomenclature of “measurement” when talking about the technical formalism.

Table 1. An overview of our notational conventions for quantum objects. Bold sans-serif denotes physical objects such as qubits and qubit registers, to distinguish them from the math italics of mathematical objects.

Object	Convention	Notation	Definition/example
Hilbert space	Calligraphic H	$\mathcal{H}$	Complete, complex inner-prod. space Single qubit $\Rightarrow \mathcal{H} \cong \mathbb{C}^2$
Hilbert space dimension	Uppercase Latin, middle of alphabet	N	$\in \mathbb{N}$ $\mathcal{H}$ finite-dim. $\Rightarrow \mathcal{H} \cong \mathbb{C}^N$
Qubit count	Lowercase Latin, middle of alphabet	n	$\in \mathbb{N}$ n qubits $\Rightarrow \mathcal{H} \cong (\mathbb{C}^2)^{\otimes n} \cong \mathbb{C}^{2^n}$
Unit sphere	Calligraphic S	$\mathcal{S}$	Unit-norm subset of inner-prod. space $\mathcal{S}(\mathcal{X}) = \{\mathbf{x} \in \mathcal{X} \mid \ \mathbf{x}\ = 1\}$
Bitstring	Lowercase Latin, end of alphabet	x	$\in \{0, 1\}^n$
Hadamard string	Bitstring with hat	$\hat{x}$	$\in \{+, -\}^n$ $ \hat{x}\rangle = 2^{-n} \sum_x (-1)^{\hat{x} \cdot x} x\rangle$
Quantum amplitude	Lowercase Greek, start of alphabet	α	$\in \mathbb{C}, \alpha \leq 1$
State label	Lowercase Greek, middle of alphabet	ψ	(any label is permissible)
Conjugate transpose	Dagger superscript	$A^\dagger$	$:= (A^*)^T$ Note: $(AB)^\dagger = B^\dagger A^\dagger$
Computational basis-state vector	“Ket” with bitstring label	$ x\rangle$	$= [0 \dots 0 \ 1 \ 0 \dots 0]^T$ single-unit column vector $\in \mathcal{S}(\mathcal{H})$
Quantum state vector	“Ket” with state label	$ \psi\rangle$	$= \sum_x \alpha_x x\rangle$ s.th. $\sum_x \alpha_x^* \alpha_x = 1$ unit-norm column vector $\in \mathcal{S}(\mathcal{H})$
Conjugate state vector	“Bra” with state label	$\langle\psi $	$:= \psi\rangle^\dagger$ unit-norm row vector $\in \mathcal{S}(\mathcal{H})$
Inner product	“Bra-ket”	$\langle\psi \varphi\rangle$	$\in \mathbb{C}, \langle\psi \varphi\rangle \leq 1$ $\langle x y\rangle = \delta_{xy} \Rightarrow \langle\psi \varphi\rangle = \sum_x \alpha_x^* \beta_x$
Unitary transformation	Uppercase Latin, late alphabet	U	$\in \mathbb{C}^{N \times N}$ s.th. $U^\dagger U = 1$ U is an n -qubit op. $\Rightarrow U \in \mathbb{C}^{2^n \times 2^n}$
Controlled unitary	C(unitary)	CU	$\in \mathbb{C}^{N \times N}$ n -qubit op. $U \Rightarrow (n+1)$ -qubit op. CU
Orthogonal projector	Uppercase Latin, middle-late alphabet	P	$\in \mathbb{C}^{N \times N}$ s.th. $P^2 = P = P^\dagger$ Single-state projector: $P_\psi = \psi\rangle\langle\psi $
PVM (Projector-valued Measure)	Calligraphic M	$\mathcal{M}$	Complete, orthogonal set $\{P_i\}_{i \in [k]}$ for k mutually-exclusive outcomes
Induced probability distribution	Distribution with measurement- and state-dependencies	$\mathcal{D}_\psi^\mathcal{M}$	$\mathcal{D}_\psi^\mathcal{M}(i) = \langle\psi P_i \psi\rangle$ $\mathcal{D}_\psi^{\text{comp}}(x) = \alpha_x ^2$
Post-measurement state	Sans-serif with measurement- and outcome-dependencies	$\text{Collapse}_i^\mathcal{M}(\psi\rangle)$	$= P_i \psi\rangle / \sqrt{\langle\psi P_i \psi\rangle}$ $\text{Collapse}_x^{\text{comp}}(\psi\rangle) = x\rangle$
Qubit	Lowercase Latin, bold sans-serif	q	Two-level physical system
Qubit state	State with index subscript	$ \psi\rangle_i$	$= \mathbf{q}_i\rangle \in \mathbb{C}^2$
Register	Uppercase Latin, bold sans-serif	R	List of qubits
Register state	State with register subscript	$ \psi\rangle_\mathbf{R}$	$= \mathbf{R}\rangle \in \mathbb{C}^{2^n}$ for n -qubit register
Physical equivalence	Equivalence relation	$ \psi\rangle \sim \varphi\rangle$	$\exists \theta \in [0, 2\pi)$ s.th. $ \psi\rangle = e^{i\theta} \varphi\rangle$
Physical state	Congruence class of state vectors	$[\psi\rangle]$	$:= \{ \varphi\rangle \mid \varphi\rangle \sim \psi\rangle\}$
The set of physical quantum states	Capital psi with Hilbert space dependency	$\Psi_\mathcal{H}$	$:= \mathcal{S}(\mathcal{H}) / \sim = \mathcal{S}(\mathcal{H}) / U(1)$ Unit sphere modulo the circle group

as desired.

If you are ever confused about a derivation using these Dirac “bra-kets”, it can be very useful to make the translation into vectors and matrices, and go through the explicit calculation using the familiar language of linear algebra. Table 1 provides a helpful overview of our conventions and their definitions.

Multi-Qubit Basis States. In general, two quantum systems (i.e. states describing distinct physical systems, such as two qubits) may be combined via the tensor product, for which we may use either of the following notational conventions:

$$|\psi\rangle \otimes |\varphi\rangle := |\psi\rangle |\varphi\rangle := |\psi, \varphi\rangle := |\psi\varphi\rangle .$$

If $|\psi\rangle$ describes an N -dimensional quantum system, and $|\varphi\rangle$ describes an M -dimensional quantum system, then the combined state $|\psi, \varphi\rangle$ describing the combined system will be NM -dimensional. (Note how dimensions are *multiplied* rather than *added*, as one might have naïvely expected; the addition happens in the *qubit count*, which lives in the exponent.)

Given a collection of n qubits $|b_i\rangle$, each initialized to one of the two basis states $|0\rangle$ or $|1\rangle$ (so that $b_i \in \{0, 1\}$), we may construct the combined state

$$\begin{aligned} |b_0\rangle \otimes |b_1\rangle \otimes \dots \otimes |b_{n-1}\rangle &:= |b_0\rangle |b_1\rangle \dots |b_{n-1}\rangle \\ &:= |b_0, b_1, \dots, b_{n-1}\rangle \\ &:= |b_0 b_1 \dots b_{n-1}\rangle . \end{aligned}$$

We will use any and all of the above conventions when denoting combined quantum states, choosing whichever one we deem clearest for the given context. For example, non-binary labels like ψ and φ will typically be separated by commas, while binary-valued labels will typically not be, to match the standard nomenclature for bit-string concatenation: $b_0 b_1 := b_0 \| b_1$.

Since each qubit is a two-dimensional quantum state, and there are n qubits, the combined state will have dimension $N := 2^n$, and the resulting Hilbert space will be characterized by the basis states

$$\{|i\rangle\}_{i \in [N]} = \{|b_0 b_1 \dots b_{n-1}\rangle\}_{b_0, \dots, b_{n-1} \in \{0, 1\}} .$$

Of course, any vector space permits an infinitude of basis choices by rotating the basis vectors; the above choice of basis is just the simplest one. We refer to this as the “computational basis”, since we have set our basis vectors up in such a way that they map to classical bitstrings. (We will meet another common choice of basis in Sect. 5.3 below.)

As indicated, the fact that the dimension of quantum states increases exponentially as the number of qubits increases sits at the core of quantum computers’ advantage over classical computers. An illustrative example is the simple three-qubit system: for two qubits, we only needed four basis states, but for three qubits this has now doubled to eight:

$$\begin{aligned} \{|i\rangle\}_{i \in [2^3]} &= \{|000\rangle, |001\rangle, |010\rangle, |011\rangle, |100\rangle, |101\rangle, |110\rangle, |111\rangle\} \\ &= \left\{ \begin{bmatrix} 1 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \end{bmatrix}, \begin{bmatrix} 0 \\ 1 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \end{bmatrix}, \begin{bmatrix} 0 \\ 0 \\ 1 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \end{bmatrix}, \begin{bmatrix} 0 \\ 0 \\ 0 \\ 1 \\ 0 \\ 0 \\ 0 \\ 0 \end{bmatrix}, \begin{bmatrix} 0 \\ 0 \\ 0 \\ 0 \\ 1 \\ 0 \\ 0 \\ 0 \end{bmatrix}, \begin{bmatrix} 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 1 \\ 0 \\ 0 \end{bmatrix}, \begin{bmatrix} 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 1 \\ 0 \end{bmatrix}, \begin{bmatrix} 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 1 \end{bmatrix} \right\} . \end{aligned}$$

As you can see, multi-qubit basis states also satisfy orthonormality:

$$\langle x | y \rangle = \delta_{xy} .$$

Finally, it will sometimes be useful to perform a repeated tensor product over identical states, for instance when initializing n qubits to the $|0\rangle$ state; this can be written in either of the following ways:

$$\bigotimes_{i=0}^{n-1} |0\rangle := |0\rangle^{\otimes n} := |0^n\rangle := \underbrace{|0 \dots 0\rangle}_n .$$

5.3 Uniform Superposition States

Of course, this is *quantum* computing, and so we are interested in qubit states which are more than just classical bitstrings written in kets. The unique thing about qubits is that they can be in *superpositions* of $|0\rangle$ and $|1\rangle$, usually popularized to the qubit somehow being “both 0 and 1 at once”, before inevitably failing to explain what makes this situation any different from simply being ignorant about the qubit’s “true” value until we observe it, which is just stochastic computing. Let us demonstrate.

Plus and Minus. For a single qubit, the simplest superposition states are common enough to warrant their own names: they are the “plus” and “minus” states

$$|+\rangle := \frac{|0\rangle + |1\rangle}{\sqrt{2}}, \quad |-\rangle := \frac{|0\rangle - |1\rangle}{\sqrt{2}}.$$

If you were to observe these states directly, you would see 0 or 1 with equal probability (since $\Pr[\text{observed } 0] = \Pr[\text{observed } 1] = (1/\sqrt{2})^2 = 1/2$), and if this were all, then these states would be no different from the probability distribution of a coin-toss.

What makes quantum superpositions different from classical stochastics is that despite both states being uniform superpositions of 0 and 1, they can be *perfectly distinguished from each other*. This is because, unlike the classical world, we are no longer stuck with only computational-basis measurements.

One of the elementary quantum operations that we can apply to a qubit is the *Hadamard gate* H , and it applies the self-inverse map

$$|0\rangle \xrightarrow{H} |+\rangle \xrightarrow{H} |0\rangle, \quad |1\rangle \xrightarrow{H} |-\rangle \xrightarrow{H} |1\rangle.$$

So, if we are given a qubit, and we are told that it is either in the state $|+\rangle$ or in the state $|-\rangle$, then measuring the qubit directly (in the computational basis) would give us no information about which of the two it was in. But if we *first* apply H and *then* measure it, then we learn with certainty which state it was in, since now observing 0 can only happen if we were given the state $|+\rangle$, and observing 1 can only happen if we were given the state $|-\rangle$.

We will refer to the procedure of first applying a Hadamard gate to a qubit and then measuring it in the computational basis as a “Hadamard-basis measurement”, for reasons that will become clear shortly.

The Hadamard Transformation. For n qubits, the uniform superposition over every length- n bit-string is the state

$$\frac{|0\dots 00\rangle + |0\dots 01\rangle + \dots + |1\dots 11\rangle}{\sqrt{2^n}} = \sum_{x \in \{0,1\}^n} \frac{1}{\sqrt{2^n}} |x\rangle.$$

As it happens, this is equal to the state

$$|+\rangle^{\otimes n} := |+\rangle |+\rangle \dots |+\rangle = \left(\frac{|0\rangle + |1\rangle}{\sqrt{2}} \right) \left(\frac{|0\rangle + |1\rangle}{\sqrt{2}} \right) \dots \left(\frac{|0\rangle + |1\rangle}{\sqrt{2}} \right),$$

since multiplying out the paranthesis gives a sum over all possible bitstrings. This is exactly the state that we will get if we have the apply the gate H to each of n qubits in the $|0\rangle$ state:

$$|0^n\rangle \xrightarrow{H} |+\rangle^n \xrightarrow{H} |0^n\rangle.$$

We will refer to the procedure of applying the H operation to every qubit of a system as the *Hadamard transformation*.

What happens do we get if we start with computational basis states other than 0^n ? Say we have five qubits, arranged to represent the decimal 21:

$$|[21]_2\rangle = |10101\rangle.$$

After applying H gates to each qubit, we get the state

$$|-\rangle |+\rangle |-\rangle |+\rangle |-\rangle = \left(\frac{|0\rangle - |1\rangle}{\sqrt{2}} \right) \left(\frac{|0\rangle + |1\rangle}{\sqrt{2}} \right) \left(\frac{|0\rangle - |1\rangle}{\sqrt{2}} \right) \left(\frac{|0\rangle + |1\rangle}{\sqrt{2}} \right) \left(\frac{|0\rangle - |1\rangle}{\sqrt{2}} \right) = \sum_{x \in \{0,1\}^5} \frac{(-1)^{x \cdot [21]_2}}{\sqrt{2^5}} |x\rangle,$$

where $x \cdot [21]_2$ denotes the inner product of the two bitstrings modulo 2 (so for example, $[5]_2 \cdot [21]_2$ evaluates to $00101 \cdot 10101 = 0 \cdot 1 \oplus 0 \cdot 0 \oplus 1 \cdot 1 \oplus 0 \cdot 0 \oplus 1 \cdot 1 = 0$, so the branch $|5\rangle$ will have a positive phase). This is also a uniform superposition, and measuring it directly would again just give the uniform distribution. Yet nothing is lost: all the information about the original number 21 is preserved in the *relative phases* of the different branches of the superposition, and can be recovered exactly by applying the Hadamard transformation a second time. Generalizing, we can write the effect of the Hadamard transformation on a computational-basis state as

$$|x\rangle \xrightarrow{H^{\otimes n}} \sum_{y \in \{0,1\}^n} \frac{(-1)^{x \cdot y}}{\sqrt{2^n}} |y\rangle.$$

The multi-qubit Hadamard-basis measurement is then the procedure that first applies the Hadamard transformation to the state, then measures it in the computational basis.

The Hadamard Basis. Actually, we don't have to imagine explicitly applying the H gates to our qubits before measuring them, as Hadamard-basis measurements are perfectly well-defined on their own merits, as one choice among a continuous infinity of measurements we could choose to apply. Perhaps it is better, then, to view the Hadamard basis as a sort of “dual domain” under which we can view our qubit, with two possible questions we can ask our state: Either “Are you $|0\rangle$ or $|1\rangle$?”, or “Are you $|+\rangle$ or $|-\rangle$?” Either way, the qubit will provide a definite answer, after which its state will “collapse” to the answer you got.

This is a point worth internalizing, so let us repeat it: *The set of possible post-measurement states changes to match the question you chose to ask.* If you ask “Are you 0 or 1?”, then the post-measurement state will be $|0\rangle$ or $|1\rangle$; if you ask “Are you + or −”, then the post-measurement state will be $|+\rangle$ or $|-\rangle$. So although at one level, the “collapse” of the quantum state is just the quantum version of how one would use Bayes' theorem to update a prior probability distribution to a marginal distribution conditioned on the outcome, at another level, it represents the deepest and strangest mystery in all of quantum physics. The grand mystery of quantum physics really comes down to this one fact, that the outcome statistics and the post-observation state both depend on *which measurement you chose to apply*, and it has led to a century of heated philosophical debate over how quantum theory should be interpreted. Ask five physicists for their opinion on the matter, and you are likely to get five different answers!¹⁵

What about a question like “Are you $|0\rangle$ or are you $|+\rangle$?” Under our notion of measurement, this question is not well-defined, since $|0\rangle$ and $|+\rangle$ are not orthogonal states and therefore do not lead to mutually-exclusive outcomes. More on this in Sect. 5.7, but for now, think of a measurement as always being defined in terms of a set of mutually-exclusive outcomes.

Of course, any vector space has a continuous infinity of bases to choose from, each of which define a binary “question” we can pose to our qubit, and the Hadamard basis is just one particularly nice choice: geometrically, it is just the basis rotated by 45° relative to the original one, as illustrated in Fig. 22. To appreciate how this makes for a perspective “dual” to the computational basis, take a look at the following identities:

$$|+\rangle = \frac{|0\rangle + |1\rangle}{\sqrt{2}}, \quad |-\rangle = \frac{|0\rangle - |1\rangle}{\sqrt{2}} \quad \Longleftrightarrow \quad |0\rangle = \frac{|+\rangle + |-\rangle}{\sqrt{2}}, \quad |1\rangle = \frac{|+\rangle - |-\rangle}{\sqrt{2}}.$$

Notice the symmetry: The computational basis states $|0\rangle$ and $|1\rangle$ are to the Hadamard basis *exactly* what the plus state and minus states are to the computational basis; they really are *dual views*, which can be translated between via the map H .¹⁶

¹⁵ The term “collapse” that we used above is typically associated with the rather conservative *Copenhagen interpretation*, championed by Niels Bohr, and commonly characterized as the “shut-up-and-calculate” approach, since it simply posits that “such questions are disallowed”. Another popular interpretation is the many-worlds interpretation due to Hugh Everett, favoured by father of quantum computing theory David Deutsch among others. Instead of the quantum state “collapsing” to a “classical” state, this interpretation says that both terms are ill-defined, and that the universe should instead be understood to be nothing but one big quantum superposition over all the worlds that could have possibly arisen from the initial conditions of the universe. (After all, we the observers are made of quantum particles too, so should we not then also be considered part of the quantum state?) If you're curious to learn more about the various interpretations of quantum mechanics, and the strengths and weaknesses of each attempt at a consistent resolution, Carlo Rovelli's book *Helgoland* provides an excellent overview.

¹⁶ The notion of a “dual domain” will sound familiar to anyone who has ever come across Fourier analysis, and not without reason: The Hadamard map is exactly the single-qubit Fourier transformation.

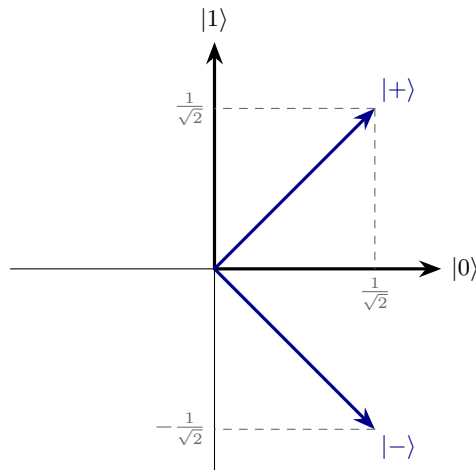


Fig. 11. The computational basis $\{|0\rangle, |1\rangle\}$ and the Hadamard basis $\{|+\rangle, |-\rangle\}$ describe the same qubit, seen along axes turned by 45° relative to each other.

How does this duality generalize to multi-qubit states? We had that

$$|+\rangle^n = \frac{|0\dots 00\rangle + |0\dots 01\rangle + \dots + |1\dots 11\rangle}{\sqrt{2^n}} = \sum_{x \in \{0,1\}^n} \frac{1}{\sqrt{2^n}} |x\rangle$$

With $\{|0\rangle, |1\rangle\}$ and $\{|+\rangle, |-\rangle\}$ being exact duals of each other, then it must also be true that

$$|0\rangle^n = \frac{|+\dots +\rangle + |+\dots +-\rangle + \dots + |-\dots -\rangle}{\sqrt{2^n}} = \sum_{\hat{x} \in \{+,-\}^n} \frac{1}{\sqrt{2^n}} |\hat{x}\rangle.$$

Here, we have introduced a new kind of bitstring, one that is defined over the alphabet $\{+, -\}$ rather than the alphabet $\{0, 1\}$. We will use the convention that whenever a number or variable is represented with a hat, then it is understood to be a *Hadamard-bitstring* like above. For example, whereas we previously represented the decimal 21 by the bitstring $[21]_2 = 10101$, writing $\hat{21}$ would imply that it is represented by the Hadamard-bitstring $[\hat{21}]_2 = -+-+ -$.

As we might have anticipated, just like the all-zero state $|0\rangle^n$ maps to the decimal state $|0\rangle$, the uniform superposition $|+\rangle^n$ then maps to the Hadamard-zero state $|\hat{0}\rangle$. Thus, we recover our dual views on the quantum state space, which we can translate between as desired via the Hadamard transformation. Defining $\hat{\mathbb{Z}}_N := \{\hat{x} \mid x \in \mathbb{Z}_N\}$, and letting the dual-alphabet inner bitstring product $\hat{x} \cdot x$ simply treat $\{+, -\}$ as if it were $\{0, 1\}$, we can then express both directions of the Hadamard transformation in purely decimal notation:

$$|\hat{x}\rangle = \sum_{x \in \mathbb{Z}_N} \frac{(-1)^{\hat{x} \cdot x}}{\sqrt{2^n}} |x\rangle, \quad |x\rangle = \sum_{\hat{x} \in \hat{\mathbb{Z}}_N} \frac{(-1)^{x \cdot \hat{x}}}{\sqrt{2^n}} |\hat{x}\rangle.$$

The above notation will at times allow us to greatly simplify our notation, making arguments clearer. Consider for example the state

$$|\psi\rangle = -\frac{1}{\sqrt{2^n}} |0\rangle + \sum_{x \in \mathbb{Z}_N \setminus \{0\}} \frac{1}{\sqrt{2^n}} |x\rangle,$$

i.e. the uniform superposition state that singles out the $|0\rangle$ state (implicitly understood to be the $|0\rangle^n$ state since we are using decimal notation) and applies a negative phase to it. Computing the Hadamard transformation of such a state can lead to a mess of double sums, but using the above notation, not only is applying the transformation as simple as adding hats everywhere,

$$-\frac{1}{\sqrt{2^n}} |0\rangle + \sum_{x \in \mathbb{Z}_N \setminus \{0\}} \frac{1}{\sqrt{2^n}} |x\rangle \xrightarrow{H^{\otimes n}} -\frac{1}{\sqrt{2^n}} |\hat{0}\rangle + \sum_{\hat{x} \in \hat{\mathbb{Z}}_N \setminus \{\hat{0}\}} \frac{1}{\sqrt{2^n}} |\hat{x}\rangle,$$

but we can then even choose to write out the state using a mix of notations, “unpacking” the complex-phase term for closer study while keeping the real-valued terms around in their Hadamard-bitstring

form:

$$-\frac{1}{\sqrt{2^n}}|\hat{0}\rangle + \sum_{\hat{x} \in \hat{\mathbb{Z}}_N \setminus \{\hat{0}\}} \frac{1}{\sqrt{2^n}}|\hat{x}\rangle = \sum_{x \in \mathbb{Z}_N} -\frac{1}{2^n}|x\rangle + \sum_{\hat{x} \in \hat{\mathbb{Z}}_N \setminus \{\hat{0}\}} \frac{1}{\sqrt{2^n}}|\hat{x}\rangle.$$

5.4 General Superposition States

Greek vs. Latin Letters. To distinguish computational basis states from more general quantum states, we will use Latin letters like x, y, z to represent classical binary strings. Thus, a state $|x\rangle$ will always be understood to be a computational basis state. Whenever we are talking specifically about single-qubit computational basis states, we will use b to represent a single bit, and write $|b\rangle$.

General quantum states, on the other hand, are traditionally labelled by letters taken from the middle of the Greek alphabet, such as ψ, φ, ξ , etc. We will follow this tradition whenever there isn't an obvious descriptive label to be used for the state.

Quantum *amplitudes*, meanwhile, are traditionally represented with letters from the start of the Greek alphabet, such as α or β , and the amplitude corresponding to a particular basis state $|x\rangle$ is typically written α_x .

Expanding a State in Terms of a Basis. A general single-qubit state may be written in terms of the computational basis states as follows:

$$|\psi\rangle = \alpha_0|0\rangle + \alpha_1|1\rangle,$$

where the amplitudes α_b are complex numbers satisfying

$$|\alpha_0|^2 + |\alpha_1|^2 := \alpha_0^* \alpha_0 + \alpha_1^* \alpha_1 = 1.$$

As already covered, whenever α_0 and α_1 are both non-zero, the qubit is in a superposition over the 0 and 1 values. If we observe the state via a computational-basis measurement, the probability of outcome 0 resp. 1 is given by the respective amplitude squared:

$$\Pr[\text{outcome } b] = |\alpha_b|^2 = \alpha_b^* \alpha_b = |\langle\psi|b\rangle|^2.$$

Likewise, a general, n -qubit quantum state may be written in terms of the computational basis states:

$$|\psi\rangle = \alpha_0|0\dots 00\rangle + \alpha_1|0\dots 01\rangle + \dots + \alpha_{N-1}|1\dots 11\rangle = \alpha_0|0\rangle + \alpha_1|1\rangle + \dots + \alpha_{N-1}|N-1\rangle = \sum_{x \in \mathbb{Z}_N} \alpha_x |x\rangle,$$

where we have again made the switch to decimal notation.

When expanded over the basis states, the inner product of two multi-qubit quantum states $|\psi\rangle = \sum_{x \in \mathbb{Z}_N} \alpha_x |x\rangle$ and $|\varphi\rangle = \sum_{y \in \mathbb{Z}_N} \beta_y |y\rangle$ may be written

$$\langle\psi|\varphi\rangle = \sum_{x, y \in \mathbb{Z}_N} \alpha_x^* \beta_y \langle x|y\rangle = \sum_{x, y \in \mathbb{Z}_N} \alpha_x^* \beta_y \delta_{xy} = \sum_{x \in \mathbb{Z}_N} \alpha_x^* \beta_x.$$

We have recovered the inner product of two complex-valued vectors, which is just the usual vector inner product with an extra complex conjugation. We may therefore adopt the usual operational interpretation of the inner product, in which two unit vectors are orthogonal when their inner product is 0 and parallel if it is 1. Together with the requirement that quantum states have unit norm, this means that if two quantum states have an inner product of 1, then they are actually *identical* ... up to an unobservable phase factor.

Exercise 4. Let $|\psi\rangle$ be a quantum state, and let $|\varphi\rangle = e^{i\theta}|\psi\rangle$, where the complex phase θ is a real number between 0 and 2π . Show that no measurement can distinguish $|\psi\rangle$ from $|\varphi\rangle$.
Hint: Try expanding $|\psi\rangle$ in terms of an arbitrary orthonormal basis, then compare the probabilities of observing each outcome.

The Set of Quantum States. Until now, we have used that quantum states $|\psi\rangle$ live in the Hilbert space $\mathcal{H}$, which is correct but not very precise: The Hilbert space of N -dimensional complex vectors include vectors of arbitrary norm, whereas our quantum (pure) states are always unit-length.

In most cases, this is not really a problem, but what if we want to quantify over all possible quantum states for our system? Writing $\forall |\psi\rangle \in \mathcal{H}$ won't cut it, since this would seem to imply that $|\psi\rangle$ can take any norm. We might instead want to write something like

$$\forall |\psi\rangle \in \mathcal{S}(\mathcal{H}) := \{|\psi\rangle \in \mathcal{H} \mid \|\psi\rangle\| = 1\},$$

i.e. quantify over the unit sphere of $\mathcal{H}$. However, you saw in Exercise 4 that quantum states that only differ by a global phase factor cannot be distinguished by observation, so quantifying over the unit sphere would lead to double-counting – again, not a problem most of the time, but not very satisfactory either. Can we do better?

What we really want is a set that contains exactly *one* element per observationally-distinct quantum state. The standard way to construct such a set is to invoke the machinery of equivalence classes. Let $\sim$ be the equivalence relation given by

$$|\psi\rangle \sim |\varphi\rangle \iff \exists \theta \in [0, 2\pi) \text{ s.t. } |\psi\rangle = e^{i\theta} |\varphi\rangle,$$

i.e. $|\psi\rangle \sim |\varphi\rangle$ iff the two states only differ by a global phase factor. Then let $[[\psi]]$ denote the corresponding equivalence class:

$$[[\psi]] := \{|\varphi\rangle \mid |\varphi\rangle \sim |\psi\rangle\},$$

i.e. the set of states observationally equivalent to $|\psi\rangle$. Then the set of observationally-distinct quantum states is captured by the set of equivalence classes of states on the unit sphere of the Hilbert space $\mathcal{H}$:

$$\Psi_{\mathcal{H}} := \mathcal{S}(\mathcal{H})/\sim = \{[[\psi]] \mid |\psi\rangle \in \mathcal{S}(\mathcal{H})\}.$$

We may then write $\forall |\psi\rangle \in \Psi_{\mathcal{H}}$, and be sure that we only quantify over each observably-distinct state only once, leaving implicit the fact that we are actually quantifying over *equivalence classes* of objects rather than the objects themselves, which is well-defined exactly because “differ by at most a global phase” and “cannot be distinguished by any measurement” perfectly imply each other.

5.5 Qubit Registers

If the qubit a quantum state is describing is not clear from context, we may use subscripts to specify which qubit it describes: For example, if $|\psi\rangle$ describes the state of the i 'th qubit, we may write $|\psi\rangle_i$.

To see how this is useful, say we have n qubits all initialized to $|0\rangle$, and say we want to put the i 'th qubit in some single-qubit state $|\psi\rangle$ and the j 'th qubit in another state $|\varphi\rangle$. We could write this operation as

$$|0\rangle^{\otimes n} \longrightarrow |0\rangle^{\otimes i} |\psi\rangle |0\rangle^{\otimes (j-i)} |\varphi\rangle |0\rangle^{\otimes (n-j)},$$

but indexing by qubit number allows us to express the same operation much more concisely:

$$|0\rangle_i |0\rangle_j \longrightarrow |\psi\rangle_i |\varphi\rangle_j,$$

where by convention the operation acts like the identity on the state of any qubit not listed.

Given a collection of n qubits, we may divide them into *registers*. Qubit registers are simply lists of qubits, so if $\mathbf{q}_1, \mathbf{q}_2, \mathbf{q}_3$ are three qubits, then collecting them into a list would define a qubit register $\mathbf{R} = (\mathbf{q}_1, \mathbf{q}_2, \mathbf{q}_3)$. Here and elsewhere, we use boldface to indicate that an object such as a list (and later, oracles) should be considered *quantum*.¹⁷ If the register $\mathbf{R}$ is in the quantum state $|\varphi\rangle$, we may specify that $|\psi\rangle$ is a description of $\mathbf{R}$ by writing $|\psi\rangle_{\mathbf{R}}$, or even $|\mathbf{R}\rangle$.

¹⁷ And what kind of mathematical objects are the qubits $\mathbf{q}_i$? Well, this is where our formal treatment bottoms out: all we can say is that $\mathbf{q}_i \in \{\text{physical systems that admit a two-level quantum-mechanical description}\}$.

5.6 Unitaries

Since we are working over finite-dimensional Hilbert spaces, operations that evolve our quantum state vectors may be represented as $N \times N$ matrices. What properties should they have?

Clearly, any operation that we perform on a quantum state should yield another valid quantum state. Since quantum states must be unit-norm, this means that any operation we perform on them (other than measuring – we will get back to that in Sect. 5.7) must be norm-preserving. This fact turns out to be sufficient to restrict the space of allowed operations to the unitary matrices, i.e. the set of invertible square matrices U that satisfy $U^\dagger U = \mathbb{1}$, or in other words that $U^{-1} = U^\dagger$.

Exercise 5. Show that a square matrix U acting on a state vector $|\psi\rangle$ is norm-preserving if and only if it is unitary.

Hint: You will need the identities $\|\psi\| = \sqrt{\langle\psi|\psi\rangle}$ and $(U|\psi\rangle)^\dagger = \langle\psi|U^\dagger$.

Whenever which qubit or qubit register an operation is being applied to is not sufficiently clear from context, we can specify this via subscripts. For example, U_i would denote a single-qubit operation applied to qubit i , $U_{i,j}$ would denote a two-qubit operation applied to qubit i and j (where generally $U_{i,j} \neq U_{j,i}$), and $U_{\mathbf{R}}$ would denote an operation applied to the multi-qubit register $\mathbf{R}$.

Here are some common single-qubit unitaries, or qubit *gates*, in reference to the logical gates of Boolean circuits:

$$X = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}, \quad Y = \begin{bmatrix} 0 & -i \\ i & 0 \end{bmatrix}, \quad Z = \begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix}, \quad H = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix}.$$

The X gate is also known as the “not” gate, since it has the effect $|0\rangle \rightarrow |1\rangle$, $|1\rangle \rightarrow |0\rangle$; the Z gate is known as the phase-flip gate, since it has the effect $|0\rangle \rightarrow |0\rangle$, $|1\rangle \rightarrow -|1\rangle$; with the Y gate rarely referenced by name, as it is equal to iXZ ; finally, the H gate is the Hadamard gate, and represents the superposition-generating Hadamard transformation that we already met in Sect. 5.3.

Exercise 6. Write the effect of X , Z , and H on the basis states of the Hadamard basis,

$$|+\rangle := \frac{|0\rangle + |1\rangle}{\sqrt{2}}, \quad |-\rangle := \frac{|0\rangle - |1\rangle}{\sqrt{2}}.$$

Compare with the effect of each gate on the computational basis states. What has changed?

The most common two-qubit gate is the “controlled-not” gate:

$$CX = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{bmatrix}.$$

Hans: readthrough cont

You may have noticed that CX has the effect of the first qubit *controlling* an X operation on the second qubit: if the starting state is $|b_1 b_2\rangle$, then the operation is equivalent to $X_2 |b_1 b_2\rangle$ *provided* $b_1 = 1$, acting like the identity otherwise. Crucially, unlike classical if-else logic, this “check” happens coherently, i.e. inside of the superposition – no need to measure the state and check.

Exercise 7. Find the unitary that has the same effect on the Hadamard basis as CX has on the computational basis. What should we call this unitary?

In fact, any operation U acting on n qubits implies a controlled version CU of the same operation acting on $n + 1$ qubits. If the first qubit acts like a control for the rest, this will have the shape:

$$CU = \begin{bmatrix} 1 & 0 \\ 0 & U \end{bmatrix}.$$

Exercise 8. Write down a matrix for the CCX gate, known as the Toffoli gate. Which classically-irreversible logic gate does this gate reversibly implement?

Exercise 9. Starting with the two-qubit state $|\psi\rangle = |\mathbf{q}_1\mathbf{q}_2\rangle = |0\rangle_1|0\rangle_2$, apply H_1 followed by CX. What are the possible outcomes of observing the resulting state, and what are their probabilities? Compare with the result of instead applying the two single-qubit gates H_1 and H_2 .
Hint: Recall that the probability of observing each outcome is found by squaring its amplitude.

Another two-qubit unitary that will turn out to be useful is the SWAP gate, which acts on the computational-basis qubit states like

$$|b_1b_2\rangle \xrightarrow{\text{SWAP}} |b_2b_1\rangle .$$

Watch its effect on two general single-qubit states:

$$\begin{aligned} |\psi\varphi\rangle &= (\alpha_0|0\rangle + \alpha_1|1\rangle) \otimes (\beta_0|0\rangle + \beta_1|1\rangle) = \alpha_0\beta_0|00\rangle + \alpha_0\beta_1|01\rangle + \alpha_1\beta_0|10\rangle + \alpha_1\beta_1|11\rangle \\ &\xrightarrow{\text{SWAP}} \alpha_0\beta_0|00\rangle + \alpha_0\beta_1|10\rangle + \alpha_1\beta_0|01\rangle + \alpha_1\beta_1|11\rangle = (\beta_0|0\rangle + \beta_1|1\rangle) \otimes (\alpha_0|0\rangle + \alpha_1|1\rangle) = |\varphi\psi\rangle . \end{aligned}$$

We see that the SWAP gate has the effect of swapping the *general* states of two qubits with each other, earning its name.¹⁸

Exercise 10. Give a matrix representation of the SWAP gate.

Hint: Use its effect on each of the two-qubit computational basis states to “read off” the matrix.

Transformations across Space and Time. You may have noticed that we have been dealing with two different kinds of product for unitary matrices: the usual matrix product UV , and the tensor product $U \otimes V$. What does each kind of product represent?

To illustrate, let $|\psi\rangle$ and $|\varphi\rangle$ be two single-qubit states, combined in the two-qubit state $|\psi, \varphi\rangle$. Then, if U and V are single-qubit operations, so represented by 2×2 unitary matrices, and you want to apply U to $|\psi\rangle$ and V to $|\varphi\rangle$, this would be equivalent to the operation

$$(U \otimes V)(|\psi\rangle \otimes |\varphi\rangle) = (U \otimes V)|\psi, \varphi\rangle .$$

The unitary matrix formed by the tensor product $U \otimes V$ will be a 4×4 unitary, a two-qubit operation that combine the effects of both single-qubit unitaries in parallel, (that is, in the same time step).

Following the register-subscript convention introduced earlier, if we wanted to apply U only to the first qubit, we would like to be able to write

$$U_1|\psi, \varphi\rangle = U_1|\psi\rangle_1|\varphi\rangle_2 ,$$

which we may now define as

$$U_1(|\psi\rangle \otimes |\varphi\rangle) := (U \otimes \mathbf{1})(|\psi\rangle \otimes |\varphi\rangle) ,$$

and likewise if we want to only apply V to the second qubit:

$$V_2|\psi, \varphi\rangle := (\mathbf{1} \otimes V)(|\psi\rangle \otimes |\varphi\rangle) .$$

Extending to multi-qubit registers is straightforward: Let $|\psi\rangle = |\psi\rangle_{\mathbf{R}, \mathbf{S}, \mathbf{T}}$ be defined over a number of qubits that can be split into three sub-registers $\mathbf{R}$, $\mathbf{S}$, and $\mathbf{T}$, comprising n , m , and k qubits, respectively. Say you have a $2^m \times 2^m$ -unitary U , and you would like to apply it to the qubits in register $\mathbf{S}$. Then we may write:

$$U_{\mathbf{S}}|\psi\rangle := (\mathbf{1}_{2^n \times 2^n} \otimes U \otimes \mathbf{1}_{2^k \times 2^k})|\psi\rangle .$$

¹⁸ Were the *states* swapped, or did the *qubits* physically switch places? The answer is “either works!”, which is why we deliberately avoided specifying registers in this example. Fundamentally, this is because qubits must be made of physically indistinguishable particles. If they weren’t, the fact that they could be distinguished from each other could be used to construct a larger basis, say, $\{|0_a\rangle, |1_a\rangle, |0_b\rangle, |1_b\rangle\}$, at which point this is no longer a qubit system at all. What’s more, if particle a can only be observed to be in the a basis states, and particle b can only be observed in the b basis states, then there is no way for their quantum states to interfere!

In this sense, the tensor product represents combinations of operations happening at the same moment in time, but across different points in the *space* of qubit registers.

Now, imagine that you instead have two (single- or multi-qubit) unitaries U and V , and that you would like to first apply U , then V . In other words, you want something to the effect of

$$|\psi\rangle \xrightarrow{U} |\psi'\rangle \xrightarrow{V} |\psi''\rangle .$$

This is given by the following product.

$$VU |\psi\rangle = V(U |\psi\rangle) = V |\psi'\rangle = |\psi''\rangle .$$

Hence, the matrix product represents operations happening in different moments in time, in this case across two time steps, with the right unitary applied first and the left one thereafter.

Note the reverse time-ordering! Given that most matrices don't commute, this order clearly matters, and the above operational interpretation tells us why: Two unitaries commute if and only if applying the operations in either order always yields the same result, regardless of state.

But hold on – is “time step” even a well-defined unit here? After all, with U and V being arbitrary unitary matrices, we could always define a new unitary $W = VU$, and say that *this* is the operation that was applied, now apparently in unit time! This is why before analysing the time it would take to run a quantum algorithm, we first have to agree on an *elementary gate set*, so that we can take those to be unit-time, and let the algorithm we actually analyze be the implementation “compiled” to this gate set, factoring the final unitary into tensor and matrix products of the elementary unitaries. More on this in Sect. 5.9, when we discuss *complexity*.

To summarize, the tensor product represents unitaries applied in *parallel*, while the matrix product represents unitaries applied in *series*.

5.7 Measurements

At the end of our quantum algorithm, we would usually like to read out some classical information by measuring the end-state. We mentioned earlier that measurements constitute an exception to the rule that any operation performed on a quantum state must be unitary. We will give two explanations: First, the easily-applied intuition, then the more powerful mathematical formalism.

State Collapse. If you perform a standard, computational-basis measurement on a state $|\psi\rangle = \sum_x \alpha_x |x\rangle$, then you will observe value x with probability $|\alpha_x|^2 = \alpha_x^* \alpha_x$. After observation, if you continue to perform the same measurement on the state, you will continue to see the same value x – we say that the superposition state $|\psi\rangle$ has *collapsed* to the basis state $|x\rangle$. In other words, this post-collapse state has $\alpha_x = 1$, with all the other α set to 0.

$$|\psi\rangle = \sum_x \alpha_x |x\rangle \xrightarrow{\text{collapse}} |\psi'\rangle = |x\rangle .$$

Note the irreversibility: Once the state collapses, any information about the prior state is *deleted*, except for the single bit of information that α_x had non-zero magnitude.

That is really all you need to know about measurements to get started with quantum computing. But if you'd like a peek at the machinery required to place measurements on formal ground, read on – for if observing a quantum state is not a unitary operation, then what kind of operation *is* it?

Exercise 11. Let $|\psi\rangle$ be the single-qubit state given by

$$|\psi\rangle = \frac{e^{i\theta}|0\rangle + e^{i\phi}|1\rangle}{\sqrt{2}},$$

where $\theta, \phi \in [0, 2\pi)$ are *phases*.

- a) What is the probability of observing each outcome under a computational-basis measurement?
- b) Show that $|\psi\rangle$ is observationally equivalent (for every choice of basis) to the state

$$|\psi\rangle = \frac{|0\rangle + e^{i\theta'}|1\rangle}{\sqrt{2}},$$

where θ' is the *relative phase* between $|0\rangle$ and $|1\rangle$. What is θ' , in terms of θ and ϕ ?

Hint: Recall Exercise 4.

Projectors. To see what kind of non-unitary matrix is needed to represent the act of measuring a state, recall that after state collapse, further observations is guaranteed to just yield the same result. The state has already collapsed; it cannot collapse *further*.¹⁹

Let P represent such an *idempotent* operation. Then,

$$P(P|\psi\rangle) = P|\psi\rangle \iff P^2 = P.$$

Operators like P are known as *projectors*, so named because they effectively project higher dimensional objects onto some lower-dimensional hyperplane. For single-qubit states, it is easy to verify that the outer product of a basis state with itself will satisfy the definition: Let P_0 and P_1 be given by

$$P_0 = |0\rangle\langle 0| = \begin{bmatrix} 1 \\ 0 \end{bmatrix} \begin{bmatrix} 1 & 0 \end{bmatrix} = \begin{bmatrix} 1 & 0 \\ 0 & 0 \end{bmatrix}, \quad P_1 := |1\rangle\langle 1| = \begin{bmatrix} 0 \\ 1 \end{bmatrix} \begin{bmatrix} 0 & 1 \end{bmatrix} = \begin{bmatrix} 0 & 0 \\ 0 & 1 \end{bmatrix}.$$

From these matrix representations, we can see that effect of applying P_b to a single-qubit state is to single out the $|b\rangle$ -part of the state from the superposition. For example, applying P_0 to a general qubit state yields:

$$P_0|\psi\rangle = P_0(\alpha|0\rangle + \beta|1\rangle) = \begin{bmatrix} 1 & 0 \\ 0 & 0 \end{bmatrix} \begin{bmatrix} \alpha_0 \\ \alpha_1 \end{bmatrix} = \alpha_0 \begin{bmatrix} 1 \\ 0 \end{bmatrix} = \alpha_0|0\rangle.$$

Applying P_0 a second time, then, has no effect:

$$P_0^2|\psi\rangle = P_0(\alpha_0|0\rangle) = \begin{bmatrix} 1 & 0 \\ 0 & 0 \end{bmatrix} \begin{bmatrix} \alpha_0 \\ 0 \end{bmatrix} = \alpha_0|0\rangle = P_0|\psi\rangle.$$

If we instead apply P_1 as the second projector, we get the zero vector:

$$P_1P_0|\psi\rangle = P_1(\alpha_0|0\rangle) = \begin{bmatrix} 0 & 0 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} \alpha_0 \\ 0 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \end{bmatrix}.$$

Hence, these projectors are *orthogonal* in the sense that they satisfy $P_iP_j = \delta_{ij}P_i$. What's more, the set $\{P_0, P_1\}$ is *complete* in the sense that it sums to identity:

$$P_0 + P_1 = \begin{bmatrix} 1 & 0 \\ 0 & 0 \end{bmatrix} + \begin{bmatrix} 0 & 0 \\ 0 & 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} = \mathbb{1}.$$

On the other hand, these matrices are manifestly *not* unitary, as can be seen from the fact that they are not norm-preserving (cf. Exercise 5): If $|\psi\rangle$ starts out as a superposition state, then $P_0|\psi\rangle$ will not be unit-length and therefore not a valid quantum state. Alternatively, you may have noticed that the projector matrices are not even *invertible*, reflecting the information loss incurred by measurements.

Hans: We also need Hermiticity if we want our probabilities to end up positive; make counterexample an exercise?

¹⁹ Unless you switch your measurement basis, that is; more on this in a minute.

Definition 6 (Positive Semi-Definite (PSD), Positive Definite (PD)). Let $\mathcal{H}$ be a Hilbert space, and let A be a Hermitian operator on $\mathcal{H}$ ($A^\dagger = A$). Then A is positive semi-definite, written $A \succeq 0$, if for every pure state $|\psi\rangle \in \mathcal{H}$,

$$\langle \psi | A | \psi \rangle \geq 0.$$

If the inequality is strict, then A is positive definite (PD), written $A \succ 0$.

Definition 7 (Complete, Orthogonal Projectors). A set of k projectors $\{P_i\}_{i \in [k]}$ on a Hilbert space $\mathcal{H}$ is said to be complete if the projectors sum to identity:

$$\sum_{i \in [k]} P_i = \mathbb{1},$$

and orthogonal if the projectors are Hermitian, idempotent, and mutually orthogonal:

$$P_i^\dagger = P_i, \quad P_i P_j = \delta_{ij} P_i.$$

Then the projectors P_i are PSD operators with eigenvalues in $\{0, 1\}$ (as follows from $P_i = P_i^\dagger$ and $P_i^2 = P_i$) which project $\mathcal{H}$ onto mutually orthogonal subspaces $\mathcal{S}_i := P_i \mathcal{H}$ that together compose the full Hilbert space (as follows from orthogonality and completeness):

$$\bigoplus_{i \in [k]} \mathcal{S}_i = \mathcal{H}.$$

Projective Measurements. We already mentioned the Born rule, which says that the probability of observing an outcome is equal to the norm-squared of the corresponding amplitude; hence, quantum states (which are vectors of amplitudes) are the quantum equivalent of probability distributions (lists of probabilities). For example, performing a computational-basis measurement on a qubit in the state $|\psi\rangle = \alpha_0 |0\rangle + \alpha_1 |1\rangle$ induces the distribution

$$\mathcal{D}_\psi^{\text{comp}} = (|\alpha_0|^2, |\alpha_1|^2) \iff \mathcal{D}_\psi^{\text{comp}}(b) = |\alpha_b|^2 = \|P_b |\psi\rangle\|^2 = \langle \psi | P_b | \psi \rangle,$$

Hans: kinda feels like we have no choice but to count list elements from 0 instead of 1

where we used the properties of the projectors P_0, P_1 to rewrite the probability of observing outcome b . This suggests a natural n -qubit generalization: Let $|\psi\rangle = \sum_x \alpha_x |x\rangle$. Then performing a computational-basis measurement on all n qubits induces the distribution

$$\mathcal{D}_\psi^{\text{comp}}(x) = |\alpha_x|^2 = \langle \psi | P_x | \psi \rangle.$$

What about other measurements that you might want to perform? Say, for example, you only want to measure whether the first bit x_0 of the string x is equal to 0 or 1. To express this as a measurement on $|\psi\rangle$, you might define the projectors

$$P_{x_0=0} := P_0 \otimes \mathbb{1}_{2^{n-1}}, \quad P_{x_0=1} := P_1 \otimes \mathbb{1}_{2^{n-1}}.$$

Then $\{P_{x_0=0}, P_{x_0=1}\}$ is a complete set of orthogonal projectors, as they must be, since they each correspond to mutually-exclusive outcomes whose probabilities sum to one. Call the set of these two projectors $\mathcal{M}_{x_0}$. Performing the corresponding measurement then induces the distribution

$$\mathcal{D}_\psi^{\mathcal{M}_{x_0}}(b) = \langle \psi | P_{x_0=b} | \psi \rangle = \sum_{x: x_0=b} |\alpha_x|^2.$$

The set $\mathcal{M}_{x_0}$ is an example of a *Projector-Valued Measure* (PVM). Any set of complete, orthogonal projectors defines a PVM; conversely, for any *sharp* measurement (meaning measurements such that repeating them yields the same outcome) such that there are k possible, mutually exclusive outcomes, there is a corresponding PVM.²⁰ We call such measurements *projective*.

²⁰ The more general class of measurements are the POVMs, or “Positive Operator-Valued Measures”, which capture the class of measurements with k possible, mutually exclusive outcomes (like above), but which may be *non-sharp*, meaning that repeating the measurement may yield a different results. (Intuitively, you may think of POVMs as being to PVMs what mixed states are to pure states.)

Definition 8 (Projective Measurement). Let $\{P_i\}_{i \in [k]}$ be a complete, orthogonal set of projectors on a Hilbert space $\mathcal{H}$. Then $\mathcal{M} = \{P_i\}_{i \in [k]}$ is a Projection-Valued Measure (PVM), under which measuring a state $|\psi\rangle \in \mathcal{H}$ induces a probability distribution $\mathcal{D}_\psi^\mathcal{M}$ over outcomes $i \in [k]$, given by

$$\mathcal{D}_\psi^\mathcal{M}(i) := \|P_i |\psi\rangle\|^2 = \langle \psi | P_i | \psi \rangle .$$

We will use $X \leftarrow \mathcal{M}(|\psi\rangle)$ as shorthand for sampling a random variable X from the distribution $\mathcal{D}_\psi^\mathcal{M}$.

The projected state $P_i |\psi\rangle$ will not (usually) be unit-norm, and hence not a valid quantum state, so we still need to define an update procedure for the post-measurement state, given that a particular outcome i was observed: If we observed outcome i , then the projected state vector $P_i |\psi\rangle$ will be pointing in the right direction in Hilbert space, but its magnitude will be $\|P_i |\psi\rangle\| = |\alpha_i| \leq 1$. Physically speaking, observing a quantum state has the effect of “snapping” the vector from the old direction in Hilbert space, to a new direction consistent with the observed outcome – the magnitude never shifts away from 1. Mathematically, we can recover this behaviour by defining the post-measurement state to be the projected state vector $P_i |\psi\rangle$ (giving the direction), *renormalized* to 1 by dividing away the factor $|\alpha_i|$.

Definition 9 (Post-Measurement State). Let $\mathcal{M} = \{P_i\}_{i \in [k]}$ be a PVM. Upon observing a state $|\psi\rangle$, if outcome i is observed, the state will immediately²¹ update (or “collapse”) to the state

$$|\psi\rangle \xrightarrow{\mathcal{M} \rightarrow i} \text{Collapse}_i^\mathcal{M}(|\psi\rangle) := \frac{P_i |\psi\rangle}{\|P_i |\psi\rangle\|} = \frac{P_i |\psi\rangle}{\sqrt{\langle \psi | P_i | \psi \rangle}} ,$$

where conditioning on outcome i guarantees $\Pr[\text{observe } i] > 0 \Rightarrow \|P_i |\psi\rangle\| > 0$.

Note how $\text{Collapse}_i^\mathcal{M}(|\psi\rangle)$ is a *non-linear* map on the Hilbert space, in stark contrast to the unitary operations we have encountered so far. In fact, $\text{Collapse}_i^\mathcal{M}(|\psi\rangle)$ is not even injective – there is an infinite continuity of pre-measurement states that could have yielded the post-measurement state – which shows how measurements are fundamentally *irreversible operations*.

Exercise 12.

- a) Show that the matrix $P_{x_0=b} = P_b \otimes \mathbb{1}_{2^{n-1}}$ is a projector.
- b) Show that the set $\mathcal{M}_{x_0} = \{P_{x_0=0}, P_{x_0=1}\}$ is complete and orthogonal.
- c) Show that the projector $P_{x_0=b}$ is equal to the sum $\sum_{x: x_0=b} P_x$.

Hint: Consider the effect of applying $P_{x_0=b}$ to $|\psi\rangle = \sum_x \alpha_x |x\rangle$.

Hans: Exercise idea: find the projectors P_+ , P_- , expressed in the computational basis, and verify that transforming them to the Hadamard basis makes them take the form P_0 , P_1 .

Distinguishing Between Quantum States. Say that you are given qubit register $\mathbf{R}$ in some unknown state $|\varphi\rangle$, but you are promised that $|\varphi\rangle$ is equal to one of two known states (which you also know how to prepare), $|\psi\rangle$ or $|\varphi\rangle$. What measurement should you do to maximize your likelihood of telling them apart? Note that $|\psi\rangle$ and $|\varphi\rangle$ need not be orthogonal, so there may be many possible measurement outcomes that are consistent with $|\varphi\rangle$ being either state.

The SWAP Test. A natural first thing to try is the procedure known as the “SWAP test”: Prepare a new register $\mathbf{S}$ in one of the two states, say $|\psi\rangle$, and prepare an ancilla qubit $\mathbf{q}$ in the uniform superposition $|+\rangle$. Then, perform a controlled SWAP of the states $|\varphi\rangle$ and $|\psi\rangle$, controlled on the ancilla qubit:

$$|+\rangle_{\mathbf{q}} |\varphi\rangle_{\mathbf{R}} |\psi\rangle_{\mathbf{S}} = \left(\frac{|0\rangle_{\mathbf{q}} + |1\rangle_{\mathbf{q}}}{\sqrt{2}} \right) |\varphi\rangle_{\mathbf{R}} |\psi\rangle_{\mathbf{S}} \xrightarrow{\text{CSWAP}} \frac{1}{\sqrt{2}} \left(|0\rangle_{\mathbf{q}} |\varphi\rangle_{\mathbf{R}} |\psi\rangle_{\mathbf{S}} + |1\rangle_{\mathbf{q}} |\psi\rangle_{\mathbf{R}} |\varphi\rangle_{\mathbf{S}} \right) .$$

Then, apply a Hadamard gate to $\mathbf{q}$ and perform a computational-basis measurement on it. If the unknown state is $|\psi\rangle$, then the CSWAP operation had no effect:

$$\left(\frac{|0\rangle_{\mathbf{q}} + |1\rangle_{\mathbf{q}}}{\sqrt{2}} \right) |\psi\rangle_{\mathbf{R}} |\psi\rangle_{\mathbf{S}} \xrightarrow{\text{CSWAP}} \left(\frac{|0\rangle_{\mathbf{q}} + |1\rangle_{\mathbf{q}}}{\sqrt{2}} \right) |\psi\rangle_{\mathbf{R}} |\psi\rangle_{\mathbf{S}} \xrightarrow{\text{H}} |0\rangle_{\mathbf{q}} |\psi\rangle_{\mathbf{R}} |\psi\rangle_{\mathbf{S}} ,$$

²¹ Whether the state update upon measuring is truly instantaneous, or if there is a continuous, dynamical process underlying it (or if the question is ill-posed), remains a hotly debated question in the foundations of physics.

so that when we measure $\mathbf{q}$, we are guaranteed to observe the value 0. Hence, if we observe a 1, then we learn that $|\varphi\rangle$ is *not* identical to $|\psi\rangle$, from which we may conclude that $|\varphi\rangle$ must be $|\varphi\rangle$.

What is the probability that we observe 1 in this case? If $|\varphi\rangle$ is equal to $|\varphi\rangle$, then we get

$$\begin{aligned} \left(\frac{|0\rangle_{\mathbf{q}} + |1\rangle_{\mathbf{q}}}{\sqrt{2}}\right) |\varphi\rangle_{\mathbf{R}} |\psi\rangle_{\mathbf{S}} &\xrightarrow{\text{CSWAP}} \frac{1}{\sqrt{2}} \left(|0\rangle_{\mathbf{q}} |\varphi\rangle_{\mathbf{R}} |\psi\rangle_{\mathbf{S}} + |1\rangle_{\mathbf{q}} |\psi\rangle_{\mathbf{R}} |\varphi\rangle_{\mathbf{S}}\right) \\ &\xrightarrow{\text{H}} \frac{1}{\sqrt{2}} \left(\left(\frac{|0\rangle_{\mathbf{q}} + |1\rangle_{\mathbf{q}}}{\sqrt{2}}\right) |\varphi\rangle_{\mathbf{R}} |\psi\rangle_{\mathbf{S}} + \left(\frac{|0\rangle_{\mathbf{q}} - |1\rangle_{\mathbf{q}}}{\sqrt{2}}\right) |\psi\rangle_{\mathbf{R}} |\varphi\rangle_{\mathbf{S}}\right) \\ &= \frac{1}{2} (|0\rangle (|\varphi\rangle_{\mathbf{R}} |\psi\rangle_{\mathbf{S}} + |\psi\rangle_{\mathbf{R}} |\varphi\rangle_{\mathbf{S}}) + |1\rangle (|\varphi\rangle_{\mathbf{R}} |\psi\rangle_{\mathbf{S}} - |\psi\rangle_{\mathbf{R}} |\varphi\rangle_{\mathbf{S}})) . \end{aligned}$$

Let $P_b^{(\mathbf{q})}$ be the projector $P_b \otimes \mathbb{1}_{\mathbf{R}} \otimes \mathbb{1}_{\mathbf{S}}$, i.e. the projector that acts like P_b on $\mathbf{q}$ and like the identity on $\mathbf{R}$ and $\mathbf{S}$. Then, Def. 8 tells us that the probability that measuring $\mathbf{q}$ will yield outcome 1 is

$$\begin{aligned} &\left\| P_1^{(\mathbf{q})} \frac{1}{2} (|0\rangle (|\varphi\rangle_{\mathbf{R}} |\psi\rangle_{\mathbf{S}} + |\psi\rangle_{\mathbf{R}} |\varphi\rangle_{\mathbf{S}}) + |1\rangle (|\varphi\rangle_{\mathbf{R}} |\psi\rangle_{\mathbf{S}} - |\psi\rangle_{\mathbf{R}} |\varphi\rangle_{\mathbf{S}})) \right\|^2 \\ &= \left\| \frac{1}{2} |1\rangle (|\varphi\rangle_{\mathbf{R}} |\psi\rangle_{\mathbf{S}} - |\psi\rangle_{\mathbf{R}} |\varphi\rangle_{\mathbf{S}}) \right\|^2 = \frac{1}{4} |1\rangle \langle 1| \langle \langle \psi|_{\mathbf{S}} \langle \varphi|_{\mathbf{R}} - \langle \varphi|_{\mathbf{S}} \langle \psi|_{\mathbf{R}} (|\varphi\rangle_{\mathbf{R}} |\psi\rangle_{\mathbf{S}} - |\psi\rangle_{\mathbf{R}} |\varphi\rangle_{\mathbf{S}}) | \\ &\quad \frac{1}{4} |\langle \varphi|_{\mathbf{R}} \langle \psi|_{\mathbf{S}} + \langle \psi|_{\mathbf{R}} \langle \varphi|_{\mathbf{S}} - \langle \psi|_{\mathbf{R}} \langle \varphi|_{\mathbf{S}} - \langle \varphi|_{\mathbf{R}} \langle \psi|_{\mathbf{S}}| = \frac{1}{2} - \frac{1}{2} |\langle \psi|_{\mathbf{R}} \langle \varphi|_{\mathbf{S}}|^2 . \end{aligned}$$

Note how, even if $|\psi\rangle$ and $|\varphi\rangle$ are orthogonal, then the probability of observing 1 is only $\frac{1}{2} - 0 = \frac{1}{2}$, so that even in the best case, while observing a 1 would allow us to immediately conclude that $|\varphi\rangle = |\varphi\rangle$, observing the zero outcome would only provide relatively weak evidence towards $|\varphi\rangle$ being $|\psi\rangle$, achieving in expectation the success rate

$$\mathbb{E}[\text{success}] = \frac{1}{2} \Pr[0 \leftarrow \mathcal{M}_{\text{Hel}}(|\psi\rangle)] + \frac{1}{2} \Pr[1 \leftarrow \mathcal{M}_{\text{Hel}}(|\varphi\rangle)] = \frac{1}{2} + \frac{1}{2} \left(\frac{1}{2} - \frac{1}{2} |\langle \psi|_{\mathbf{R}} \langle \varphi|_{\mathbf{S}}|^2 \right) = \frac{3}{4} - \frac{1}{4} |\langle \psi|_{\mathbf{R}} \langle \varphi|_{\mathbf{S}}|^2 .$$

As a sanity check, notice how the expected success rate falls to $1/2$ when the states are identical, and how orthogonal states maximize the rate.

Given several copies of $|\varphi\rangle$, we could of course amplify our distinguishing advantage by repeating the experiment. But what if we only had the one copy of $|\varphi\rangle$? With the state being unknown to us, the *no-cloning theorem* (see Sect. 5.8) bars us from generating copies of it, and we are stuck with a single measurement. Can we do better?

The Helstrom Measurement. As it turns out, the answer is yes, and the *optimal* distinguishing measurement is known as the *Helstrom measurement* after its discoverer.

Theorem 4 (The Helstrom Measurement). *Given a register $\mathbf{R}$ promised to be in either state $|\psi\rangle$ or state $|\varphi\rangle$ with equal probability, then the Helstrom measurement $\mathcal{M}_{\text{Hel}}$ correctly identifies the state with probability*

$$\Pr[0 \leftarrow \mathcal{M}_{\text{Hel}}(|\psi\rangle)] = \Pr[1 \leftarrow \mathcal{M}_{\text{Hel}}(|\varphi\rangle)] = \frac{1}{2} + \frac{1}{2} \sqrt{1 - |\langle \psi|_{\mathbf{R}} \langle \varphi|_{\mathbf{S}}|^2} ,$$

which is optimal.

Unlike our SWAP test, if the two possible states $|\psi\rangle$, $|\varphi\rangle$ are orthogonal, then the Helstrom measurement is guaranteed to distinguish them:

$$|\langle \psi|_{\mathbf{R}} \langle \varphi|_{\mathbf{S}}| = 0 \implies \Pr[0 \leftarrow \mathcal{M}_{\text{Hel}}(|\psi\rangle)] = \Pr[1 \leftarrow \mathcal{M}_{\text{Hel}}(|\varphi\rangle)] = \frac{1}{2} + \frac{1}{2} \sqrt{1 - 0} = 1 .$$

In that case, the measurement is simply given by the PVM

$$\mathcal{M}_{\text{Hel}} = \{P_{\psi}, P_{\varphi}\} = \{|\psi\rangle\langle\psi|, |\varphi\rangle\langle\varphi|\} .$$

What about the more general case? Well, if $|\psi\rangle$ and $|\varphi\rangle$ are non-orthogonal, then $\{|\psi\rangle\langle\psi|, |\varphi\rangle\langle\varphi|\}$ is not a PVM, since these projectors will not be orthogonal either – we need to build two *orthogonal* projectors from $|\psi\rangle$ and $|\varphi\rangle$. The Helstrom measurement does this by picking out the optimal orientation in the 2D

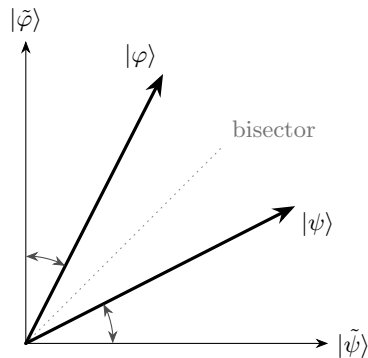


Fig. 12. The orthogonal states $|\tilde{\psi}\rangle$ and $|\tilde{\varphi}\rangle$ defining the (equal-priors) Helstrom measurement live in the 2D plane spanned by $|\psi\rangle$ and $|\varphi\rangle$, oriented symmetrically about their bisecting line.

plane spanned by $|\psi\rangle$ and $|\varphi\rangle$, as illustrated in Fig. 12: Let $|\tilde{\psi}\rangle$ and $|\tilde{\varphi}\rangle$ be the two orthogonal states that 1) sit in the plane spanned by $|\psi\rangle$ and $|\varphi\rangle$, and 2) are oriented such that the bisecting line between $|\psi\rangle$ and $|\varphi\rangle$ sits at a 45° angle to both states. Then the Helstrom measurement is given by the PVM

$$\mathcal{M}_{\text{Hel}} = \{P_\psi, P_\varphi\} = \{|\tilde{\psi}\rangle\langle\tilde{\psi}|, |\tilde{\varphi}\rangle\langle\tilde{\varphi}|\},$$

which optimally distinguishes the two states, provided either state were given to us with equal probability.²²

Keep in mind, however, that in order to perform a Helstrom measurement, we need to not only *know* the states $|\psi\rangle$ and $|\varphi\rangle$, but also be able to *efficiently prepare them*, which is a capability we may or may not possess in any given situation. Contrast with the SWAP test, where it sufficed for us to be able to efficiently prepare one of the two candidate states.

Exercise 13. Let $|\psi\rangle = \cos \frac{\theta}{2} |0\rangle + \sin \frac{\theta}{2} |1\rangle$ and $|\varphi\rangle = \cos \frac{\theta}{2} |0\rangle - \sin \frac{\theta}{2} |1\rangle$ for an angle $\theta \in [0, 2\pi)$, and let $|sum\rangle = \frac{|\psi\rangle + |\varphi\rangle}{\| |\psi\rangle + |\varphi\rangle \|}$ and $|diff\rangle = \frac{|\psi\rangle - |\varphi\rangle}{\| |\psi\rangle - |\varphi\rangle \|}$ be the states given by their (normalized) sum and difference.

- Show that $|\tilde{\psi}\rangle = \frac{1}{\sqrt{2}} (|sum\rangle + |diff\rangle)$ and $|\tilde{\varphi}\rangle = \frac{1}{\sqrt{2}} (|sum\rangle - |diff\rangle)$.
- Use this to show that $\mathcal{M}_{\text{Hel}} = \{|\tilde{\psi}\rangle\langle\tilde{\psi}|, |\tilde{\varphi}\rangle\langle\tilde{\varphi}|\}$ satisfies the definition of a PVM.
- Show that for any two states $|\psi\rangle, |\varphi\rangle$, there exists a choice of θ such that they can be expressed as above (up to an unobservable global phase $e^{i\phi}$).

5.8 Entanglement

Just like classical probabilities can be correlated, so can quantum states be *entangled*. Consider the following, two-qubit product state:

$$|\psi\rangle = |+\rangle |+\rangle = \left(\frac{|0\rangle + |1\rangle}{\sqrt{2}} \right) \left(\frac{|0\rangle + |1\rangle}{\sqrt{2}} \right) = \frac{|00\rangle + |01\rangle + |10\rangle + |11\rangle}{2}.$$

Regardless of which measurement you apply to $|\psi\rangle$, the value observed in the first qubit will bear no implication for the value that the second qubit will take. We say that the state is *separable*, which is to say that any measurement on $|\psi\rangle$ will induce a *product distribution* along the same separating line.

²² There is also the *weighted* Helstrom measurement, for the case that the probability of receiving each state is shifted away from $1/2$ by some known amount. If the probability of receiving each state is p resp. $1 - p$, it effectively biases the measurement by rotating the measurement axes $|\tilde{\psi}\rangle$ and $|\tilde{\varphi}\rangle$ in Fig. 12 by the angle (which can be derived from p) that maximizes the expected success rate, achieving the optimal rate

$$\mathbb{E}[\text{success}] = p \Pr[0 \leftarrow \mathcal{M}_{\text{Hel}}(|\psi\rangle)] + (1 - p) \Pr[1 \leftarrow \mathcal{M}_{\text{Hel}}(|\varphi\rangle)] = \frac{1}{2} + \frac{1}{2} \sqrt{1 - 4p(1 - p)|\langle\psi|\varphi\rangle|^2}.$$

Contrast with the following two-qubit state:

$$|\varphi\rangle = \frac{|00\rangle + |11\rangle}{\sqrt{2}}.$$

With the middle $|01\rangle$ and $|10\rangle$ missing, there is no way to write this state as a product of two single-qubit states; the state is *non-separable*, which is to say that observing the state will yield outcome distributions with correlations between the values of the two qubits. In this case it is easy to see, as the two qubits are perfectly correlated: If you perform a computational-basis measurement, you will see the string 00 or the string 11, never 01 or 10. Hence, if you observe the first qubit and see 1, you *know* that observing the second qubit will also yield the value 1.

This is the phenomenon known as quantum *entanglement*, and despite its reputation as something deeply mysterious, it really is just the quantum version of correlated probability distributions. The apparent mysteriousness comes from the fact that the way we formalized measurements in Sect. 5.7, observing one of the qubits of the state $|\varphi\rangle$ above leads to collapse of the full state, so that the second qubit – which previously was in a superposition of 0 and 1 – is suddenly fully determined. The immediateness of this effect means the state collapse will not even respect the light-speed limit, so that Alice measuring a qubit on Earth will immediately collapse the superposition of Bob’s qubit, even if Bob finds himself at the opposite side of the galaxy. This insight famously led to Einstein’s lament that quantum theory contained “spooky action at a distance”²³, and that it must therefore be fundamentally flawed.

Entanglement vs. Randomness. The modern view is that the above is taking the pure-state formalism too literally: The actual state that Bob would receive upon receiving the second qubit would be a *maximally mixed state*, which is to say a state that is perfectly consistent with a classical coin-flip. The only way to distinguish the entangled state from a random one would be to bring the two qubits back together, which by assumption is beyond Bob’s capability. With this mixed-state formalism in hand, it becomes clear that measurements applied to the qubit on Earth are completely irrelevant to the statistics observed by Bob at the other end of the galaxy.

We will work out the details of this thought experiment when we later introduce the density-matrix formulation, but for now, the take-away message is one that will turn out to be central to the compressed-oracle technique, covered in Sect. 12:

Inaccessible entanglement is perfectly indistinguishable from classical randomness.

Entanglement the Resource. All this is not to undersell the *power* of entanglement, as it is one of the central resources of quantum algorithms and interactive quantum protocols alike! Indeed, the word “resource” is justified by the fact that entanglement is a *basis-independent*, and hence somehow “objective”, property of the quantum state: the presence of correlations between different parts of the quantum system does not depend on which measurement you choose to apply to the state.

Exercise 14.

- a) Show that measuring $|\varphi\rangle$ under a computational-basis and a Hadamard-basis measurements lead to the same outcome statistics by showing the identity

$$|\varphi\rangle = \frac{|00\rangle + |11\rangle}{\sqrt{2}} = \frac{|++\rangle + |--\rangle}{\sqrt{2}}.$$

- b) Entanglement may be measurement-independent, but outcome statistics may not be: Let

$$|\xi\rangle = \frac{|00\rangle - |11\rangle}{\sqrt{2}}.$$

What are the distributions induced by performing a computational-basis versus a Hadamard-basis measurement on $|\xi\rangle$?

²³ *Spukhafte Fernwirkung* in the original German.

No Cloning. However, entanglement can in practice be as much a hinderance to quantum algorithm design as a resource. The simplest example of this is perhaps the case of copying, or “cloning”, quantum states. Say you are given a qubit in some unknown superposition state

$$|\psi\rangle = \alpha|0\rangle + \beta|1\rangle.$$

Naturally, you might want to create a copy of it, i.e. to create the state

$$|\psi\rangle|\psi\rangle = (\alpha|0\rangle + \beta|1\rangle)(\alpha|0\rangle + \beta|1\rangle) = \alpha^2|00\rangle + \alpha\beta|01\rangle + \alpha\beta|10\rangle + \beta^2|11\rangle.$$

The first thing to try would be to prepare a second qubit in the $|0\rangle$ state, then apply the controlled-not operation from the first to the second qubit, which will flip the qubit to $|1\rangle$ exactly in the β branch of the original state. The result is:

$$|\psi\rangle|0\rangle = (\alpha|0\rangle + \beta|1\rangle)|0\rangle \xrightarrow{\text{CX}} \alpha|00\rangle + \beta|11\rangle \neq |\psi\rangle|\psi\rangle.$$

Rather than a product of two copies of the original state, we have ended up with a maximally entangled state, fundamentally different from the superposition state we started with! If only we could somehow remove the entanglement by *inserting* the missing $|01\rangle$ and $|10\rangle$ terms, appropriately scaled, then perhaps we could get to the desired product state.

Alas, this turns out to be a fundamental constraint of superposition states, known today as the *no-cloning theorem*.

Exercise 15. In this exercise you will prove the no-cloning theorem. Let U be a two-qubit unitary operation that clones the single-qubit state $|\psi\rangle$ into an initialized qubit state, as follows:

$$U|\psi\rangle|0\rangle = |\psi\rangle|\psi\rangle.$$

The no-cloning theorem then states that no single unitary U can clone several arbitrary qubit states.^a

- a) Let $|\varphi\rangle$ be a second state, also cloneable via U in the same manner as above. Prove the theorem by deriving a restriction on the possible values that the inner product $\langle\psi|\varphi\rangle$ can take.
- b) The previous result showed that, while cloning is not possible in general, some qubit states *can* be cloned. What are these “clonable” states? How should they be interpreted?

^a Trivially, there exists a unitary U that clones $|\psi\rangle$ *specifically*, namely the unitary implemented by whatever circuit built $|\psi\rangle$ in the first place.

5.9 Universality

Earlier, we argued that the matrix product of unitaries represents operations happening in series, i.e. at different time steps. While true, this does not give a well-defined measure of time, as the product of two unitaries is just another unitary, which begs the question: did the operation happen in two time-steps, or one?

This is why, just like for classical circuits, we need to choose a set of *elementary quantum gates*. Then, time can be measured by counting how many of the elementary gates must be run *in series* in order to implement your unitary; this is known as the circuit *depth*, corresponding to the algorithm’s time complexity. Gates that can be run in parallel need not count against the circuit’s runtime, and the amount of parallelization your circuit employs at its highest is known as the circuit *width*. The circuit width tells you how much space (how many qubits) your circuit needs (sometimes referred to as the algorithm’s space complexity).

Universal Quantum Gate Sets. Of course, in order for your set of elementary gates to be able to implement *any* unitary, it better be *universal*. Classically, we know that any Boolean circuit can be generated by the AND, OR and NOT gates. However, on quantum computers we need all our operations (sans measurements) to be unitary, which at minimum means reversible (represented by invertible matrices). The AND and OR operations take two inputs and return one – they delete information, and they are therefore *not* reversible. Is this avoidable?

Years before he started thinking about quantum protocols, Charles Bennett²⁴ was interested in reversible computing. He showed that, yes, with a little overhead, any Boolean circuit can be made reversible [Ben73]. As a small example, say you have a non-injective function f , implemented as an irreversible circuit, and you want to make it reversible. Then you may instead compute (using quantum notation in anticipation of our intended application):

$$|x\rangle |y\rangle \rightarrow |x\rangle |y \oplus f(x)\rangle .$$

As it turns out, the classical three-bit *Toffoli* gate CCX (controlled-controlled-not: it flips the third bit iff the first two are both 1) suffices for universality: any reversible circuit (which, remember, can also implement any *irreversible* circuit) can be built from a series of Toffoli gates [Tof80]. In other words, for classical reversible computing, a universal elementary gate set is simply {CCX}.²⁵ We may then measure the time and space required by your circuit by first compiling it down to a Toffoli-only circuit, and then count the number of Toffolis the resulting circuit applies in series vs. in parallel.

So that covers the classical case. What do we need to get to universal quantum computing? The answer is quite satisfactory: Take the classical universal gate set {Tof}, and add the ability to create quantum superpositions via the Hadamard gate H; the quantum gate set {CCX, H} is universal for quantum computers [Aha03], and we may count time and space exactly like before.²⁶ In fact, with multi-qubit gates being much more resource-intensive in current quantum devices than single-qubit gates such as H, “number of Toffolis” has arisen as a standard measure of the depth of near-term quantum circuits.

The Magic of Quantum Computation. Another universal quantum gate set one often comes across is²⁷ {CX, H, S, T}, known as “Clifford + T”, the Clifford gates being {CX, H, S}. Here, the S and T gates are defined as

$$S = \sqrt{Z} = \begin{bmatrix} 1 & 0 \\ 0 & i \end{bmatrix}, \quad T = \sqrt{S} = \begin{bmatrix} 1 & 0 \\ 0 & e^{i\frac{\pi}{4}} \end{bmatrix}.$$

As written, this set is slightly redundant, since one can implement the S gate by applying the T gate twice; hence, the gate set {CX, H, T} would also be universal.²⁸

Could we get away with only the Clifford gate set, {CX, H, S}? This gate set certainly *looks* like it has all the required ingredients: the CNOT gate gives us most of classical computation, the Hadamard gate gives us superpositions, combining them gives us entanglement, and the S gate gives us not only negative, but also complex phases. (Recall that the {CCX, H} gate set reached a kind of computational universality even without any use of complex phases.)

And yet this turns out to not be enough: Restricting to the Clifford gates restricts the unitaries we can construct to a subgroup of the full unitary group known as the Clifford group. Worse, a famous result known as the Gottesman–Knill theorem [Got99] tells us that *any unitary of the Clifford group can be efficiently simulated on a classical computer*. In other words, if the Clifford gates were all we had, then any super-polynomial quantum advantage would vanish!

²⁴ Charles Bennett is the father not only of reversible computing, but also a founding father of quantum cryptography, having discovered Quantum Key-Distribution (QKD) together with Gilles Brassard in 1984, today known as the BB84 protocol [BB84]. In 1993, the pair (together with several coauthors) also discovered the deeply fundamental and surprising fact that entanglement enables “teleporting” quantum states across space using only classical communication [BBC⁺93]. In honour of their discoveries, Bennett and Brassard were awarded the 2025 Turing Award [ACM26].

²⁵ The old, irreversible Boolean circuits had a universal gate set with only two-bit gates, namely AND, OR and NOT. Is there a universal gate set for (classical) reversible computing that only contains two-bit gates? The answer turns out to be *no*: Toffoli showed that given only one- and two-bit gates, you could at most simulate *linear* Boolean circuits; simulating non-linear circuits, including products, requires at least one three-bit gate [Tof80]. Surprisingly, as we will shortly see, this is no longer true for quantum circuits, where quantum universality *can* be reached with only one- and two-qubit gates – which therefore also achieves classical reversible universality!

²⁶ Strictly speaking, starting from a real-valued initial state, the gate set {CCX, H} can only reach other real-valued states, since CCX and H are both real-valued matrices: states like $(|0\rangle + i|1\rangle)/\sqrt{2}$ will be forever out of reach. However, Bernstein and Vazirani showed that any n -qubit quantum circuit can be transformed into a observationally-equivalent, $n + 1$ -qubit, real-values-only quantum circuit, where the extra ancilla effectively keeps track of whether a given amplitude should be read as real or imaginary [BV97]. The gate set {CCX, H} is therefore sometimes said to be “computationally universal”, rather than strictly so.

²⁷ Beware a common confusion: The T ($\sqrt{S}$) gate $\neq$ the Toffoli (CCX) gate!

²⁸ Observe: No three-qubit gates!

Adding the innocuous-seeming $T = \sqrt{S}$ gate to the set flips this picture on its head: Suddenly, not only can the gate set escape the classically-simulatable Clifford group, but it reaches full universality, quantum advantage and all. This seems to indicate that there is something special about the T gate, since, when compiling circuits down to the Clifford + T gate set, if the circuit can be implemented without using any (or using very few) T gates, then it can also be classically simulated; contrapositively, if it can *not* be classically simulated, then the compiled circuit must make use of many T gates! Hence, the “ T -ness” of a quantum circuit says something about *how far beyond classical simulation* the circuit is, indicating a possible measure for its quantum advantage (assuming, of course, that the compilation method down to elementary gates minimizes the number of T gates used, hence why it is useful to include both S and T gates in the gate set).

This measure has a name, and it is *quantum magic*. Besides giving us a way to quantify exactly what we mean by “quantum advantage”, it has been recognized as a quantum *resource* on par with entanglement, central to developments in quantum error correction.

5.10 Complexity

When we compared classical and quantum computation above, we were actually touching upon the theory of quantum *complexity*, the framework needed to understand *if* and *how* quantum computers outperform classical ones. You may be familiar with the classes P (Polynomial time) and NP (Non-deterministic Polynomial time) from classical complexity theory, and the million-dollar question of their (in)equality [Coo00]. The corresponding quantum complexity classes are BQP (Bounded Quantum Polynomial time) and QMA (Quantum Merlin–Arthur) respectively, which come with their own foundational “is $BQP = QMA$?” question. In this view, asking “is quantum better than classical?” is *really* asking “is $P = BQP$?”²⁹

These classes also come with their own theories of completeness: just like there are NP -complete problems, there are QMA -complete problems, as well as BQP -complete problems. The latter is a particularly interesting class of problems, since if there is *any* superpolynomial quantum advantage at all, then these problems *must* be classically hard! In contrast, while Shor’s algorithm shows that integer factorization is in BQP , it is not (known to be) BQP -complete, leaving the door still open for that classical factorization algorithm which has eluded mathematicians for centuries.

5.11 Further Reading

This concludes our brief tour of the theory of quantum computing. The aim has been to introduce the notation and mathematical formalism that we will employ, to the level of being able to follow arguments and perform simple computations on quantum states, while hopefully dispelling a few common misconceptions along the way. Of course, there is much this primer did not touch upon, including illustrations of quantum circuits, rotation gates and the Bloch-sphere visualization of single-qubit states, quantum algorithms such as Grover’s and Shor’s, quantum-classical mixedness and more. If you want the full tour, Nielsen and Chuang’s book *Quantum Computation and Quantum Information* [NC10] is *the* standard reference (and deservedly so!), although the mathematically-minded among you may prefer the introductory chapters of Mark Wilde’s *Quantum Information Theory* [Wil17], which treats the subject with a higher level of rigor than Nielsen and Chuang that still does not sacrifice the helpful intuitions.

Needless to say, there is also a lot more to be said about computational complexity than the brief remarks given in Sect. 5.10. Complexity theory is a deep and endlessly fascinating subject, regularly taking us the outer edges of what is knowable in the physical universe (and beyond!); if you want to know more, there is perhaps no better place to start than Scott Aaronson’s highly engaging book *Quantum Computing Since Democritus* [Aar13].

²⁹ Technically the question is “is $BPP = BQP$?”, with Bounded Probabilistic Polynomial time being the class of problems solvable by classical randomized algorithms.

6 The Quantum Random Oracle Model (QROM)

6.1 Extra Preliminaries

Hans: dumped from Sect. 2 (old preliminaries)

Quantum(-accessible) oracles³⁰ are oracles that instead take a set of *qubit registers* as input, apply some operation on them, before returning the registers to the querying party. The operation applied may be probabilistic in nature and may include measurements (which has the effect of transforming pure quantum states into mixed states, though it will often be required to be unitary). For quantum oracle algorithms, we will take a classical oracle to be a quantum oracle that performs a computational-basis measurement on the registers before operating on them. Hence, we do not make a distinction between classical and quantum oracle access in our notation, writing $A^{\mathcal{O}}$ for either case.

In ??, we define a quantum pseudocode that allows us to talk about general quantum oracles, which we apply to quantum random oracles in Sect. 3. For now, we note that an oracle that evaluates a function f on superpositions of inputs is usually defined as the quantum oracle $\mathcal{O}_f$ that applies the unitary operation

$$|x, y\rangle \rightarrow |x, y \oplus f(x)\rangle,$$

where $|x, y\rangle$ are the computational basis states of the qubit registers input to the oracle. (In case this notation is unfamiliar to you, Sect. 5 provides a gentle introduction.)

6.2 What Is a Quantum Random Oracle?

A classical oracle for a function f is a black-box procedure $\mathcal{O}_f$ such that, given input x , it returns $f(x)$ in such a way that no information about f , other than that which can be inferred from its output, is provided. It is helpful to think of the oracle as something that exists beyond your reach, “in the sky” so to speak, and with which you can only interact by sending inputs and receiving outputs. A random oracle, then, is simply a function oracle where the function f was chosen uniformly at random from the space of all functions from $\{0, 1\}^n$ to $\{0, 1\}^m$.

However, the operation $x \rightarrow f(x)$ is generally not reversible, and therefore not *unitary*, as is required of operations on quantum states. Therefore, we define *quantum-accessible* function oracles to be oracles that apply the unitary U_f , defined as the operation that takes computational basis states $|x, y\rangle$ to $|x, y \oplus f(x)\rangle$. Generally, the state of the input may be entangled with whoever is performing the query, so saying that the oracle takes as input a quantum *state*, e.g. $\sum_i \alpha_i |x_i, y_i\rangle$, would not be well-defined. Instead, we imagine that the query literally ships a collection of qubits over to the oracle, collected in the input and output registers ($\mathbf{X}, \mathbf{Y}$), and that the oracle then applies the U_f to the registers before shipping them back (see Fig. 13 for an illustration).³¹

To see that this operation is unitary, it is enough to observe that it is *reversible*: Making a query with the same registers a second time yields the original state, as seen by its effect on the computational basis states:

$$|x, y\rangle \xrightarrow{\mathcal{O}_f} |x, y \oplus f(x)\rangle \xrightarrow{\mathcal{O}_f} |x, y \oplus f(x) \oplus f(x)\rangle = |x, y\rangle.$$

With this, we are ready to define quantum random oracles.

Definition 10 (Quantum Random Oracle). An n -bit-to- m -bit Quantum Random Oracle (*QRO*), denoted $\mathcal{H}$, is a quantum-accessible function oracle $\mathcal{O}_f$ for a uniformly random function $f \in \mathcal{F}_{n \rightarrow m}$.

Remark 1. Classical random oracles typically follow the standard for hash functions in being defined from the space of *all* finite-length bit strings $\{0, 1\}^*$ to the space of fixed-length bitstrings $\{0, 1\}^m$. Technically, this means that f is not a function, but rather a collection of random functions f_ℓ , one for each input length ℓ . However, since quantum random oracles must operate by unitary means, we will make life easier for ourselves by explicitly restricting the input domain of the random function to be $\{0, 1\}^n$, and then just take for granted that any result trivially generalizes to collections of oracles with varying input length.

³⁰ Some works distinguish between quantum-accessible oracles, which evaluate a function on a superposition of inputs, and *quantum* oracles, which apply some quantum operation on the input registers. We do not distinguish between these two kinds of oracle, and rather view the former as a special case of the latter.

³¹ If any of this terminology confuses you, then this would be a good time to have a read through the quantum primer found in Sect. 5!

K: I'd move the quantum oracle part to the quantum section – we didn't explain qubits, registers, unitaries.... yet. We could place a pointer here that points to the respective section. (The only people who can parse this are the people who already know these concepts and thus wouldn't need the paragraph in the first place.)

To see why this is the case, observe that, even when loading up a hash function on real-world quantum hardware, the input *length* to the hash function may not be placed in superposition, as this would not correspond to a well-defined unitary operation. Intuitively speaking, every “branch” in a superposition must be defined over the same number of qubits. Or in other words: Adding and removing qubits to an operation is a classical control mechanism, and not itself a quantum operation!

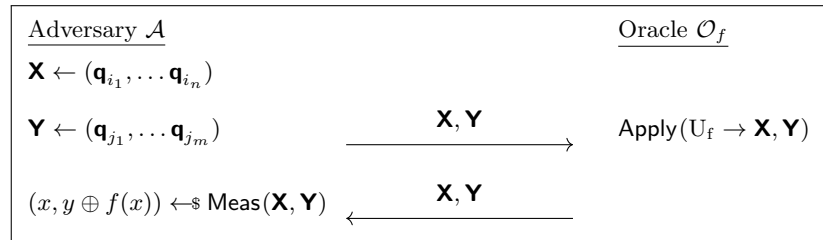


Fig. 13. An adversary $\mathcal{A}$ prepares the qubit registers $\mathbf{X}, \mathbf{Y}$, and sends them to a quantum-accessible function oracle $\mathcal{O}_f$, who applies unitary U to them and sends them back to the adversary. $\mathcal{A}$ then measures the registers to learn the values $(x, y \oplus f(x))$ corresponding to an element (x, y) in the initial superposition state of $\mathbf{X}, \mathbf{Y}$, chosen at random according to the distribution inferred by the quantum state.

6.3 Classical Oracles in a Quantum World

Sometimes, we want to provide a quantum algorithm with access to a classical oracle, but being unitary, we still face the issue of the operation $x \rightarrow f(x)$ not being unitary. The way to fix, and make classical oracles quantum-compatible, is to simply define classical oracles to be quantum oracles that first measure the input register, collapsing it to some basis state $|x\rangle$.

Definition 11 (Quantum-Compatible Classical Oracle). *A quantum-compatible classical oracle is a function oracle for f that, on input qubit registers $(\mathbf{X}, \mathbf{Y})$, first performs the computational-basis measurement $\text{Meas}(\mathbf{X})$, and then performs the operation $\text{Apply}(U_f \rightarrow \mathbf{X}, \mathbf{Y})$, before returning the registers to the sender. Here, f is the function defining the output of the classical oracle on input x , given any state and/or randomness internal to the oracle.*

Note that there is no need to collapse the output register $\mathbf{Y}$: If the input state is some superposition $\sum_i \alpha_i |x_i, y_i\rangle$, then after measuring the $\mathbf{X}$ register, it will have collapsed to the state $|x\rangle (\sum_i \beta_i |y_i\rangle)$ for some x , and some set of amplitudes $\{\beta_i\}$. Then the output state will be $|x\rangle (\sum_i \beta_i |y_i \oplus f(x)\rangle)$. Observe that the function f is still only evaluated on a single input, as desired.

With this definition in hand, we will go on to ignore these implementation details and write classical oracles, such as those appearing in our security games, in terms of their classical input/output interface.

6.4 Equivalent Implementations

Random oracles are typically treated as fully black-box, or “monolithic”, meaning that we don’t assume anything about their inner workings other than that it satisfies Def. 10. Still, it is instructive to see how a quantum random oracle *could* be implemented. Following are several possible implementations, which turn out to all be equivalent [Zha19].

Standard Oracles. This “standard” oracle, shown to the left in Fig. 14, is the oracle that simply implements our intuition from the start of the section: It first draws a function f from $\{0, 1\}^n$ to $\{0, 1\}^m$ uniformly at random, storing the whole truth table as one exponential-sized list in the internal state σ , and thereafter operates as the quantum-accessible function oracle $\mathcal{O}_f$.

Phase Oracles. The “phase” oracle, shown in the middle of Fig. 14, operates exactly like the standard oracle, except that it performs a phase operation on the output register rather than an xor. It therefore provides a different interface to the adversary, and is in that sense not equivalent in the sense of being indistinguishable, but *are* easily reducible to each other, as one kind of oracle becomes identical to the other if you just apply the Hadamard transformation $H^{\otimes m}$ to the output register $\mathbf{Y}$ before and after each query.

Standard oracle $\mathcal{F}(\mathbf{X}, \mathbf{Y})$	Phase oracle $\mathcal{G}(\mathbf{X}, \mathbf{Y})$	Superposition oracle $\mathcal{H}(\mathbf{X}, \mathbf{Y})$
if $\sigma = \varepsilon$:	if $\sigma = \varepsilon$:	if $\sigma = \varepsilon$:
$f \leftarrow \mathcal{F}_{n \rightarrow m}$	$g \leftarrow \mathcal{F}_{n \rightarrow m}$	$ \mathbf{H}\rangle \leftarrow \sum_{\forall h \in \mathcal{F}_{n \rightarrow m}} h\rangle$
$\sigma \leftarrow f$	$\sigma \leftarrow g$	$\sigma \leftarrow \mathbf{H}$
let U_f :	let U_g :	let U_h :
$ x, y\rangle \rightarrow x, y \oplus f(x)\rangle$	$ x, y\rangle \rightarrow (-1)^{y \cdot g(x)} x, y\rangle$	$ x, y\rangle h\rangle \rightarrow x, y \oplus h(x)\rangle h\rangle$
Apply ($U_f \rightarrow \mathbf{X}, \mathbf{Y}$)	Apply ($U_g \rightarrow \mathbf{X}, \mathbf{Y}$)	Apply ($U_h \rightarrow \mathbf{X}, \mathbf{Y}, \mathbf{H}$)
return $(\mathbf{X}, \mathbf{Y})$	return $(\mathbf{X}, \mathbf{Y})$	return $(\mathbf{X}, \mathbf{Y})$

Fig. 14. Various equivalent implementations of quantum random oracles. Here, σ is a state internal to the oracle, assumed initialized to the empty string ε , and $\mathbf{H}$ is a $(2^n \cdot m)$ -length qubit register.

Superposition Oracles. Instead of using a classically stochastic process to sample the random function at the outset, one can exploit the randomness inherent to quantum physics and simply provide a uniform superposition of *all* functions from $\{0, 1\}^n$ to $\{0, 1\}^m$. This “superposition” oracle, shown to the right in Fig. 14, is just as (in)efficiently implemented, and is completely indistinguishable from a standard oracle from the point of view of the adversary.

The reason, in a nutshell, is that when applying the unitary U_h , the oracle *entangles* the adversary with the gigantic superposition that it keeps. Hiding away entanglement in this manner has the effect, from the “local” point of view of the adversary, of introducing randomness into the state of $\mathbf{Y}$. In technical terms, the state will no longer be “pure” (but rather “mixed”), which could either be because of some hidden entanglement somewhere, or due to interference with a classically stochastic process—or both! However, there is no way to tell these cases apart without a full view of the combined system.

Of course, with the choice of interface being orthogonal to the choice of sampling vs. superposition, there is also a “superposition phase oracle”. We leave it to you to work out the details.

Compressed Oracles. It turns out that there is a procedure through which we can efficiently implement a superposition oracle, as described above, in a manner completely analogous to the ubiquitous “lazy-sampling” technique for classical random oracles. Introduced by Zhandry [Zha19], these “compressed oracles” are enabled by the crucial insight that a uniform superposition over possible outputs will simply be represented by the all-zero string after a transformation to the Hadamard domain. Thus, we only have to store the output values for inputs that are *not* in a uniform superposition, due to having been entangled to the adversary via a query, leading to an efficient simulation procedure.

Of course, the above is an oversimplification: We still haven’t said anything about how to deal with large superpositions over inputs, for example. To get there, and to see that what we end up with really is a faithful simulation, we will first need to lay some more groundwork. Sect. 12 is dedicated to this.

6.5 Domain-Separating Quantum Random Oracles

Sometimes, a construction uses more than one random oracle, which are then required to be independent. A standard way to solve this in practice is via *domain-separation* [BDG20]: Given a random oracle $\mathcal{H}$ from $\{0, 1\}^{n+1}$ to $\{0, 1\}^m$ (or a hash function like SHA3, if you are in the real world), let the random oracle $\mathcal{F}$ from $\{0, 1\}^n$ to $\{0, 1\}^m$ be defined as $\mathcal{F}(x) := \mathcal{H}(0\|x)$, and likewise let the random oracle $\mathcal{G}(x)$ be given by $\mathcal{G}(x) := \mathcal{H}(1\|x)$. Clearly, this works in the classical case, since the output of $\mathcal{H}$ is uniformly random on each input, so flipping even a single bit makes the outputs completely uncorrelated. Further, it should be clear that the other direction also holds: You don’t get anything *more* with oracle access to $\mathcal{H}$ directly than you do given oracle access to $\mathcal{F}$ and $\mathcal{G}$ separately.

But what about the quantum case? Now, we are supposed to be able to call the oracles in superposition, yet we can only call one oracle at a time. In particular, it seems that we can’t make a query that is “in a superposition of going to $\mathcal{F}$ and to $\mathcal{G}$ ”, even though this would be a trivial task given oracle access to $\mathcal{H}$. Is it conceivable that we can do *more* if we are given oracle access to $\mathcal{H}$ instead?

The following lemma answers this question in the negative: Domain-separation works just as well in the QROM as it does in the classical ROM—thankfully!³²

³² I’m grateful to Joseph Jaeger for showing me this reduction.

Lemma 5 (Two Oracles $\cong$ One Oracle). *Let $\mathcal{F}$ and $\mathcal{G}$ be quantum random oracles from $\{0,1\}^n$ to $\{0,1\}^m$, and let $\mathcal{H}$ be a quantum random oracle from $\{0,1\}^{n+1}$ to $\{0,1\}^m$. Then, for any adversary $\mathcal{A}$ that expects simultaneous oracle access to $\mathcal{F}$ and $\mathcal{G}$, there is a reduction $\mathcal{B}$ that, given oracle access to $\mathcal{H}$, provides $\mathcal{A}$ with a perfect simulation of $\mathcal{F}$ and $\mathcal{G}$.*

And vice versa: For any adversary $\mathcal{A}$ that expects oracle access to $\mathcal{H}$, there is a reduction $\mathcal{B}$ that, given simultaneous access to $\mathcal{F}$ and $\mathcal{G}$, provides $\mathcal{A}$ with a perfect simulation of $\mathcal{H}$.

Proof. The first direction is trivial, and works exactly like the classical case: Whenever $\mathcal{A}$ makes a call to $\mathcal{F}$ with registers $(\mathbf{X}, \mathbf{Y})$, $\mathcal{B}$ makes an oracle call to $\mathcal{H}$ with registers $(\mathbf{q} \parallel \mathbf{X}, \mathbf{Y})$, where $\mathbf{q}$ is a qubit placed in the state $|\mathbf{q}\rangle = |0\rangle$; and likewise for calls to $\mathcal{G}$, except with $|\mathbf{q}\rangle$ set to $|1\rangle$. The registers $(\mathbf{X}, \mathbf{Y})$ are then returned to $\mathcal{A}$ as usual.

For the other direction, let the controlled swap gate CSWAP be defined by the following operation on computational basis states (where $|x\rangle = |z\rangle$):

$$|0\rangle |x\rangle |z\rangle \xrightarrow{\text{CSWAP}} |0\rangle |x\rangle |z\rangle, \quad |1\rangle |x\rangle |z\rangle \xrightarrow{\text{CSWAP}} |0\rangle |z\rangle |x\rangle.$$

The reduction $\mathcal{B}$ sets up a new n -qubit register $\mathbf{Z}$, initialized to the uniform superposition over all $z \in \{0,1\}^n$. Then, on input a query to $\mathcal{H}$ with the registers $(\mathbf{q} \parallel \mathbf{X}, \mathbf{Y})$, $\mathcal{B}$ does the following:

1. Apply CSWAP on $\mathbf{X}$ and $\mathbf{Z}$, with qubit $\mathbf{q}$ as the control;
2. Send $(\mathbf{X}, \mathbf{Y})$ to $\mathcal{O}[F]$, and $(\mathbf{Z}, \mathbf{Y})$ to $\mathcal{O}[G]$;
3. Apply CSWAP a second time to the same registers;
4. Return $(\mathbf{X}, \mathbf{Y})$ to $\mathcal{A}$.

To see that this leads to the correct distribution, consider the result of applying this procedure to the first query if qubit $\mathbf{q}$ is in some superposition state $|\mathbf{q}\rangle = \alpha|0\rangle + \beta|1\rangle$, and $\mathbf{X}$ and $\mathbf{Y}$ are set to the computational basis states $|x\rangle_{\mathbf{X}} |y\rangle_{\mathbf{Y}}$. Then we start out with the combined state

$$\begin{aligned} & (\alpha|0\rangle_{\mathbf{q}} + \beta|1\rangle_{\mathbf{q}}) |x\rangle_{\mathbf{X}} |y\rangle_{\mathbf{Y}} \left(\sum_{z \in \{0,1\}^n} \frac{1}{\sqrt{2^n}} |z\rangle_{\mathbf{Z}} \right) \\ &= \frac{1}{\sqrt{2^n}} \sum_{z \in \{0,1\}^n} \alpha |0\rangle_{\mathbf{q}} |x\rangle_{\mathbf{X}} |y\rangle_{\mathbf{Y}} |z\rangle_{\mathbf{Z}} + \beta |1\rangle_{\mathbf{q}} |x\rangle_{\mathbf{X}} |y\rangle_{\mathbf{Y}} |z\rangle_{\mathbf{Z}}. \end{aligned}$$

After applying the first CSWAP, we get the state

$$\xrightarrow{1.} \frac{1}{\sqrt{2^n}} \sum_{z \in \{0,1\}^n} \left(\alpha |0\rangle_{\mathbf{q}} |x\rangle_{\mathbf{X}} |y\rangle_{\mathbf{Y}} |z\rangle_{\mathbf{Z}} + \beta |1\rangle_{\mathbf{q}} |z\rangle_{\mathbf{X}} |y\rangle_{\mathbf{Y}} |x\rangle_{\mathbf{Z}} \right).$$

At this point, the registers $(\mathbf{X}, \mathbf{Y})$ are sent to $\mathcal{F}$, and then registers $(\mathbf{Z}, \mathbf{Y})$ are sent to $\mathcal{G}$, with the resulting state

$$\xrightarrow{2.} \frac{1}{\sqrt{2^n}} \sum_{z \in \{0,1\}^n} \left(\alpha |0\rangle_{\mathbf{q}} |x\rangle_{\mathbf{X}} |y \oplus \mathcal{F}(x) \oplus \mathcal{G}(z)\rangle_{\mathbf{Y}} |z\rangle_{\mathbf{Z}} + \beta |1\rangle_{\mathbf{q}} |z\rangle_{\mathbf{X}} |y \oplus \mathcal{F}(z) \oplus \mathcal{G}(x)\rangle_{\mathbf{Y}} |x\rangle_{\mathbf{Z}} \right).$$

Applying CSWAP a second time gives us

$$\xrightarrow{3.} \frac{1}{\sqrt{2^n}} \sum_{z \in \{0,1\}^n} \left(\alpha |0\rangle_{\mathbf{q}} |x\rangle_{\mathbf{X}} |y \oplus \mathcal{F}(x) \oplus \mathcal{G}(z)\rangle_{\mathbf{Y}} |z\rangle_{\mathbf{Z}} + \beta |1\rangle_{\mathbf{q}} |x\rangle_{\mathbf{X}} |y \oplus \mathcal{F}(z) \oplus \mathcal{G}(x)\rangle_{\mathbf{Y}} |z\rangle_{\mathbf{Z}} \right),$$

at which point the registers $\mathbf{X}$ and $\mathbf{Y}$ are returned to $\mathcal{A}$.

Note that register $\mathbf{Y}$ ends up entangled with register $\mathbf{Z}$. As explained when discussing superposition oracles, hidden entanglement induces a classical probability distribution when it is hidden from view, and maximal entanglement with a hidden uniform superposition is indistinguishable from the uniform probability distribution. Since the values of $\mathcal{F}$ and $\mathcal{G}$ are xored into the output state, this uniformity carries over; and yet, again thanks to entanglement, we see that we are guaranteed consistency upon repeat queries.

Since the correct behaviour is exhibited on all input basis states $|b\rangle|x, y\rangle$, thanks to the linearity of quantum mechanics, we may conclude that the simulation exhibits the correct behaviour on all input states. $\square$

6.6 When ROM Security Implies QROM Security

Hans: Essentially whenever the random oracle could just as well be instantiated with a standard-model hash function.

Part III

Non-Adaptive Programming

7 The One-Way to Hiding (O2H) Lemmas

rough structure

1. (assuming the classical proofs were shown in previous sections)
2. From OW-CPA to IND-CPA via the U(check)-transformation
 - (a) First via Unruh’s original O2H
 - (b) then tighter via the semi-classical “find” O2H
 - (c) then tighter in one direction via the double-sided O2H
 - (d) then tighter in another direction via Measure-Rewind-Measure
 - (e) and finally (since we assume dPKE) Measure-Rewind-Extract
3. Introduce the fixed-permutation formulation?
4. From NMA NIZK to (s?)EUF-CMA signature via adaptively-reprogrammable O2H

Ever since their first introduction by Unruh [Unr14b], the One-Way to Hiding (O2H) lemmas have served as indispensable tools for cryptographers seeking to prove security in the quantum random oracle model. Note the plural: Over the years, the original O2H lemma, which only introduced a very limited capability of reprogramming QROs, has been repeatedly generalized and expanded, often to different and seemingly-incomparable settings. Thus, there is not one “multi-tool” O2H Lemma, but several variants that together make up a “toolkit”—and, as with any good toolkit, different tools are best fit for different jobs!

In the following sections, we will introduce the different variants in increasing generality and complexity, starting with the very simplest variant, and showing along the way how to apply each to a security reduction. An overview of the O2H lemmas that will be presented is provided in Table 2.³³

7.1 A Classical One-Way to Hiding Lemma

Hans: Extract both “find” and “guess” versions from the above proofs.

Hans: Add dot for the double-sided requirement, which MRM (and MRE?) also comes with, with the single-sided case still an open question?

7.2 Guess versus Find

Let us show that the extractor having access to both $\mathcal{H}$ and $\mathcal{G}$ is equivalent to the extractor having access to a quantum-accessible indicator function.

Lemma 6 (Double-Sided Oracle Access $\Leftrightarrow$ Indicator Oracle Access). *Let $\mathcal{S} \subseteq \mathcal{X}$, let $\mathcal{H}, \mathcal{G} : \mathcal{X} \rightarrow \mathcal{Y}$ be classical random oracles satisfying $\forall x \notin \mathcal{S} : \mathcal{H}(x) = \mathcal{G}(x)$, and let $\mathbb{1}$ be an indicator oracle for $\mathcal{S}$. A quantum oracle algorithm with oracle access $\mathbb{1}$ can faithfully simulate $\mathcal{H}$ and $\mathcal{G}$, and a quantum oracle algorithm with oracle access to $\mathcal{H}$ and $\mathcal{G}$ can simulate $\mathbb{1}$, up to collisions in $\mathcal{S}$.*

Proof (sketch). First direction: The algorithm maintains two independent simulations of random oracles using the compressed random oracle technique; call them $\mathcal{H}_1$ and $\mathcal{H}_2$. Upon receiving a query to $\mathcal{H}$, the algorithm responds using $\mathcal{H}_1$. Upon receiving a query to $\mathcal{G}$, the algorithm computes the output of both $\mathcal{H}_1$ and $\mathcal{H}_2$, storing each in separate output registers, then uses $\mathbb{1}$ to coherently implement a control for a CSWAP operation that swaps the first and second output register. The simulated $\mathcal{G}$ then behaves like $\mathcal{H}_1$ on inputs $x \in \mathcal{S}$, and like $\mathcal{H}_2$ on inputs $x \notin \mathcal{S}$.

Second direction: Recall that the indicator oracle operates on an n -length input register and a single-qubit output register, which is flipped if the input x is in $\mathcal{S}$. To simulate $\mathbb{1}$, the algorithm pairs the input register with an m -length output register initialized to $|0^m\rangle$, then first calls $\mathcal{H}$, followed by $\mathcal{G}$. The resulting state will have basis states $|x, \mathcal{H}(x) \oplus \mathcal{G}(x)\rangle$. Using CNOTs to implement an OR-function, the output bit is then flipped iff the state of the output is not equal to $|0^m\rangle$. Thus, we get an oracle that indicates whether $\mathcal{H}(x)$ is unequal to $\mathcal{G}(x)$, which is identical to the indicator function for $\mathcal{S}$ up to collisions in $\mathcal{S}$ (for which the simulation will return a false negative). Before returning the result, the state of the output register is uncomputed via one more call each to $\mathcal{H}$ and $\mathcal{G}$, removing unwanted entanglement.

Hans: To-do: Quantify both the loss due to collisions and due to any residue entanglement.

³³ There are other historical variants of the O2H lemma that are not covered here, due later variants being strict generalisations. These include (...)

Table 2. An overview of the various One-way to Hiding (O2H) lemmas. Here, Q = Quantum-accessible oracles, M = Multi-point reprogramming, A = Adaptive reprogramming, B = fully (as opposed to rewindable) Black-box access to the distinguisher, C = Computationally (as opposed to statistically) hidden reprogramming, L = allows Leaking information about reprogrammed points after reprogramming, and G = Guess, meaning the extractor does not have access to an indicator function for the target value(s). q bounds the total number of oracle calls, and r bounds the number of reprogrammed points. Variants in paranthesis have been superseded by later results, and are included mainly for historical reasons. Most bounds presented here are simplified and/or approximate, removing sums and ignoring constant factors and information-theoretic terms; for the exact statement, see the respective Lemma. Finally, to make for easier comparison, the “Set λ to” column gives a numerical example for how much one would have to increase the security parameter if one wants to bound the distinguishing advantage by 2^{-128} , assuming 1) that the extractors advantage is bounded by $\varepsilon \leq 2^{-\lambda}$, 2) $q \leq 2^{80}$ queries to the random oracle (a common estimate for the total number of SHA3 evaluations over the course of its lifetime), and 3) $r \leq 2^{10}$ reprogrammed points (following the example of Bellare et al. [BRTZ24], who bounded the number of user corruptions similarly); the closer to the optimal value $\lambda = 128$ the better.

Ref.	O2H Variant	Guessing	Quantum	Multi-point	Adaptive	Computational	Leaking	Black-box	Bound (simplified)	Set λ to
[BR93]	Classical O2H (guess)	●	○	●	●	●	●	●	$\leq q\varepsilon$	208
[BR93]	Classical O2H (search)	○	○	●	●	●	●	●	$\leq \varepsilon$	128
[Unr14b]	Original O2H	●	●	○	○	●	●	●	$\lesssim q\sqrt{\varepsilon}$	416
[Unr14a]	“Adaptive” O2H	●	●	○	●	●	●	●	$\lesssim q\sqrt{\varepsilon}$	416
[AHU19]	“Semi-Classical” O2H (guess)	●	●	●	○	●	●	●	$\lesssim q\sqrt{\varepsilon}$	416
[AHU19]	“Semi-Classical” O2H (search)	○	●	●	○	●	●	●	$\lesssim \sqrt{q\varepsilon}$	336
[GHHM21]	Resampling Lemma	●	●	●	●	○	●	●	$\lesssim r\sqrt{q\varepsilon}$	356
[PZ24]	Comp. Ad. Reprog. Framew.	●	●	●	●	●	○	●	$\lesssim rq\sqrt{\varepsilon}$	436
[Jae24]	“Fixed-Permutation” O2H	●	●	●	●	●?	●?	●	$\lesssim \sqrt{q\varepsilon}?$	336?
[BHH ⁺ 19]	“Double-sided” O2H	○	●	○	○	●	●	●	$\lesssim \sqrt{\varepsilon}$	256
[KSS ⁺ 20]	Meas.-Rewind-Meas.	○	●	●	○	●	●	○	$\lesssim q\varepsilon$	208
[GLX24]	Meas.-Rewind-Ext.	○	●	●	○	●	●	○	$\lesssim \sqrt{q\varepsilon}$	168

7.3 Non-Adaptive State of the Art

In the lemmas below, $\mathcal{H}$, $\mathcal{G}$, $\mathcal{S}$ (or x^*), and z are random with arbitrary joint distributions.

Lemma 7 (Classical O2H [BR93]). *Let $\mathcal{S} \subseteq \mathcal{X}$, let $\mathcal{H}, \mathcal{G} : \mathcal{X} \rightarrow \mathcal{Y}$ be classical random oracles satisfying $\forall x \notin \mathcal{S} : \mathcal{H}(x) = \mathcal{G}(x)$, let $\mathbb{1}$ be an indicator oracle for $\mathcal{S}$, and let z be arbitrary. Then there are SFBB extractors $\mathcal{E}_{\text{find}}, \mathcal{E}_{\text{guess}}$ such that, for all distinguishers $\mathcal{D}$ making at most $\mathcal{H}$ calls to its random oracle,*

$$|\Pr[1 \leftarrow \mathcal{D}^{\mathcal{H}}(z)] - \Pr[1 \leftarrow \mathcal{D}^{\mathcal{G}}(z)]| \leq \Pr[x \leftarrow \mathcal{E}_{\text{find}}^{\mathbb{1}}(z) : x \in \mathcal{S}],$$

and

$$|\Pr[1 \leftarrow \mathcal{D}^{\mathcal{H}}(z)] - \Pr[1 \leftarrow \mathcal{D}^{\mathcal{G}}(z)]| \leq \mathcal{H} \cdot \Pr[x \leftarrow \mathcal{E}_{\text{guess}}(z) : x \in \mathcal{S}].$$

The extractors run in time that of $\mathcal{D}$ plus the overhead of simulating at most $\mathcal{H}$ oracle calls, and $\mathcal{E}_{\text{find}}$ additionally makes at most $\mathcal{H}$ calls to its indicator oracle.

Lemma 8 (Original O2H Lemma [Unr14b]). *Let $\mathcal{H}, \mathcal{G} : \mathcal{X} \rightarrow \mathcal{Y}$ be random oracles satisfying $\forall x \neq x^* : \mathcal{H}(x) = \mathcal{G}(x)$ (and independent otherwise), and let z be a value that depends arbitrarily on $\mathcal{H}, \mathcal{G}$, and x^* . Then there is an SFBB extractor $\mathcal{E}$ such that, for all distinguishers $\mathcal{D}$ making at most $\mathcal{H}$ calls to its random oracle,*

$$|\Pr[1 \leftarrow \mathcal{D}^{\mathcal{H}}(z)] - \Pr[1 \leftarrow \mathcal{D}^{\mathcal{G}}(z)]| \leq 2\mathcal{H}\sqrt{\Pr[x^* \leftarrow \mathcal{E}(z)]},$$

where the extractor runs in time that of $\mathcal{D}$, plus the overhead of simulating at most $\mathcal{H}$ oracle responses.

Lemma 9 (Multi-Point O2H [AHU19]). Let $\mathcal{S} \subseteq \mathcal{X}$, let $\mathcal{H}, \mathcal{G} : \mathcal{X} \rightarrow \mathcal{Y}$ be quantum random oracles satisfying $\forall x \notin \mathcal{S} : \mathcal{H}(x) = \mathcal{G}(x)$, let $\mathbb{1}$ be a quantum-accessible indicator oracle for $\mathcal{S}$, and let z be arbitrary. Then there are SFBB extractors $\mathcal{E}_{\text{find}}, \mathcal{E}_{\text{guess}}$ such that, for all distinguishers $\mathcal{D}$ making at most $\mathcal{H}$ calls to its random oracle, $\forall e \in \{1, 1/2\}$,

$$\left| \Pr[1 \leftarrow_{\$} \mathcal{D}^{\mathcal{H}}(z)]^e - \Pr[1 \leftarrow_{\$} \mathcal{D}^{\mathcal{G}}(z)]^e \right| \leq 2\sqrt{(\mathcal{H} + 1) \cdot \Pr[x \leftarrow_{\$} \mathcal{E}_{\text{find}}^{\mathbb{1}}(z) : x \in \mathcal{S}]},$$

and

$$\left| \Pr[1 \leftarrow_{\$} \mathcal{D}^{\mathcal{H}}(z)]^e - \Pr[1 \leftarrow_{\$} \mathcal{D}^{\mathcal{G}}(z)]^e \right| \leq 2\mathcal{H}\sqrt{\Pr[x \leftarrow_{\$} \mathcal{E}_{\text{guess}}(z) : x \in \mathcal{S}]}.$$

The extractors run in time that of $\mathcal{D}$ plus the overhead of simulating at most $\mathcal{H}$ oracle responses, and $\mathcal{E}_{\text{find}}$ additionally makes at most $\mathcal{H}$ calls to its indicator oracle.

Lemma 10 (Single-Point O2H, Find [BHH⁺19]). Let $x^* \in \mathcal{X}$, let $\mathcal{H}, \mathcal{G} : \mathcal{X} \rightarrow \mathcal{Y}$ be quantum random oracles satisfying $\forall x \neq x^* : \mathcal{H}(x) = \mathcal{G}(x)$, let $\mathbb{1}$ be a quantum-accessible indicator oracle for x^* , and let z be arbitrary. Then there is an SFBB extractor $\mathcal{E}$ such that, for all distinguishers $\mathcal{D}$ making at most $\mathcal{H}$ calls to its random oracle, $\forall e \in \{1, 1/2\}$,

$$\left| \Pr[1 \leftarrow_{\$} \mathcal{D}^{\mathcal{H}}(z)]^e - \Pr[1 \leftarrow_{\$} \mathcal{D}^{\mathcal{G}}(z)]^e \right| \leq 2\sqrt{\Pr[x^* \leftarrow_{\$} \mathcal{E}^{\mathbb{1}}(z)]},$$

where $\mathcal{E}$ runs in time that of $\mathcal{D}$ plus the overhead of simulating at most $2\mathcal{H}$ oracle responses, and makes at most $\mathcal{H}$ calls to its indicator oracle.

Lemma 11 (Multi-Point O2H with Rewinding, Find [GLX24]). Let $\mathcal{S} \subseteq \mathcal{X}$, let $\mathcal{H}, \mathcal{G} : \mathcal{X} \rightarrow \mathcal{Y}$ be quantum random oracles satisfying $\forall x \notin \mathcal{S} : \mathcal{H}(x) = \mathcal{G}(x)$, let $\mathbb{1}$ be a quantum-accessible indicator oracle for $\mathcal{S}$, and let z be arbitrary. Then there is an RBB extractor $\mathcal{E}$ such that, for all distinguishers $\mathcal{D}$ making at most $\mathcal{H}$ calls to its random oracle,

$$\left| \Pr[1 \leftarrow_{\$} \mathcal{D}^{\mathcal{H}}(z)] - \Pr[1 \leftarrow_{\$} \mathcal{D}^{\mathcal{G}}(z)] \right| \leq 4\sqrt{\mathcal{H}} \cdot \Pr[x \leftarrow_{\$} \mathcal{E}^{\mathbb{1}}(z) : x \in \mathcal{S}],$$

where $\mathcal{E}$ runs in time thrice that of $\mathcal{D}$ (rewinding twice), plus the overhead of simulating at most $6\mathcal{H}$ oracle responses, and makes at most $3\mathcal{H}$ calls to its indicator oracle.

Hans: MRM required access to both $\mathcal{H}$ and $\mathcal{G}$; exchanged for indicator function, making MRE strictly more general. Also double-check that it is rewinding twice.

Hans: Attempt at putting them all in one lemma

Lemma 12 (Non-Adaptive O2H, State of the Art). Let $\mathcal{H}, \mathcal{G} : \mathcal{X} \rightarrow \mathcal{Y}$ be quantum random oracles, let $\mathcal{S} \subseteq \mathcal{X}$ be such that $\forall x \notin \mathcal{S} : \mathcal{H}(x) = \mathcal{G}(x)$, let $\mathbb{1}$ be a quantum-accessible indicator oracle for $\mathcal{S}$, and let z be a value that depends arbitrarily on $\mathcal{H}, \mathcal{G}$ and $\mathcal{S}$. Let $\mathcal{D}$ be a distinguisher that makes at most $\mathcal{H}$ calls to its random oracle. Then,

1. **(Classical guess)** There is an SFBB extractor $\mathcal{E}$ such that, for all $\mathcal{D}$ that only make classical calls to the random oracle,

$$\left| \Pr[1 \leftarrow_{\$} \mathcal{D}^{\mathcal{H}}(z)] - \Pr[1 \leftarrow_{\$} \mathcal{D}^{\mathcal{G}}(z)] \right| \leq \mathcal{H} \cdot \Pr[x \leftarrow_{\$} \mathcal{E}(z) : x \in \mathcal{S}],$$

where $\mathcal{E}$ runs in time that of $\mathcal{D}$ plus the overhead of simulating at most $\mathcal{H}$ oracle responses.

2. **(Classical search)** There is an SFBB extractor $\mathcal{E}$ such that, for all $\mathcal{D}$ that only make classical calls to the random oracle,

$$\left| \Pr[1 \leftarrow_{\$} \mathcal{D}^{\mathcal{H}}(z)] - \Pr[1 \leftarrow_{\$} \mathcal{D}^{\mathcal{G}}(z)] \right| \leq \Pr[x \leftarrow_{\$} \mathcal{E}^{\mathbb{1}}(z) : x \in \mathcal{S}],$$

where $\mathcal{E}$ runs in time that of $\mathcal{D}$ plus the overhead of simulating at most $\mathcal{H}$ oracle responses, and makes at most $\mathcal{H}$ calls to its indicator oracle.

3. **(Multi-point guess)** There is an SFBB extractor $\mathcal{E}$ such that, for all $\mathcal{D}$ and $e \in \{1, 1/2\}$,

$$\left| \Pr[1 \leftarrow_{\$} \mathcal{D}^{\mathcal{H}}(z)]^e - \Pr[1 \leftarrow_{\$} \mathcal{D}^{\mathcal{G}}(z)]^e \right| \leq 2\mathcal{H}\sqrt{\Pr[x \leftarrow_{\$} \mathcal{E}(z) : x \in \mathcal{S}]},$$

where $\mathcal{E}$ runs in time that of $\mathcal{D}$ plus the overhead of simulating at most $\mathcal{H}$ oracle responses.

4. (**Multi-point search**) There is an SFBB extractor $\mathcal{E}$ such that, for all $\mathcal{D}$ and $e \in \{1, 1/2\}$,

$$\left| \Pr[1 \leftarrow \$ \mathcal{D}^{\mathcal{H}}(z)]^e - \Pr[1 \leftarrow \$ \mathcal{D}^{\mathcal{G}}(z)]^e \right| \leq 2\sqrt{(\mathcal{H} + 1) \cdot \Pr[x \leftarrow \$ \mathcal{E}^1(z) : x \in \mathcal{S}]},$$

where $\mathcal{E}$ runs in time that of $\mathcal{D}$ plus the overhead of simulating at most $\mathcal{H}$ oracle responses, and makes at most $\mathcal{H}$ calls to its indicator oracle.

5. (**Single-point search**) For $\mathcal{S} = \{x^*\}$, there is an SFBB extractor $\mathcal{E}$ such that, for all $\mathcal{D}$ and $e \in \{1, 1/2\}$,

$$\left| \Pr[1 \leftarrow \$ \mathcal{D}^{\mathcal{H}}(z)]^e - \Pr[1 \leftarrow \$ \mathcal{D}^{\mathcal{G}}(z)]^e \right| \leq 2\sqrt{\Pr[x^* \leftarrow \$ \mathcal{E}^1(z)]},$$

where $\mathcal{E}$ runs in time that of $\mathcal{D}$ plus the overhead of simulating at most $2\mathcal{H}$ oracle responses, and makes at most $\mathcal{H}$ calls to its indicator oracle.

6. (**Multi-point search with rewinding**) There is an RBB extractor $\mathcal{E}$ such that, for all $\mathcal{D}$,

$$\left| \Pr[1 \leftarrow \$ \mathcal{D}^{\mathcal{H}}(z)] - \Pr[1 \leftarrow \$ \mathcal{D}^{\mathcal{G}}(z)] \right| \leq 4\sqrt{\mathcal{H}} \cdot \Pr[x \leftarrow \$ \mathcal{E}^1(z) : x \in \mathcal{S}],$$

where $\mathcal{E}$ runs in time at most thrice that of $\mathcal{D}$ (rewinding twice), plus the overhead of simulating at most $6\mathcal{H}$ oracle responses, and makes at most $3\mathcal{H}$ calls to its indicator oracle.

7.4 Making the Lemmas Adaptive

8 Interlude: Understanding Tightness Losses

Hans: Either insert after O2H section as an “Interlude”, or leave as an appendix.

Hans: Note: The brute-force setting will still be relevant in a world with cryptographically-relevant quantum computers, given how much cheaper classical attack resources are. (And also, for many of the relevant problems, the best classical attacks see little speed-up from Grover.)

8.1 Kinds of Loss: Additive vs. Multiplicative vs. Roots

Say you have a construction built on top of a primitive Π , that incurs some tightness loss. For illustration, we study how this tightness loss affects security under the assumption that the best-known attack against the primitive is either brute-force search (linear, so $\epsilon_\Pi = t/2^{-\lambda}$), Grover search or (classical) collision-finding (quadratic, $\epsilon_\Pi = t^2/2^{-\lambda}$), or quantum collision-finding (cubic, $\epsilon_\Pi = t^3/2^{-\lambda}$).

??-?? show how different kinds of tightness losses degrade the provable security of our construction, under different assumptions. Here, the primitive Π is assumed to achieve 128-bit security against various assumed-optimal attacks: Brute-force search in ??, Grover search and/or classical collision-finding in ??, and quantum collision-finding in ??. Table 3 shows how high one would have to set the security parameter λ of the construction in order to sufficiently compensate for the tightness so that the construction achieves 128-bit security.

Note that this is not a particularly realistic setting, as even primitives that are considered highly secure (such as AES) usually face attacks that are significantly better than the generic ones we consider here. Further, different attacks strategies may be optimal for different ranges of the time budget t .

Hans: OLD CAPTION 1: The effect of an additive loss $q/2^n \leq 2^{80-128} = 2^{-48}$.

Hans: OLD CAPTION 2: Attack advantages ruled out by a reduction from the security of a primitive Π to the security of a construction, if the reduction incurred a $q \leq 2^{80}$ loss factor in the advantage (and no loss in the running time) (top), and the bit-security implied for the construction (bottom), assuming that the best attack on the primitive is scales linearly as in a brute-force search (left) and quadratically as with Grover search and collision-finding (right). We assume that the primitive achieves 128-bit security against the relevant attack, and display the ruled-out advantages/bit security as a function of the bound on the adversarial run-time t , to highlight the time-probability trade-off that adversaries generally face. (So for example, if high t gives lower bit-security, the adversary will be incentivized to use all the time available to it, while if the lowest bit-security sits at low t , the optimal attack may involve performing the same low-advantage attack a large number of times in parallel.)

Hans: OLD CAPTION 3: Same as ??, except if the reduction incurred a square-root loss instead of a factor q . Interestingly, if the best attack on the primitive already scales quadratically, then a square-root yields the same bit-security, the only difference being that now we can’t rule out matching attacks for any values of t (where previously the only matching attack was at $t = t_{\max} = 2^{128}$). Contrast with the linear-scaling case, where a square-root loss would halve the bit-security.

8.2 Understanding the O2H Losses

See Table 3 and Fig. 17.

Table 3. Compensating for tightness losses: Assuming the primitive Π achieves 128-bit security against Attack (and assuming that Attack is the most effective attack against the primitive), how large should we set the security parameter of the construction to compensate for the security loss and provably achieve 128-bit security? Here, as elsewhere, we assume $q \leq 2^{80}$. Note how once we assume a primitive attack that scales like $\epsilon \propto t^c$ for $c \geq 2$, then a square-root loss no longer needs to be compensated for.

For 128-bit security against Attack on Π , set λ to ...				
O2H Variant	Tightness loss $\epsilon \lesssim \dots$	Brute-force search $\Rightarrow \epsilon_\Pi \propto t, \lambda_\Pi = 128$	Grover search/collision finding $\Rightarrow \epsilon_\Pi \propto t^2, \lambda_\Pi = 256$	Quantum collision finding $\Rightarrow \epsilon_\Pi \propto t^3, \lambda_\Pi = 384$
Classical Find	ϵ_Π	$\lambda := 128 (= \lambda_\Pi)$	$\lambda := 256 (= \lambda_\Pi)$	$\lambda := 384 (= \lambda_\Pi)$
MRE-Find	$\sqrt{q} \cdot \epsilon_\Pi$	$\lambda := 168 (= \lambda_\Pi + 40)$	$\lambda := 296 (= \lambda_\Pi + 40)$	$\lambda := 424 (= \lambda_\Pi + 40)$
Classical Guess	$q \cdot \epsilon_\Pi$	$\lambda := 208 (= \lambda_\Pi + 80)$	$\lambda := 336 (= \lambda_\Pi + 80)$	$\lambda := 464 (= \lambda_\Pi + 80)$
DS-Find	$\sqrt{\epsilon_\Pi}$	$\lambda := 256 (= 2 \cdot \lambda_\Pi)$	$\lambda := 256 (= \lambda_\Pi)$	$\lambda := 384 (= \lambda_\Pi)$
SC-Find	$\sqrt{q \cdot \epsilon_\Pi}$	$\lambda := 336 (= 2 \cdot \lambda_\Pi + 80)$	$\lambda := 336 (= \lambda_\Pi + 80)$	$\lambda := 464 (= \lambda_\Pi + 80)$
Guess	$q\sqrt{\epsilon_\Pi}$	$\lambda := 416 (= 2 \cdot (\lambda_\Pi + 80))$	$\lambda := 416 (= \lambda_\Pi + 2 \cdot 80)$	$\lambda := 544 (= \lambda_\Pi + 2 \cdot 80)$

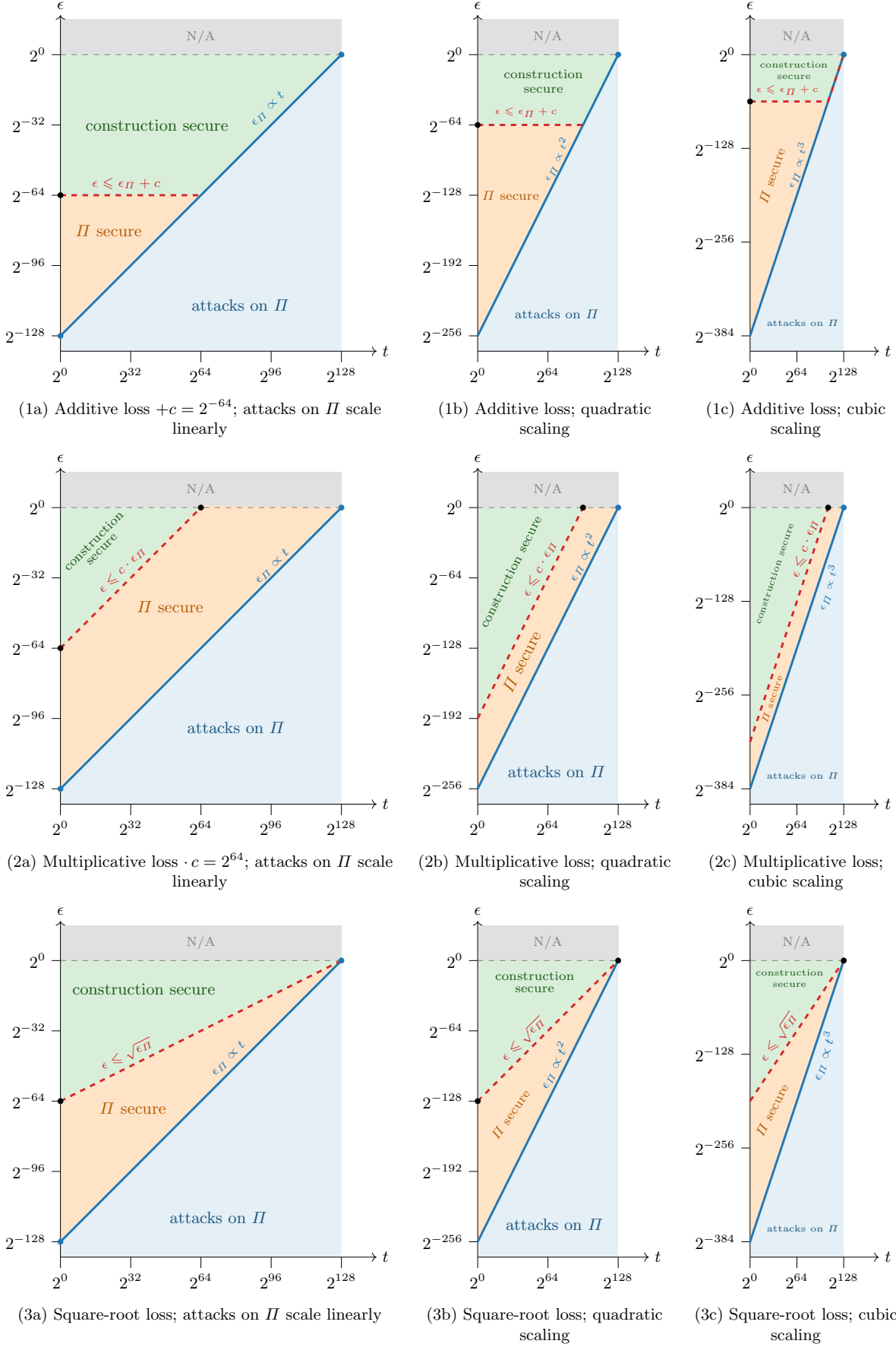


Fig. 15. The t -vs.- ϵ attack landscape for a construction built from some primitive Π , if the security reduction incurred 1) the additive loss $+c = 2^{-64}$ (top row), 2) the multiplicative loss $\cdot c = 2^{64}$ (middle row), or 3) a square-root loss, assuming that the optimal advantage against Π scales a) linearly with time, e.g. brute-force search (left column), b) quadratically with time, e.g. Grover search/classical collision-finding (middle column), and c) cubically with time, e.g. quantum collision-finding (right column). We assume that Π achieves 128-bit security against the relevant attack type, and that the reduction did not incur any significant increase in runtime. Dots mark the corresponding bit-security, see Fig. 16.

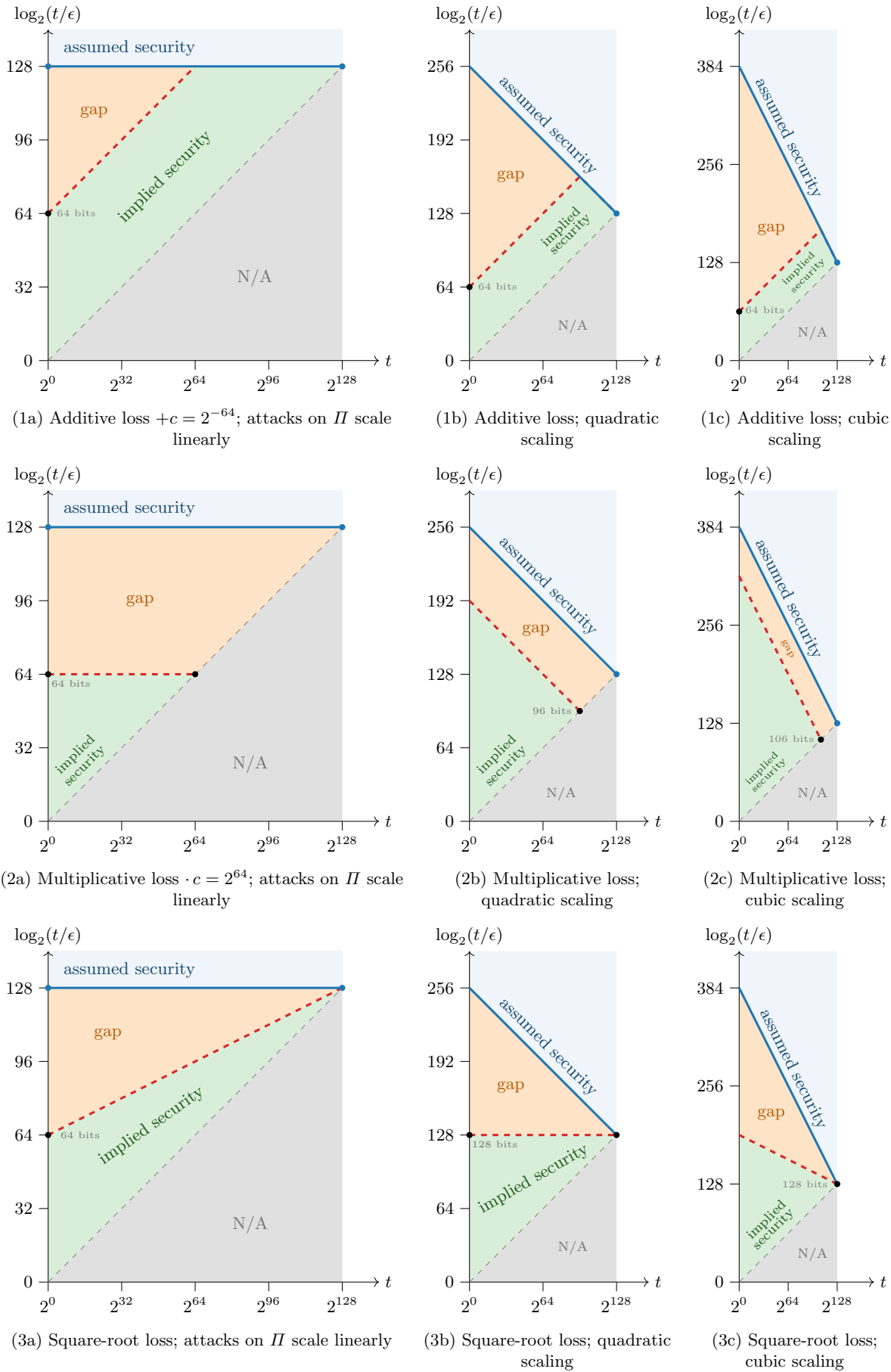


Fig. 16. The bit-security of the primitive Π for each kind of assumed-best attack, and the implied bit-security of the construction for a given reduction type, as a function of the upper-bound t on the run-time of the adversary. The panels correspond to the panels of Fig. 15, and dots mark the assumed/implied bit-security when $t \leq 2^{128}$. Observe how a square-root loss does not lead to a loss in bit-security once the underlying assumed-optimal attack scales quadratically or better.

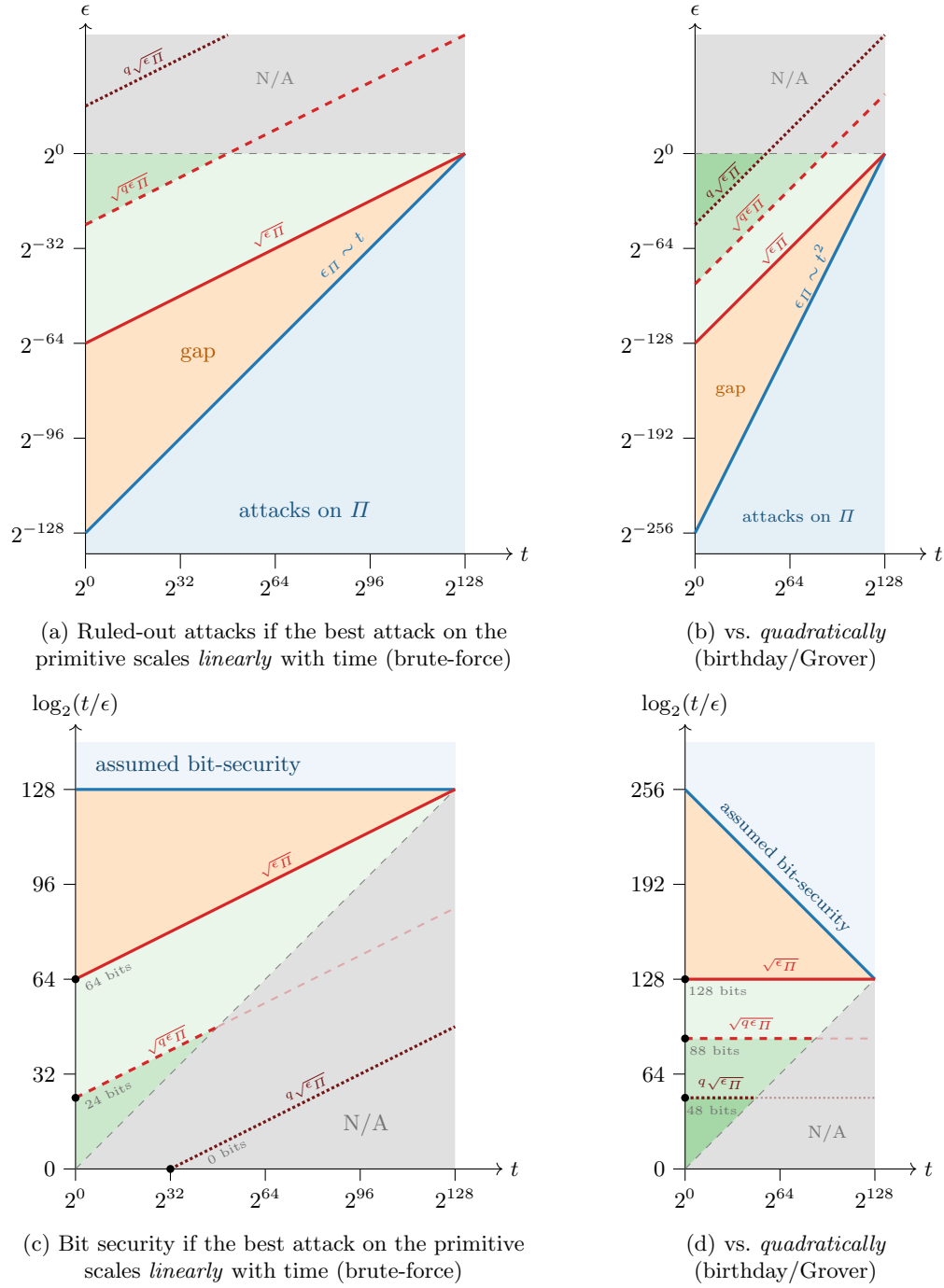


Fig. 17. Same as ?? and ??, except if the reduction incurred each of the O2H losses $\epsilon \sim \sqrt{\epsilon\Pi}$ (line), $\epsilon \sim \sqrt{q \cdot \epsilon\Pi}$ (line), $\epsilon \sim q\sqrt{\epsilon\Pi}$ (line). Observe how, if parameter were chosen to resist a linearly-scaling attack with 128-bit security, then the tightness loss is large enough to completely void the reduction bound, meaning the theorem would imply no security for the construction in this case.

9 Example 1b: Post-Quantum Indistinguishability from the $\mathbf{U}$ -transform

10 Interlude: A Quantum Register Pseudocode

Hans: moved to after QRO section; to be rewritten. (Merge the two sections?)

Hans: “Oracle pseudocode” because the main utility is the ability to describe operations on registers in unknown states (and to not (for now) oversell our claims of utility)? Also possible with circuit diagrams (and in fact examples of how to write a set of operations using both would probably be good), but this is where Dirac notation falls short.

Hans: Missing from primer: How to use projectors for coherent if-else logic. Introduce if/when needed (to make the pseudocode formal?)

We finally have everything we need to define our quantum pseudocode. Our goal is to construct a well-defined pseudocode rich enough to draw interaction diagrams like the one in Fig. 18, with local operations agnostic to the global state of the system and any existing entanglement between its parts. Rather than quantum states, the central objects of our pseudocode will be *qubit registers*, which is nothing but an ordered list of qubits, in analogy to the bit registers of classical computation.

Table 4. An overview of our quantum pseudocode notation.

Notation	Example usage	Equivalent to
R	$ \psi\rangle_{\mathbf{R}}$	Qubit register (list of qubits)
U	$\mathbf{U}(\mathbf{X})$	Apply unitary operation to register: $ x\rangle \rightarrow \mathbf{U} x\rangle$
$\leftarrow$	$\mathbf{X} \leftarrow x\rangle$	Initialize new register to a known state: $ \mathbf{X}\rangle := 0^{ x }\rangle \rightarrow x\rangle$
$\emptyset$	$\mathbf{A} \leftarrow \emptyset$	Discard register (NB: Register must first be returned to a pure state!)
$\leftarrow \oplus$	$\mathbf{Y} \leftarrow \oplus f(\mathbf{X})$	XOR function evaluation into register: $ x\rangle y\rangle \rightarrow x\rangle y \oplus f(x)\rangle$
$\leftrightarrow$	$\mathbf{A} \leftrightarrow \mathbf{B}$	Apply the SWAP unitary on two registers: $ x\rangle y\rangle \rightarrow y\rangle x\rangle$
$\leftarrow \text{I}$	$x \leftarrow \text{I} \mathbf{X}$	Measure register in the computational basis and store the result
for branch (value) of (register)	for branch x of $\mathbf{X}$:	Let the following unitary operations be controlled on the register’s state being equal to basis state
	$\mathbf{Y} \leftarrow \oplus x$	

On the Need for a Quantum Register Pseudocode.

Hans: copy-pasted from primer; rewrite for consistency with entanglement subchapter

We have established notation for computational basis states $|x\rangle$, general superposition states $|\psi\rangle$, unitary transformations $\mathbf{U}$, and the the result of applying a unitary transformation $\mathbf{U}$ on a quantum state $|\psi\rangle$, written $|\psi\rangle \xrightarrow{\mathbf{U}} |\varphi\rangle$.

And still this isn’t enough. This is because we are going to be talking about communicating parties. Consider the situation that Alice has two quantum bits initialized to the zero state, so that her state is $|\psi\rangle = |0\rangle_0 |0\rangle_1$. Now she wants to send her second qubit register, $|0\rangle_1$, over to Bob, who is going to apply some single-qubit unitary to it, and then send it back. We might write this as $|0\rangle_1 \xrightarrow{\mathbf{U}_1} |\xi\rangle_1$, where $|\xi\rangle_1$ is some unknown, single-qubit state, and say that Bob returns $|\xi\rangle_1$ to Alice at the end of the protocol.

But what if Alice had prepared her two qubits in the state $|\varphi\rangle = \frac{|00\rangle + |11\rangle}{\sqrt{2}}$ instead? Then, the two-qubit state is *non-separable*, meaning that it can no longer be cleanly factored into a tensor product of two single-qubit states. In the world of quantum physics, this means that the two qubits are *entangled*, a fancy word describing a phenomenon with many deep consequences for physics and philosophy alike, but which mathematically speaking is nothing but the quantum generalization of correlated probabilities.³⁴

³⁴ In other words, the term “separable” here carries exactly the same meaning as the term “product” does when talking about probability distributions. More precisely, if an arbitrary quantum state is *separable* along some

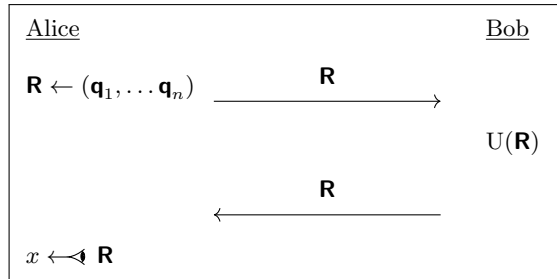


Fig. 18. Alice sends register R to Bob, who applies unitary U to it and sends it back to Alice, who then measures it in the computational basis, observing the value x . Note how, unlike in other formalizations, the element being returned to Alice is the same as what Alice sent to begin with (namely a qubit register in some unknown state).

Alice can of course still send her second qubit to Bob, and Bob can still apply the operation U to it; our problem merely one of notation, as we have no way of writing the operation that Bob is performing on $|0\rangle_1$ without referencing the entire state, like so:

$$\frac{|00\rangle_{0,1} + |11\rangle_{0,1}}{\sqrt{2}} \longrightarrow (\mathbb{1}_0 \otimes U_1) \left(\frac{|00\rangle_{0,1} + |11\rangle_{0,1}}{\sqrt{2}} \right).$$

What we want, is to be able to write the operations that Alice and Bob perform, without having to reference qubits that they don't have access to, regardless of any entanglement. In other words, what we want is an entanglement-agnostic quantum *pseudocode*.³⁵

Qubit Registers. Let q be a physical qubit, in the sense of being some kind of particle or quantum system existing in the physical world. Now, guided by our intuition, we want to say that Alice, instead of sending a quantum *state* to Bob, actually sends him the *physical qubit* q .³⁶ Anyone who receives a qubit, should also gain the ability to apply any transformation or measurement on it, while those who sent it should lose the ability to do so, unless and until the qubit is returned to them. Crucially, we should be able to describe these actions *locally*, i.e. without reference to any qubits *not* in the possession of the receiving party.

So, with this goal in mind, let us introduce a little more formalism.

Definition 12 (Qubit). A qubit q is a two-dimensional physical system, i.e. a physical system associated with an observable $\hat{q}$, taking values $\{m_0, m_1\}$ which can be mapped to $\{0, 1\}$. Each qubit is additionally associated with a unique register address $i \in \mathbb{N}$.³⁷

dividing line, such that $|\psi\rangle = |\varphi\rangle \otimes |\xi\rangle$, then the probability distribution over the possible outcomes of observing $|\psi\rangle$ will form a product distribution, factoring along the same dividing line. This remains true regardless of which basis you use to observe the state, implying that entanglement is an observation-independent (and therefore “objective”) property of the state.

³⁵ For readers ahead of the curve: Yes, one may reference one half of an entangled state using the generalized notion of a *mixed state*, by moving from Dirac's bra-ket notation to the density matrix formulation of Von Neumann. This formulation combines quantum states with classical probabilities, and informally speaking a density matrix (written $\rho := \sum_i p_i |\psi_i\rangle \langle \psi_i|$) is just a clever way of expressing classical probability distributions over quantum states. Then, you can easily write down the state corresponding to one half of an entangled state by observing that the state should look exactly the same to Bob whether Alice performed a measurement on Bob's qubit before sending it, or an operation entangling her qubit with Bob's. While such a formulation might be workable, we avoid this solution since going from a pure state to a mixed states loses information about the original state (since you can no longer tell from the state whether the qubit was measured or entangled), which would mean extra bookkeeping keeping track of the global state, which is precisely what we wanted to abstract away to begin with. Nonetheless, density matrices are invaluable to quantum information-theoretical analyses, and we will meet them again in App. F.

³⁶ In real-world quantum technologies, qubits are typically stored in one kind of system (e.g. atoms) and sent using another kind (typically photons), but due to the non-cloneability of quantum states, together with the universal ability to transfer any quantum state from one qubit to another (via the SWAP gate), our simplified model is without loss of generality.

³⁷ In terms of the original formulation of quantum Turing machines given by Deutsch [Deu85], the address i defined here corresponds to values of the *address observable* $\hat{x}$ (except that we limit ourselves to finite quantum memory).

Let us refer to the qubit sitting at address i as $\mathbf{q}_i$. If $\mathbf{q}_i$ is not entangled with any other qubit (i.e. it is in a pure state), then its state may be denoted $|\mathbf{q}_i\rangle$. If $|\mathbf{q}_i\rangle = |\psi\rangle$ for some particular single-qubit state $|\psi\rangle$, then this may be denoted $|\psi\rangle_i$.

Likewise, the state of qubit i at time t may be denoted $|\mathbf{q}_i(t)\rangle$. For the sake of illustration, say the single-qubit unitary U was applied to $\mathbf{q}_i$ at time t ; this would then be written

$$|\mathbf{q}_i(t)\rangle \xrightarrow{U_i} |\mathbf{q}_i(t+1)\rangle (= U_i |\mathbf{q}_i(t)\rangle) ,$$

(and if $\mathbf{q}$ was initialized to $|0\rangle$, then $|\mathbf{q}(0)\rangle = |0\rangle$).

Next, let us define our “universe”, that is, the full, undivided quantum memory which will act as a stage for all our plays.

Definition 13 (Quantum Memory). *An n -qubit quantum memory is an n -element list of qubits $\mathbf{Q} = (\mathbf{q}_1, \dots, \mathbf{q}_n)$ comprising a quantum system that is closed in the following sense: There are no operations that may be performed on the available qubits $\mathbf{q}_i$ such that $\mathbf{Q}$ is no longer in a pure state.*

In particular, $\mathbf{Q}$ can not get entangled with any system outside of itself—though it may be *observed*, after which it will immediately collapse to a new pure state.³⁸

Thus, our N -dimensional “global” quantum state $|\mathbf{Q}\rangle$ will always be well-defined, giving us a “smallest” element $\mathbf{q}$ and a “largest” element $\mathbf{Q}$. Then it is time to define the in-betweens.

Definition 14 (Quantum Register). *An m -qubit quantum register $\mathbf{R}$ is a subsystem of the n -qubit quantum memory $\mathbf{Q}$, that is, an ordered sub-list of $\mathbf{Q}$, where by ordered we mean that if $\mathbf{R} = (\mathbf{q}_{i_0}, \mathbf{q}_{i_1}, \dots, \mathbf{q}_{i_{m-1}})$ then $i_0 < i_1 < \dots < i_{m-1}$.*

If the qubits in $\mathbf{R}$ are not entangled with any other subsystems of $\mathbf{Q}$, then the system $\mathbf{R}$ is in a pure state, and we may write $|\mathbf{R}\rangle$. In a slight abuse of notation, let $[\mathbf{R}]$ denote the set $\{i_j\}_{j \in [m]}$ of qubit addresses associated with the qubits in $\mathbf{R}$; then $|\mathbf{R}\rangle$ is given by

$$|\mathbf{R}\rangle := \bigotimes_{\forall i \in [m]} |\mathbf{R}[i]\rangle := \bigotimes_{\forall i \in [\mathbf{R}]} |\mathbf{Q}[i]\rangle = |\mathbf{q}_{i_0}, \dots, \mathbf{q}_{i_{m-1}}\rangle .$$

If $|\mathbf{R}\rangle = |\psi\rangle$ for some specific state $|\psi\rangle$, this may then be denoted $|\psi\rangle_{[\mathbf{R}]}$, which, having satisfied ourselves that it is well-defined, we will immediately replace with the shorthand $|\psi\rangle_{\mathbf{R}}$.

But of course, the starting point of this entire exercise was that we can *not* in general write the quantum state of a qubit register as a pure state, and so we still need an alternative way to denote local operations on it.

Sending Qubits. First, Alice and Bob should be able to send qubit registers to each other. Let $\mathbf{A}$ and $\mathbf{B}$ be the registers of qubits available to Alice and Bob, respectively. Then “Alice sends $\mathbf{R}$ to Bob” can be expressed succinctly as

$$\mathbf{A} \xrightarrow{\mathbf{R}} \mathbf{B} ,$$

which removes the subregister $\mathbf{R}$ from $\mathbf{A}$ and adds it to $\mathbf{B}$.

Operating on Registers. If our (global) quantum memory is $\mathbf{Q}$, and we apply (a circuit that implements the unitary) U onto register $\mathbf{R}$, we will write this

$$\text{Apply}_{\mathbf{Q}}(U \rightarrow \mathbf{R}) : |\mathbf{Q}\rangle \longrightarrow U_{\mathbf{R}} |\mathbf{Q}\rangle .$$

With this notation, it now becomes clear that although Alice only operates on the qubits that are available to her, those very same operations may affect distant parts of the global state $|\mathbf{Q}\rangle$ through entanglement. Whenever the global state $\mathbf{Q}$ is clear from context, we will drop the subscript and simply write $\text{Apply}(U \rightarrow \mathbf{R})$.

³⁸ Of course, from the Everettian point of view, observing a state *is* merely to entangle with it.

Measuring Registers. Finally, we need to measure the qubits at some point and read out the classical information. Given a projective measurement operator $\mathcal{M}$ and a register $\mathbf{R}$, we write the act of measuring $\mathbf{R}$ according to $\mathcal{M}$ as

$$x \leftarrow \$ \text{Meas}_{\mathbf{Q}}(\mathcal{M} : \mathbf{R}) .$$

Note that x is a classical *sampled* value, as it must be, since measurements are inherently stochastic.

The post-measurement will be

$$|\mathbf{Q}\rangle \xrightarrow{\mathcal{M}} |\mathbf{Q}\rangle_{\mathcal{S}_i} ,$$

where $|\mathbf{Q}\rangle_{\mathcal{S}_i}$ is the result of projecting the qubits of $\mathbf{R}$ onto subspace $\mathcal{S}_i$, and applying the identity to the rest.

As before, we omit the subscript whenever $\mathbf{Q}$ is clear from context, and we also omit the measurement operator $\mathcal{M}$ whenever we are performing a computational-basis measurement; for example, measuring ℓ qubits and observing the bit string $x \in \{0, 1\}^\ell$ will be written simply

$$x \leftarrow \$ \text{Meas}(\mathbf{R}) .$$

The above notation should hopefully be intuitively clear, though coming up with a precise definitions takes a little extra work; you can find this in Sect. 10.

To showcase the utility of our pseudocode, we can now restate the Born rule in a very compact form.

Lemma 13 (The Born Rule). *If qubit register $\mathbf{R}$ is in pure state $|\mathbf{R}\rangle = \sum_i \alpha_i |x_i\rangle$, then*

$$\Pr[x \leftarrow \$ \text{Meas}(\mathbf{R}) : x = x_i] = |\alpha_i|^2 .$$

11 The Extractors of O2H

12 Lazy-Sampling QROs with the Compressed-Oracle Technique

Classical random oracles have the neat feature that they are easy to simulate: simply (...)

We will employ the compressed oracle technique, as introduced by Zhandry [Zha19]; specifically, we employ an adapted version of the “standard Fourier oracle”, the choices being standard/phase and computational/Fourier, depending on what oracle interface we want, and in which domain the database entries should be defined.³⁹

12.1 The Setup

Let us begin by specifying our notation. To start with, unless noted otherwise, we will assume that all random oracles have input length n and output length m , and that q is an upper bound on the total number of queries made to the oracle.

The Empty String. Let ε to be the empty string, encoded as $1\|0^n$. We will interpret the string $0\|x$ as x . Strings of the form $1\|x$ for $x \neq 0^n$ will never appear, but for the sake of reference let us call them $\neq$.

We define the $(1+n)$ -bit controlled-not operation on ε to be $CX_\varepsilon(|x\rangle_{\mathbf{A}} \rightarrow |\varepsilon\rangle_{\mathbf{B}}) = |x\rangle_{\mathbf{A}}|x\rangle_{\mathbf{B}}$. This may be implemented by performing the controlled not-operation as usual, and then flipping the leading bit of register $\mathbf{B}$. Repeating the operation yields $CX_\varepsilon(|x\rangle_{\mathbf{A}} \rightarrow |x\rangle_{\mathbf{B}}) = |x\rangle_{\mathbf{A}}|\varepsilon\rangle_{\mathbf{B}}$ again, making CX_ε an involution. (Applying CX_ε to states $|x\rangle|y\rangle$ such that $x \neq y$ will lead to the invalid state $|x\rangle|z\rangle$, where $z = x \oplus y$, but we will never apply the gate to such cases.)

Observe that $x \cdot \varepsilon = 0$ for any x , where $\cdot$ denotes the bitwise inner product.

Databases. We are going to keep a database register $\mathbf{D}$, storing q list entries of the format $(b_\varepsilon\|x, \hat{y})$, meaning $\mathbf{D}$ is a register containing $q \cdot (1+n+m)$ qubits. The database starts out in the “empty” state, with each entry initialized in the pure state $|\varepsilon, 0^m\rangle$. After several queries, the database will end up in a superposition over lists $\mathbf{D}$, with each list being split in two parts: An “active” part of tuples $(x, \hat{y})$ such that $\hat{y} \neq 0^n$, sorted by increasing x , followed by “fresh” part of entries set to the initial value $(\varepsilon, 0^m)$. More precisely, after ℓ queries, $\mathbf{D}$ will be in a superposition over basis states of the form

$$\begin{aligned} |\mathbf{D}\rangle &= |(x_1, \hat{y}_1), (x_2, \hat{y}_2) \dots, (x_k, \hat{y}_k), (\varepsilon, 0^m), \dots, (\varepsilon, 0^m)\rangle \\ &= |x_1, \hat{y}_1\rangle_1 |x_2, \hat{y}_2\rangle_2 \dots |x_k, \hat{y}_k\rangle_k |\varepsilon, 0^m\rangle_{k+1} \dots |\varepsilon, 0^m\rangle_q, \end{aligned}$$

with unique and increasing input elements $x_1 < x_2 < \dots < x_k$, non-zero output elements $\hat{y}_i$, and $k \leq \ell$. We write $\mathbf{D}[x] = \hat{y}$ if the list contains an element $(x, \hat{y})$, and $\mathbf{D}[x] = \perp$ otherwise.

The state of the database register will in general be highly entangled with the parties performing the queries; indeed, it will turn out that a list element is non-empty *if and only if* the register is entangled with the state of some querying party. Finally, the number of “active” versus “fresh” entries may differ between branches of the superposition of $\mathbf{D}$, though their sum will always be q .⁴⁰

Database Subregisters. Next, let us define a naming convention for subregisters of $\mathbf{D}$: Let $\mathbf{D}_{\mathbf{X}}$ refer to the collection of qubits that store input values in the list (so a subregister of size $q \cdot (1+n)$), and let $\mathbf{D}_{\mathbf{Y}}$ refer to the collection of qubits that stores output values (with size $q \cdot m$).

Now, provided $\mathbf{D}[x] \neq \varepsilon$ in some branch of the superposition, let $\mathbf{D}(x)$ refer to the qubits that store the tuple $(x, \hat{y})$ in that branch. Note that even in branches where this holds, the exact qubit indices corresponding to $\mathbf{D}(x)$ will in general differ between branches, since the lists in the superposition may contain a varying number of entries that precede x , making $\mathbf{D}(x)$ only well-defined when specializing to single branches of the $\mathbf{D}_{\mathbf{X}}$ state. When speaking generally, we will instead use $\mathbf{D}_i$ to refer to the qubits that store the i ’th tuple.

Combining the above, we can then refer to the subregister storing the first element of the tuple $(x, \hat{y})$ in a particular branch as $\mathbf{D}_{\mathbf{X}}(x)$, and the register storing the corresponding value $\hat{y}$ as $\mathbf{D}_{\mathbf{Y}}(x)$. Likewise, $\mathbf{D}_{\mathbf{X},i}$ (resp. $\mathbf{D}_{\mathbf{Y},i}$) refers to the subregister storing the input (resp. output) value of the i ’th tuple, irrespective of branch.

³⁹ The standard Fourier oracle is not the version employed by Zhandry in his paper, and it is only defined in an appendix, but it is the version I find most closely matches the intuition behind the compressed oracle technique.

⁴⁰ In the original formulation of the compressed oracle technique, the size of $\mathbf{D}$ was allowed to grow and shrink with each call. This leads to a superposition over databases where also the *size* of the register is in superposition. In our formalization, registers are classically well-defined lists storing qubits (which are two-level physical systems), and while it is technically possible to resize registers (by moving qubits around), we do not allow for this to happen in superposition. Therefore, we fix the size of $\mathbf{D}$ to be q a priori, and simply accept that the simulation will fail if the query count ever exceeds q .

Table 5. An overview of our notation for compressed random oracles.

Meaning	Notation	Object Class
Input register	X	n -length qubit register
Output register	Y	m -length qubit register
Database	D	$(q \cdot (1 + n + m))$ -length register
Empty string	ε	$:= 0 0^n$
Interpreted input string	x	$:= 0 x, x \in \{0, 1\}^n$
Invalid string	$\nexists$	$:= 1 x, x \in \{0, 1\}^n \setminus \{0^n\}$
Hadamard-domain value	$\hat{y}$	Labels basis states in the support of $H^{\otimes m} y\rangle$
List	D	List of q entries
List entry	$(x, \hat{y})$	$(1 + n, m)$ -sized string tuple $(b_\varepsilon x, \hat{y})$
Initialized entry	$(\varepsilon, 0^m)$	$(1 + n, m)$ -sized string tuple
List contains $(x, \hat{y})$	$D[x] = \hat{y}$	$x \in \{0, 1\}^n, \hat{y} = \{0, 1\}^m$
x is not in list	$D[x] = \perp$	Special symbol denoting “undefined”
Input subregister	D_X	$(q \cdot (1 + n))$ -sized subregister storing input values
Output subregister	D_Y	$(q \cdot m)$ -sized subregister storing output values
Entry subregister	D_[i]	$(1 + n + m)$ -sized subregister storing i th entry
Input-specific subregister	D_(x)	Subregister D_[i] storing $(x, D[x])$ in current branch
Entry I/O registers	D_{X,i}, D_{Y,i}	Subregisters storing i ’th input resp. output value
Specific I/O registers	D_X(x), D_Y(x)	Subregisters storing x resp. $D[x]$ in current branch

The Primal and the Hadamard Domain. You may have noticed the hats. These denote that the value in question is defined in the Hadamard-domain: a computational basis state $|y\rangle$ in the primal domain (left) will be the following superposition in the Hadamard domain (right):

$$|y\rangle \xleftrightarrow{H^{\otimes m}} \sum_{\forall \hat{y} \in [2^m]} \frac{(-1)^{y \cdot \hat{y}}}{\sqrt{2^m}} |\hat{y}\rangle,$$

and vice versa:

$$\sum_{\forall y \in [2^m]} \frac{(-1)^{\hat{y} \cdot y}}{\sqrt{2^m}} |y\rangle \xleftrightarrow{H^{\otimes m}} |\hat{y}\rangle.$$

In particular, we have that the all-zero state in the Hadamard domain corresponds to the uniform superposition in the primal domain:

$$\sum_{\forall y \in [2^m]} \frac{1}{\sqrt{2^m}} |y\rangle \xleftrightarrow{H^{\otimes m}} |0^m\rangle.$$

As illustrated, one can move between the primal domain and the Hadamard domain by performing the Hadamard transformation (which is just applying the H gate to each of the qubits in the register).⁴¹ We will only move between domains on the output register **Y**; the input registers **X** and **D_X** will be kept in the primal domain, and the database’s output registers **D_Y** will be kept in the Hadamard domain.

12.2 Defining Compressed Random Oracles

Updating the Database. Next, we define a unitary that acts on the combined state of the input register **X** and the database **D**, named *St* for “Store” and/or “Sort”.⁴² In essence, it is an operation that reads the input and updates the database accordingly. We define it by specifying it on all basis states, verifying along the way that each step is reversible, and conclude that *St* is unitary and therefore well-defined. Indeed, *St* will turn out to be an involution, meaning that applying it twice in a row will return the same state (or in other words, that $St^2 = \mathbb{1}$).

⁴¹ Zhandry [Zha19] varyingly refers to the Quantum Fourier Transformation and the Hadamard transformation, but he clarifies in App. A that he only ever uses the latter. (The Hadamard gate H is the single-qubit Quantum Fourier Transformation.)

⁴² This is the *FourDecomp* unitary in the language of Zhandry, for “Fourier Decompress” [Zha19, App. C].

Definition 15 (Unitary St). For each computational basis state $|x\rangle|D\rangle$ in the support of the combined state of registers $\mathbf{X}$ and $\mathbf{D}$, perform the following steps:

1. Do one of the following, depending on the case:
 - (a) If $D[x] = \perp$, set the first empty element $(\varepsilon, 0^m)$ in D to $(x, 0^m)$, by applying CX_ε from $\mathbf{X}$ to the corresponding subregister $\mathbf{D}_{\mathbf{X},i}$.
 - (b) If $D[x] = 0^m$, “forget” the element by applying CX_ε from $\mathbf{X}$ to the subregister of $\mathbf{D}_{\mathbf{X}}(x)$, yielding the entry $(\varepsilon, 0^m)$.
 - (c) If $D[x] = \hat{y} \neq 0^m$, then $\mathcal{H}(x)$ has already been set, and $\mathbf{D}(x)$ must furthermore be entangled with a querying party; do nothing.
 - (d) For all other basis states, which is to say for lists D that contain no empty entries $(\varepsilon, 0^m)$ (which can only happen if the query count exceeds q), and for lists that contain invalid values $\hat{x}$ for some $x \neq 0^n$ (which will never happen), act as the identity.
2. Use controlled SWAP gates to sort D by increasing x .

The If-statements may be implemented as multi-bit controlled operations. Notice that the cases are exhaustive and mutually exclusive, and that they define an involution. Therefore, St is unitary.

Entangling the Query. With the database updating procedure defined, all that remains is to define how the output is to be prepared.

In essence, the idea is to move the output register $\mathbf{Y}$ to the Hadamard domain, entangle it with the subregisters $\mathbf{D}_{\mathbf{Y}}(x)$ for each x in the input superposition, and then return $\mathbf{Y}$ to the primal domain, before shipping $\mathbf{X}$ and $\mathbf{Y}$ back to the querying party. The entanglement, which is hidden from view of whoever made the query, will make it look like a classical probability distribution has been induced over the possible states of $\mathbf{Y}$, just as if $\mathcal{H}$ had been pre-sampled from the set of all functions.

Let us next define the “Entangling” unitary Ent .

Definition 16. Let Ent be the unitary that acts on basis states $|x\rangle|y\rangle|D\rangle$ of registers $\mathbf{X}$, $\mathbf{Y}$, $\mathbf{D}$ in the following way:

$$|x\rangle|\hat{y}\rangle|\dots, (x, z), \dots\rangle \xrightarrow{\text{Ent}} |x\rangle|\hat{y}\rangle|\dots, (x, \hat{y} \oplus z), \dots\rangle,$$

and as the identity whenever $D[x] = \perp$.⁴³ The cases are exhaustive and mutually exclusive, and define an involution. Therefore, Ent is unitary.

The Oracle Procedure. With St and Ent defined, we are finally ready to define the action performed by our compressed random oracle upon receiving a query $\mathcal{H}(\mathbf{X}, \mathbf{Y})$.

Definition 17 (Compressed Random Oracle). Given registers $\mathbf{X}, \mathbf{Y}, \mathbf{D}$, the compressed random oracle $\mathcal{H} : \{0, 1\}^n \rightarrow \{0, 1\}^m$ performs the following operations:

1. Apply($\text{St} \rightarrow \mathbf{X}, \mathbf{D}$)
2. Apply($H^{\otimes m} \rightarrow \mathbf{Y}$)
3. Apply($\text{Ent} \rightarrow \mathbf{X}, \mathbf{Y}, \mathbf{D}$)
4. Apply($H^{\otimes m} \rightarrow \mathbf{Y}$)
5. Apply($\text{St} \rightarrow \mathbf{X}, \mathbf{D}$)

Having defined this procedure, Zhandry then proved that it provides a perfect simulation of a quantum-accessible random oracle [Zha19, Lemma 4].

Theorem 5 (Simulation Is Perfect). Let $\mathcal{H}$ be a (monolithic) quantum-accessible random oracle, and let $\mathcal{G}$ be a compressed random oracle. Then we have for all distinguishers $\mathcal{D}$,

$$\Pr[1 \leftarrow \$ \mathcal{D}^{\mathcal{H}}] = \Pr[1 \leftarrow \$ \mathcal{D}^{\mathcal{G}}].$$

12.3 But How Do They Work?

What is the effect of the operations listed in Def. 17? Let’s explore!

⁴³ Which we will again ensure never occurs.

Classical Queries. As a first sanity check, consider the result of performing the first three steps on a classical query, i.e. with $\mathbf{X}$ and $\mathbf{Y}$ put in a computational basis state $|x\rangle|y\rangle$, to the initially-empty database $|\mathbf{D}_\varepsilon\rangle$:

$$\begin{aligned} 1. & |x\rangle|y\rangle|\mathbf{D}_\varepsilon\rangle := |x\rangle|y\rangle|(\varepsilon, 0^m), \dots, (\varepsilon, 0^m)\rangle \xrightarrow{\text{St}} |x\rangle|y\rangle|(x, 0^m), (\varepsilon, 0^m), \dots, (\varepsilon, 0^m)\rangle := |x\rangle|y\rangle|\mathbf{D}_x\rangle, \\ 2. & |x\rangle|y\rangle|\mathbf{D}_x\rangle \xrightarrow{H^{\otimes m}} |x\rangle\left(\sum_{\hat{y} \in [2^m]} \frac{(-1)^{y \cdot \hat{y}}}{\sqrt{2^m}} |\hat{y}\rangle\right)|\mathbf{D}_x\rangle, \\ 3. & |x\rangle\left(\sum_{\hat{y} \in [2^m]} \frac{(-1)^{y \cdot \hat{y}}}{\sqrt{2^m}} |\hat{y}\rangle\right)|(\varepsilon, 0^m), (\varepsilon, 0^m), \dots\rangle \\ & \xrightarrow{\text{Ent}} |x\rangle\left(\sum_{\hat{y} \in [2^m]} \frac{(-1)^{y \cdot \hat{y}}}{\sqrt{2^m}} |\hat{y}\rangle|(x, \hat{y}), (\varepsilon, 0^m), \dots\rangle\right) := \sum_{\hat{y} \in [2^m]} \frac{(-1)^{y \cdot \hat{y}}}{\sqrt{2^m}} |x\rangle|\hat{y}\rangle|\mathbf{D}_{x, \hat{y}}\rangle. \end{aligned}$$

At this point, $\mathbf{Y}$ is maximally entangled with the subregister of $\mathbf{D}$ housing $\mathbf{D}[x]$: We can see from the above that their combined state now has the form (ignoring amplitudes) $|\hat{y}_1\rangle|\hat{y}_1\rangle + |\hat{y}_2\rangle|\hat{y}_2\rangle + \dots + |\hat{y}_{2^m}\rangle|\hat{y}_{2^m}\rangle$. This may be a large superposition, but the outcome of observing the two registers is guaranteed to be equal, which is exactly what it means to be maximally entangled (in general, that observing one register fully determines the state of the other). Performing the Hadamard transformation on $\mathbf{Y}$ thereafter does nothing to change this⁴⁴, but rather has the effect of moving their combined state from one maximally entangled state to another.

Finally, the second application of *St* does nothing, since the list is already sorted and there is nothing to remove. That is, with one exception: In the single branch of the wavefunction where $\hat{y} = 0^m$, x will be replaced with ε , which has the effect of “implicitly” defining the value of $\hat{y}$ in this branch.

At a glance, this may appear to be a problem, since if the querying party then by chance observes exactly this branch, and then queries the compressed oracle again on the same input, the database would not “know” that it was supposed give the same value back, leading to an inconsistency. However, Thm. 5 tells us that simulation is perfect, so *something* will have to ensure consistency in this case. We will return to the question of how at the end of this section, though I urge you to work it out yourself and discover for yourself how it happens. (Hint: Try specializing to the $m = 1$ case.)

And so, accepting the conundrum for now, we return to our maximally entangled state. The entanglement is hidden from view of the querying party, and hiding away entanglement in this manner has the effect of “mixing” the state of $\mathbf{Y}$: From their point of view, hidden entanglement is indistinguishable from a classical probability distribution over quantum states⁴⁵; and maximal entanglement yields maximally mixed states, which corresponds to a uniform probability distribution over basis states.

Thus, the querying party has the experience of having the $\mathbf{Y}$ register placed in a uniformly random computational basis state—exactly as if $\mathcal{H}(x)$ had been sampled at random and xored into the register!

Uniform Superpositions. Next, what if $\mathbf{Y}$ is set to be in the uniform superposition, $\sum_y \sqrt{2^{-m}}|y\rangle$ (with the input value still set to the computational basis state $|x\rangle$)? Well, then *nothing* happens: After being moved to the Hadamard domain, the $\mathbf{Y}$ register will be in the state $|0^m\rangle$, and xoring into the database has no effect: The second Hadamard transformation will take it back to the uniform superposition, and then return it as-is. Note that this also matches the querying party’s expectations: Since the xor of any fixed string defines a permutation, xoring into the uniform superposition should yield the uniform superposition, regardless of the value $\mathcal{H}(x)$ that is being xored. This means that, when interacting with a real random oracle, $\mathcal{H}(x)$ will remain perfectly hidden from their view. This is matched in the compressed oracle by the second application of *St* removing the point x from the list and replacing it with ε , ensuring no entanglement (which would otherwise have been a noticeable inconsistency, since the output register would no longer be in a uniform superposition, but rather in some mixed state).

What about other superpositions, for example $\sum_y (-1)^{y \cdot z} / \sqrt{2^{-m}} |y\rangle$ for some $z \in \{0, 1\}^m$? Well, then the querying party *would* expect to be able to learn something about the output, by observing how the phases shift around. The database will likewise be updated in this case: In our example, moving $\mathbf{Y}$ to the Hadamard domain will put it in the computational basis state $|z\rangle$, which will *Ent* will then store into the database as (x, z) . Now we are back to the case of maximal entanglement, as can be seen from the combined state of $\mathbf{Y}$ and $\mathbf{D}_\mathbf{Y}(x)$ being $|z\rangle|z\rangle$.

⁴⁴ This is because entanglement is a *basis-independent* feature of quantum states; in fact, it fulfills the definition of a *quantum resource*, i.e. something that can be spent to perform tasks <https://arxiv.org/abs/1806.06107>.

⁴⁵ In fact, this is the observation that lead Hugh Everett to develop his many-worlds interpretation of quantum mechanics, since the randomness induced by wavefunction collapse can just as well be explained in terms of you entangling *yourself* with the quantum state you are observing. And why not? After all, you, too, are made of quantum stuff!

Of course, in between the two extremes of no information and maximal entanglement lies the world of *some* entanglement, as results from setting $\mathbf{Y}$ to some non-uniform superposition. Additionally, $\mathbf{Y}$ may itself be arbitrarily entangled with other subsystems belonging to the querying party, in which case it will be a mixed state from the point of view of the compressed oracle, and so their combined state will, after entangling, also be a mixed state.

Superpositions over Input Values. Finally, what about superpositions over x ? The easiest way to think about this is to adopt an Everettian worldview, and imagine the superposition state of $\mathbf{X}$ describing a superposition over several possible universes. Then, one can consider each universe separately: The branch of the wavefunction where $\mathbf{X}$ is in basis state $|x\rangle$ is the branch of the wavefunction where the querying party (who will in general be entangled with $\mathbf{X}$) decided to query the random oracle on the value x . And since these registers always remain in the primal domain, we don't have to grapple with the metaphysical implications of “changing the basis” on a collection of universes.

Thus, the result of such a query again depends on the state of $\mathbf{Y}$: If $\mathbf{Y}$ is in the uniform superposition, independently of $\mathbf{X}$, then it is in the uniform superposition in every universe, and nothing is ever stored and nothing is ever learned. If $\mathbf{Y}$ is set in a computational basis state, such as $|0^m\rangle$, then the result is a maximally entangled state, appearing as a classically uniform probability distribution over basis states in all universes. Finally, if $\mathbf{X}$ and $\mathbf{Y}$ are maximally entangled in some state $|x_1\rangle|y_1\rangle + |x_2\rangle|y_2\rangle + \dots$, for unique values of x_i and y_i , then inside each universe we are performing a purely classical query, and we are back to the situation that we first considered.

Repeated Queries. Say the oracle started out in the empty state $|\mathbf{D}_\varepsilon\rangle$, and was then queried on the classical input state $|x\rangle|y\rangle$. Then we found that the result is the combined state

$$\sum_{\forall \hat{y} \in [2^m] \setminus \{0^m\}} \frac{(-1)^{y \cdot \hat{y}}}{\sqrt{2^m}} |x\rangle (H^{\otimes m} |\hat{y}\rangle) |\mathbf{D}_{x,\hat{y}}\rangle + \frac{1}{\sqrt{2^m}} |x\rangle (H^{\otimes m} |0^m\rangle) |\mathbf{D}_\varepsilon\rangle .$$

Now, what happens if the adversary queries the oracle on the registers $\mathbf{X}$ and $\mathbf{Y}$ a second time, without changing their in any way in the meantime? Well, first, the Hadamard transformation on $\mathbf{Y}$, being an involution, cancels out:

$$\sum_{\forall \hat{y} \in [2^m] \setminus \{0^m\}} \frac{(-1)^{y \cdot \hat{y}}}{\sqrt{2^m}} |x\rangle |\hat{y}\rangle |\mathbf{D}_{x,\hat{y}}\rangle + \frac{1}{\sqrt{2^m}} |x\rangle |0^m\rangle |\mathbf{D}_\varepsilon\rangle .$$

Next, for each basis state $|x\rangle|\hat{y}\rangle|\mathbf{D}_{x,\hat{y}}\rangle$ such that $\hat{y} \neq 0^m$, we will have that $\mathbf{D}[x] \neq \perp$, and so St will do nothing, while for $\hat{y} = 0^m$ it will insert the element $(x, 0^m)$ into $\mathbf{D}$. This gives us the state:

$$\sum_{\forall \hat{y} \in [2^m]} \frac{(-1)^{y \cdot \hat{y}}}{\sqrt{2^m}} |x\rangle |\hat{y}\rangle |\mathbf{D}_{x,\hat{y}}\rangle .$$

Then, Ent will have the following effect on the basis states:

$$|x\rangle|\hat{y}\rangle|(x, \hat{y}), \dots\rangle \xrightarrow{\text{Ent}} |x\rangle|y\rangle|(x, \hat{y} \oplus \hat{y}), \dots\rangle = |x\rangle|y\rangle|(x, 0^m), \dots\rangle ,$$

and with the second application of St removing all entries with $\hat{y} = 0^m$, we will end up with the unentangled state

$$\sum_{\forall \hat{y} \in [2^m]} \frac{(-1)^{y \cdot \hat{y}}}{\sqrt{2^m}} |x\rangle |\hat{y}\rangle |\mathbf{D}_\varepsilon\rangle .$$

Transforming $\mathbf{Y}$ back to the primal domain then yields

$$|x\rangle|y\rangle|\mathbf{D}_\varepsilon\rangle ,$$

which is exactly where we started before the first query!

We see that the entanglement has been removed, and the value of x has been completely removed from the oracle's view. What's more, since this “forgetting” behaviour is shown to hold for any computational basis state, it must hold for *every* quantum state that is forwarded to the oracle twice in a row, making the compressed oracle *itself* an involution! Worse, not only does the oracle seem to “forget” that it was ever queried, but so does the querying party also “forget” the value of $\mathcal{H}(x)$!

It may seem counter-intuitive, but this “forgetting” is in fact exactly what we need to happen (and was indeed a major roadblock towards lazy-sampling quantum random oracles prior to the development of this technique). To see why, consider the situation for “monolithic” random oracles, which are also involutions: Here, the classical query $|x\rangle |y\rangle$ returns the state $|x\rangle |y \oplus \mathcal{H}(x)\rangle$. If the query is then repeated with the same registers without altering their state, the result will be $|x\rangle |y \oplus \mathcal{H}(x) \oplus \mathcal{H}(x)\rangle = |x\rangle |y\rangle$. The value of $\mathcal{H}(x)$ has been forgotten!

“But”, I hear you say, “what if I store the value of $\mathcal{H}(x)$ before I make the second query? Surely I won’t be forced to automatically ‘forget’ what the value was, simply because I query the oracle a second time!” And you’d be right—but notice that we are now talking about a different quantum state, namely the state $|x\rangle_{\mathbf{X}} |y \oplus \mathcal{H}(x)\rangle_{\mathbf{Y}} |\mathcal{H}(x)\rangle_{\mathbf{A}}$, where the $\mathbf{A}$ register is some storage register local to you. By storing $\mathcal{H}(x)$ in this register (by first preparing it in the state $|y\rangle$ and then performing the m -bit CX gate from $\mathbf{Y}$ to $\mathbf{A}$), you *entangled* the two registers. And so, while the result of a second query to the monolithic oracle will indeed be the state $|x\rangle |y\rangle |\mathcal{H}(x)\rangle$, the effect on the compressed oracle will be completely different to the “forgetting” behaviour outlined above. This can be seen from the fact that the Hadamard transformation, which previously operated on the pure, unentangled state $|y\rangle$, will now take the $\mathbf{Y}$ register to some other state than the one it was in during the first query, and the cancellation of the $\hat{y}$ values in the second query won’t happen.

How Consistency Is Maintained. Now, let us return to the conundrum from the start of the section: Is it really true that forgetting x on one of 2^n points can never lead to problems? Well—yes and no. We’ll start with “yes” here, and leave the “no” for the next section.

Consider the following situation: An adversary $\mathcal{A}$ queries wants to know whether the oracle they have gotten access to is a monolithic oracle or a compressed oracle. They query it on the state $|x\rangle |y\rangle$, and then measure the y register, writing down the result. Then they query the oracle on the same state again. If the oracle is monolithic, then the second result is guaranteed to be the same as the first, and if the oracle is compressed, then it is easy to see that the result will be consistent as long as the measurement didn’t lead the database to collapse to the one out of 2^m branches where $\hat{y} = 0^m$. But what if it did?

For simplicity, let’s consider the case that $m = 1$, and calculate the exact state after each operation. In Sect. 12.2, we saw that the first three operations of the compressed oracle lead to the following state:

$$\sum_{\forall \hat{y} \in [2^m]} \frac{(-1)^{y \cdot \hat{y}}}{\sqrt{2^m}} |x\rangle |\hat{y}\rangle |D_{x,\hat{y}}\rangle ,$$

which for $m = 1$ we can write as

$$|x\rangle \frac{|0\rangle |D_{x,0}\rangle + (-1)^y |1\rangle |D_{x,1}\rangle}{\sqrt{2}} .$$

After applying St, we will have

$$|x\rangle \frac{|0\rangle |D_\varepsilon\rangle + (-1)^y |1\rangle |D_{x,1}\rangle}{\sqrt{2}} .$$

Next, we move the $\mathbf{Y}$ register back to the primal domain, which gives us the state

$$|x\rangle \frac{\frac{|0\rangle+|1\rangle}{\sqrt{2}} |D_\varepsilon\rangle + (-1)^y \frac{|0\rangle-|1\rangle}{\sqrt{2}} |D_{x,1}\rangle}{\sqrt{2}} = |x\rangle \frac{|0\rangle |D_\varepsilon\rangle + |1\rangle |D_\varepsilon\rangle + (-1)^y |0\rangle |D_{x,1}\rangle + (-1)^{1-y} |1\rangle |D_{x,1}\rangle}{2} ,$$

and send it back to $\mathcal{A}$. At this point, $\mathcal{A}$ measures the $\mathbf{Y}$ register. Let’s say they see the value 0, which they interpret as $y \oplus \mathcal{H}(x) = 0$, and so conclude that $\mathcal{H}(x) = y$. Now, to double check, they make the same query again.

From the point of view of the compressed oracle, the measurement collapsed the entangled state down to the following one, in which the $\mathbf{Y}$ register is guaranteed to be in the $|0\rangle$ state:

$$|x\rangle \frac{|0\rangle |D_\varepsilon\rangle + (-1)^y |0\rangle |D_{x,1}\rangle}{\sqrt{2}} = |x\rangle |0\rangle \frac{|D_\varepsilon\rangle + (-1)^y |D_{x,1}\rangle}{\sqrt{2}} .$$

But before returning the registers, to ensure an identical query, $\mathcal{A}$ reset the $\mathbf{Y}$ to $|y\rangle$, so that the state we receive is

$$|x\rangle |y\rangle \frac{|D_\varepsilon\rangle + (-1)^y |D_{x,1}\rangle}{\sqrt{2}} .$$

Now, the compressed oracle moves $\mathbf{Y}$ to the Hadamard domain:

$$|x\rangle \left(\frac{|0\rangle + (-1)^y |1\rangle}{\sqrt{2}} \right) \left(\frac{|\mathbf{D}_\varepsilon\rangle + (-1)^y |\mathbf{D}_{x,1}\rangle}{\sqrt{2}} \right) = |x\rangle \frac{|0\rangle |\mathbf{D}_\varepsilon\rangle + (-1)^y |1\rangle |\mathbf{D}_\varepsilon\rangle + (-1)^y |0\rangle |\mathbf{D}_{x,1}\rangle + |1\rangle |\mathbf{D}_{x,1}\rangle}{2},$$

applies St:

$$|x\rangle \frac{|0\rangle |\mathbf{D}_{x,0}\rangle + (-1)^y |1\rangle |\mathbf{D}_{x,0}\rangle + (-1)^y |0\rangle |\mathbf{D}_{x,1}\rangle + |1\rangle |\mathbf{D}_{x,1}\rangle}{2},$$

and then Ent:

$$\begin{aligned} & |x\rangle \frac{|0\rangle |\mathbf{D}_{x,0}\rangle + (-1)^y |1\rangle |\mathbf{D}_{x,1}\rangle + (-1)^y |0\rangle |\mathbf{D}_{x,1}\rangle + |1\rangle |\mathbf{D}_{x,0}\rangle}{2} \\ &= |x\rangle \frac{|0\rangle (|\mathbf{D}_{x,0}\rangle + (-1)^y |\mathbf{D}_{x,1}\rangle) + |1\rangle (|\mathbf{D}_{x,0}\rangle + (-1)^y |\mathbf{D}_{x,1}\rangle)}{2} \\ &= |x\rangle \left(\frac{|0\rangle + |1\rangle}{\sqrt{2}} \right) \left(\frac{|\mathbf{D}_{x,0}\rangle + (-1)^y |\mathbf{D}_{x,1}\rangle}{\sqrt{2}} \right). \end{aligned}$$

Thereafter, St is applied a second time, before $\mathbf{Y}$ is moved back to the primal domain and shipped back to $\mathcal{A}$, leaving us with the state

$$|x\rangle |0\rangle \frac{|\mathbf{D}_\varepsilon\rangle + (-1)^y |\mathbf{D}_{x,1}\rangle}{\sqrt{2}}.$$

This is exactly the same state that we had after measuring! We see that the combined effect of putting $\mathbf{Y}$ back in the state $|y\rangle$ and repeating the query on the same point x actually acted like the identity on the state. In particular, this means that the query can be repeated indefinitely without any further effect on the state on the combined system whatsoever.⁴⁶

Actually, since we showed that this works for both qubit states $|y\rangle$, and since all operations are performed bitwise anyway, this already implies that the oracle will exhibit the same behaviour for general superpositions, and for $m > 1$.

So, what was the crucial element that allowed consistency to be maintained? In my view, it is the fact that measuring $\mathbf{Y}$ only lead to a *partial* collapse of $\mathbf{D}$'s quantum state. In particular, some branches of the superposition still contained information about x , and (as so often happens in quantum theory) these branches just *happened* to be arranged in just the right state for phases to cancel out in such a way that consistency is maintained.

In other words, consistency crucially relied on the *database* remaining in a superposition, even after measurement. If we were to measure the *database*, then all bets would be off!

12.4 Extracting Queries

So naturally, that's exactly what we'll do next. After all, the usefulness of lazy sampling comes not only from the ability to efficiently simulate a random oracle, but also the ability to *see* what inputs the adversary is interested in!

First, if we want our simulation to remain perfect, we must avoid any and all measurements of the state of $\mathbf{D}$ as long as the game is running—only after it has ended can we perform our measurement, and see what we end up with.

So, does it work? *Are* we guaranteed that, if the adversary measures a series of queries and learns the values of $\mathcal{H}(x_1), \dots, \mathcal{H}(x_n)$, that measuring the database will give us the values $x_1, \dots, x_n$?

Well, *almost*—and this is where deleting all those x 's comes back to haunt us. There is namely a tiny probability that this leads to the adversary learning the point $\mathcal{H}(x)$, which just happened to correspond to an x that we removed in at least some of the branches of the superposition, and that upon measuring the database, the branch we end up witnessing is exactly such a branch.

Thankfully, we can show that the probability of this unfortunate outcome is exponentially small in the length of the output.⁴⁷ This is what the next theorem, also proven by Zhandry [Zha19, Lemma 5], shows.

Lemma 14 (Extraction Is Imperfect). *Let the adversary $\mathcal{A}$ be a quantum algorithm with oracle access to a random oracle $\mathcal{H}$, and that after at most q queries outputs $\ell \leq q$ tuples (x, y) with unique values for x , and let $\text{Adv}_{\mathcal{R}}(\mathcal{A}^{\mathcal{H}})$ be the probability that*

⁴⁶ To check your understanding, you may want to verify that, had $\mathcal{A}$ kept $\mathbf{Y}$ in the $|0\rangle$ state in the second query, the result would be the output state $|y\rangle$, which is consistent with the measurement outcome $\mathcal{H}(x) = y$.

⁴⁷ Which, provided we want our random oracles to be collision resistant, should be set large enough anyway.

1. For all tuples (x, y) , $\mathcal{H}(x) = y$; and
2. The collection of tuples $\{(x, y)\}$ is contained in some set $\mathcal{S}$ that we care about.

Now, let the extractor $\mathcal{E}$ simulate the random oracle for $\mathcal{A}$ as a compressed random oracle. As stated in Thm. 5, the simulation will be perfect, and so we are guaranteed that with the same probability, $\mathcal{A}$ will output tuples as before.

After $\mathcal{A}$ halts, let $\mathcal{E}$ first move the output register of the oracle database to the primal domain by applying the Hadamard transformation to $\mathbf{D}_{\mathbf{Y}}$, and then observe the database, obtaining a list of queries $\mathbf{D}$. Let $\text{Adv}_{\mathcal{S}}(\mathcal{E}^{\mathcal{A}})$ be the probability that

1. For all tuples (x, y) , $\mathbf{D}[x] = y \neq \perp$; and
2. The collection of tuples $\{(x, y)\}$ is contained in $\mathcal{S}$.

Then we have that, for all adversaries $\mathcal{A}$,

$$\sqrt{\text{Adv}_{\mathcal{S}}(\mathcal{A}^{\mathcal{H}})} \leq \sqrt{\text{Adv}_{\mathcal{S}}(\mathcal{E}^{\mathcal{A}})} + \sqrt{\frac{\ell}{2^m}},$$

which implies the slightly looser (but perhaps more practical) bound

$$\text{Adv}_{\mathcal{S}}(\mathcal{A}^{\mathcal{H}}) \leq \text{Adv}_{\mathcal{S}}(\mathcal{E}^{\mathcal{A}}) + 2\sqrt{\frac{\ell}{2^m}} + \frac{\ell}{2^m}.$$

12.5 A Quantum Collision Bound

Hans: Lower bound on the number of queries required to find a collision, as a showcase of the database formulation and applying Lemma 14, while mirroring the birthday-bound analysis in Sect. 3.

Hans: See Claude's suggested strategy in comments

13 Example 2a: Classical CCA-Security from the $\mathbf{U}$ -transform

14 The Online-Extractability Technique

rough structure

1. Introduce the Online Extractability technique
2. From IND-CPA to IND-CCA via the U(check)-transform

Hans: With example being CCA security of U. Might be easier to start from OW-(q)PCA instead of adding re-encryption to the decryption oracle. At least part of the reason is that to use the extractability tool, we need the starting point to be a probabilistic PKE (need “control of the randomness generating oracle”?). So, randomized PKE achieving OW-qPCA $\rightarrow$ KEM achieving IND-CCA.

15 Example 2b: Post-Quantum CCA-Security from the $\mathbf{U}$ -transform

16 Example 3a: Classical CMA-Security from NIZKs

rough structure

1. Classical ROM proof of NMA to CMA

17 The Resampling Lemma and Fixed-Permutation O2H

rough structure

1. Show NMA NIZK to CMA sig

Hans: Serge Fehr and Yu-Hsuan Huang pointed out to me that at least *they* do not usually think of the adaptive-reprogramming tools (like the resampling lemma) as O2H variants. Maybe it makes sense to put them under a different header, even though many papers explicitly formulate them as O2H variants? The running example is anyway different. (Not sure what order to put these sections in.)

18 Example 3b: Post-Quantum CMA-Security from NIZKs

19 Example 4a: Classical NIZKs from the Fiat-Shamir Transformation

rough structure

1. Classical ROM proof of Sigma-protocol to NIZK

20 Rewinding Quantum Adversaries with Measure-and-Reprogram

rough structure

1. Discuss rewinding, and how to do it
2. Show proof of Sigma protocol to NIZK

Hans: Maybe leave for v2

21 Example 4b: Post-Quantum NIZKs from the Fiat-Shamir Transformation

References

- Aar05. Scott Aaronson. Quantum computing, postselection, and probabilistic polynomial-time. *Proc. R. Soc. A*, 461(2063):3473–3482, September 2005. <https://arxiv.org/abs/quant-ph/0412187>.
- Aar13. Scott Aaronson. *Quantum Computing since Democritus*. Cambridge University Press, 2013.
- ACM26. Charles H. Bennett and Gilles Brassard are the recipients of the 2025 ACM A.M. Turing Award. Association for Computing Machinery, <https://awards.acm.org/turing>, March 2026.
- Aha03. Dorit Aharonov. A simple proof that Toffoli and Hadamard are quantum universal. <https://arxiv.org/abs/quant-ph/0301040>, 2003.
- AHU19. Andris Ambainis, Mike Hamburg, and Dominique Unruh. Quantum security proofs using semi-classical oracles. In Alexandra Boldyreva and Daniele Micciancio, editors, *CRYPTO 2019, Part II*, volume 11693 of *LNCS*, pages 269–295. Springer, Cham, August 2019.
- BB84. Charles H. Bennett and Gilles Brassard. Quantum cryptography: Public key distribution and coin tossing. In *Proc. IEEE Int. Conf. on Computers, Systems and Signal Processing*, pages 175–179, Bangalore, December 1984. Reprinted in *Theoret. Comput. Sci.* 560:7–11, 2014.
- BBC⁺93. Charles H. Bennett, Gilles Brassard, Claude Crépeau, Richard Jozsa, Asher Peres, and William K. Wootters. Teleporting an unknown quantum state via dual classical and Einstein-Podolsky-Rosen channels. *Phys. Rev. Lett.*, 70(13):1895–1899, 1993.
- BDG20. Mihir Bellare, Hannah Davis, and Felix Günther. Separate your domains: NIST PQC KEMs, oracle cloning and read-only indistinguishability. In Anne Canteaut and Yuval Ishai, editors, *EUROCRYPT 2020, Part II*, volume 12106 of *LNCS*, pages 3–32. Springer, Cham, May 2020.
- Ben73. Charles H. Bennett. Logical reversibility of computation. *IBM J. Res. Dev.*, 17(6):525–532, 1973.
- BHH⁺19. Nina Bindel, Mike Hamburg, Kathrin Hövelmanns, Andreas Hülsing, and Edoardo Persichetti. Tighter proofs of CCA security in the quantum random oracle model. In Dennis Hofheinz and Alon Rosen, editors, *TCC 2019, Part II*, volume 11892 of *LNCS*, pages 61–90. Springer, Cham, December 2019.
- BHY09. Mihir Bellare, Dennis Hofheinz, and Scott Yilek. Possibility and impossibility results for encryption and commitment secure under selective opening. In Antoine Joux, editor, *EUROCRYPT 2009*, volume 5479 of *LNCS*, pages 1–35. Springer, Berlin, Heidelberg, April 2009.
- BP18. Daniel J. Bernstein and Edoardo Persichetti. Towards KEM unification. Cryptology ePrint Archive, Report 2018/526, 2018.
- BR93. Mihir Bellare and Phillip Rogaway. Random oracles are practical: A paradigm for designing efficient protocols. In Dorothy E. Denning, Raymond Pyle, Ravi Ganesan, Ravi S. Sandhu, and Victoria Ashby, editors, *ACM CCS 93*, pages 62–73. ACM Press, November 1993.
- BRTZ24. Mihir Bellare, Doreen Riepel, Stefano Tessaro, and Yizhao Zhang. Count corruptions, not users: Improved tightness for signatures, encryption and authenticated key exchange. In Kai-Min Chung and Yu Sasaki, editors, *ASIACRYPT 2024, Part II*, volume 15485 of *LNCS*, pages 326–360. Springer, Singapore, December 2024.
- BV97. Ethan Bernstein and Umesh Vazirani. Quantum complexity theory. *SIAM J. Comput.*, 26(5):1411–1473, 1997.
- Coo00. Stephen Cook. The P versus NP Problem. Clay Mathematics Institute, Millennium Prize Problems, <https://www.claymath.org/wp-content/uploads/2022/06/pvsnp.pdf>, 2000.
- Deu85. David Deutsch. Quantum theory, the Church-Turing principle and the universal quantum computer. *Proc. R. Soc. Lond. A*, 400:97–117, 1985.
- DH76. Whitfield Diffie and Martin E. Hellman. New directions in cryptography. *IEEE Transactions on Information Theory*, 22(6):644–654, 1976.
- FO99. Eiichiro Fujisaki and Tatsuaki Okamoto. Secure integration of asymmetric and symmetric encryption schemes. In Michael J. Wiener, editor, *CRYPTO’99*, volume 1666 of *LNCS*, pages 537–554. Springer, Berlin, Heidelberg, August 1999.
- GHHM21. Alex B. Grilo, Kathrin Hövelmanns, Andreas Hülsing, and Christian Majenz. Tight adaptive re-programming in the QROM. In Mehdi Tibouchi and Huaxiong Wang, editors, *ASIACRYPT 2021, Part I*, volume 13090 of *LNCS*, pages 637–667. Springer, Cham, December 2021.
- GLX24. Jiangxia Ge, Heming Liao, and Rui Xue. Measure-rewind-extract: Tighter proofs of one-way to hiding and CCA security in the quantum random oracle model. In Kai-Min Chung and Yu Sasaki, editors, *ASIACRYPT 2024, Part IV*, volume 15487 of *LNCS*, pages 3–34. Springer, Singapore, December 2024.
- Got99. Daniel Gottesman. The Heisenberg representation of quantum computers. In *Proc. XXII Int. Colloquium on Group Theoretical Methods in Physics*, pages 32–43. International Press, 1999. <https://arxiv.org/abs/quant-ph/9807006>.
- HHK17. Dennis Hofheinz, Kathrin Hövelmanns, and Eike Kiltz. A modular analysis of the Fujisaki-Okamoto transformation. In Yael Kalai and Leonid Reyzin, editors, *TCC 2017, Part I*, volume 10677 of *LNCS*, pages 341–371. Springer, Cham, November 2017.
- HHMS25. Kathrin Hövelmanns, Andreas Hülsing, Christian Majenz, and Fabrizio Sisinni. (Un)breakable curses - re-encryption in the Fujisaki-Okamoto transform. In Serge Fehr and Pierre-Alain Fouque, editors, *EUROCRYPT 2025, Part II*, volume 15602 of *LNCS*, pages 245–274. Springer, Cham, May 2025.

- Jae24. Joseph Jaeger. Nonadaptive one-way to hiding implies adaptive quantum reprogramming. Cryptology ePrint Archive, Report 2024/797, 2024.
- KSS⁺20. Veronika Kuchta, Amin Sakzad, Damien Stehlé, Ron Steinfeld, and Shifeng Sun. Measure-rewind-measure: Tighter quantum random oracle model proofs for one-way to hiding and CCA security. In Anne Canteaut and Yuval Ishai, editors, *EUROCRYPT 2020, Part III*, volume 12107 of *LNCS*, pages 703–728. Springer, Cham, May 2020.
- NC10. Michael Nielsen and Isaac Chuang. *Quantum Computation and Quantum Information*. Cambridge University Press, second edition, 2010.
- NIS24. Module-Lattice-Based Key-Encapsulation Mechanism Standard. National Institute of Standards and Technology, NIST FIPS PUB 203, U.S. Department of Commerce, August 2024.
- PZ24. Jiaxin Pan and Runzhi Zeng. Selective opening security in the quantum random oracle model, revisited. In Qiang Tang and Vanessa Teague, editors, *PKC 2024, Part II*, volume 14603 of *LNCS*, pages 92–122. Springer, Cham, April 2024.
- Sho04. Victor Shoup. Sequences of games: a tool for taming complexity in security proofs. Cryptology ePrint Archive, Report 2004/332, 2004.
- Tof80. Tommaso Toffoli. Reversible computing. In *Automata, Languages and Programming (ICALP)*, volume 85 of *LNCS*, pages 632–644. Springer, 1980. Also MIT tech. report MIT/LCS/TM-151.
- Unr14a. Dominique Unruh. Quantum position verification in the random oracle model. In Juan A. Garay and Rosario Gennaro, editors, *CRYPTO 2014, Part II*, volume 8617 of *LNCS*, pages 1–18. Springer, Berlin, Heidelberg, August 2014.
- Unr14b. Dominique Unruh. Revocable quantum timed-release encryption. In Phong Q. Nguyen and Elisabeth Oswald, editors, *EUROCRYPT 2014*, volume 8441 of *LNCS*, pages 129–146. Springer, Berlin, Heidelberg, May 2014.
- Wil17. Mark M. Wilde. *Quantum Information Theory*. Cambridge University Press, second edition, 2017.
- Zha19. Mark Zhandry. How to record quantum queries, and applications to quantum indistinguishability. In Alexandra Boldyreva and Daniele Micciancio, editors, *CRYPTO 2019, Part II*, volume 11693 of *LNCS*, pages 239–268. Springer, Cham, August 2019.

A The Toolbox

Here we collect the various tools introduced over the course of the tutorial.

Table 6. The QROM toolbox TEST. Security Parameter column shows what you need to reach 128-bit security for $\epsilon \leq 2^{-\lambda}$ and $q \leq 2^{80}$.

Tool	Tightness	Security Parameter	Statement
<i>Classical</i>			
Equivalent-until-bad	<i>tight</i>	$\lambda = 128$	Lemma 2 (??)
<i>Programming</i>			
O2H (Find)	$2\sqrt{(q+1) \cdot \epsilon}$	$\lambda = 338$	??
O2H (Guess)	$2q\sqrt{\epsilon}$	$\lambda = 418$	??
<i>Simulation</i>			
Compressed oracle	<i>tight</i>	$\lambda = 128$	??
<i>Extraction</i>			
Zhandry's extraction	$\epsilon + \frac{1}{\sqrt{2^m}}?$	?	??
Online extraction	(scheme-dependent)	?	??

A.1 Programming Tools

Lemma 2 (Equivalent-Until-Bad). *xyz*

Lemma 3 (Find-O2H, Classical). *Let $\mathcal{H}, \mathcal{G} : \{0, 1\}^n \rightarrow \{0, 1\}^m$ be random oracles such that for every $x \neq x^*$, $\mathcal{H}(x) = \mathcal{G}(x)$, whereas $\mathcal{H}(x^*)$ and $\mathcal{G}(x^*)$ are independently uniform. Let z be a value that depends arbitrarily on $\mathcal{H}, \mathcal{G}$, and x^* . Let $\mathbb{1}^*$ be an indicator oracle that on input x returns 1 if $x = x^*$, and otherwise returns 0.*

Then there is a simple, fully black-box extractor $\mathcal{E}$, such that for every distinguisher $\mathcal{D}$,

$$|\Pr[1 \leftarrow \mathcal{D}^{\mathcal{H}}(z)] - \Pr[1 \leftarrow \mathcal{D}^{\mathcal{G}}(z)]| \leq \Pr[x \leftarrow \mathcal{E}^{\mathcal{D}, \mathbb{1}^*}(z) : x = x^*].$$

Lemma 4 (Guess-O2H, Classical). *Let $\mathcal{H}, \mathcal{G} : \{0, 1\}^n \rightarrow \{0, 1\}^m$ be random oracles such that for every $x \neq x^*$, $\mathcal{H}(x) = \mathcal{G}(x)$, whereas $\mathcal{H}(x^*)$ and $\mathcal{G}(x^*)$ are independently uniform. Let z be a value that depends arbitrarily on $\mathcal{H}, \mathcal{G}$, and x^* .*

Then there is a simple, fully black-box extractor $\mathcal{E}$, such that for every distinguisher $\mathcal{D}$ that makes at most q calls to its random oracle,

$$|\Pr[1 \leftarrow \mathcal{D}^{\mathcal{H}}(z)] - \Pr[1 \leftarrow \mathcal{D}^{\mathcal{G}}(z)]| \leq q \cdot \Pr[x \leftarrow \mathcal{E}^{\mathcal{D}}(z) : x = x^*].$$

A.2 Simulation Tools

A.3 Extraction Tools

B Quantum Pseudocode: Additional Definitions

Here, we show how operations on qubit registers are defined for our quantum pseudocode, as introduced in Sect. 10.

B.1 Operating on Registers

First, we formally define for what it means for a unitary to be applied to a qubit register. The main idea is that, thanks to the requirement that the global state $|\mathbf{Q}\rangle$ is in a well-defined pure state, we can define an operation on sub-registers $\mathbf{R}$ of $\mathbf{Q}$ in terms of a larger operation on the whole state.

Definition 18 (Operating on Registers). *Let $\mathbf{Q}$ be an n -qubit quantum memory in quantum state $|\mathbf{Q}\rangle$, let the register $\mathbf{R}$ be an m -qubit subsystem of $\mathbf{Q}$, and let U be an $M \times M$ unitary matrix.*

Next, define $U_{\mathbf{R}}$ to be the $N \times N$ unitary matrix that has the effect of applying U on the qubits in $\mathbf{R}$ and the identity on the qubits in $\mathbf{Q} \setminus \mathbf{R}$. Then, the application of U to the qubits in register $\mathbf{R}$ is captured in the method $\text{Apply}_{\mathbf{Q}}(U \rightarrow \mathbf{R})$, defined as

$$\text{Apply}_{\mathbf{Q}}(U \rightarrow \mathbf{R}) := |\mathbf{Q}(t)\rangle \xrightarrow{U_{\mathbf{R}}} |\mathbf{Q}(t+1)\rangle .$$

Note how local operations affecting distant qubits through entanglement is captured by $\text{Apply}_{\mathbf{Q}}(U \rightarrow \mathbf{R})$ being an evolution of the global (pure) state $|\mathbf{Q}\rangle$.

To see that $U_{\mathbf{R}}$ is well-defined, note that it may be implemented as follows:

$$U_{\mathbf{R}} := \text{SORT}^\dagger([\mathbf{R}], [\mathbf{Q} \setminus \mathbf{R}]) (U \otimes \mathbb{1}) \text{SORT}([\mathbf{R}], [\mathbf{Q} \setminus \mathbf{R}]) ,$$

where $\text{SORT}([\mathbf{R}], [\mathbf{Q} \setminus \mathbf{R}])$ is the $N \times N$ unitary matrix, composed of a series of SWAP gates, that reorders the qubits in $\mathbf{Q}$ such that the qubits of $\mathbf{R}$ precede the qubits of $\mathbf{Q} \setminus \mathbf{R}$; $\text{SORT}^\dagger$ reverses the operation of SORT ; and $\mathbb{1}$ is the $2^{n-m} \times 2^{n-m}$ identity matrix.

B.2 Sending Registers

In order to say “Alice sends a qubit register to Bob”, we give Alice and Bob to the disjoint subregisters $\mathbf{A}$ and $\mathbf{B}$, respectively, of the total system $\mathbf{Q}$. Then, Alice sending register $\mathbf{R}$ to Bob simply amounts to transferring $\mathbf{R}$ from $\mathbf{A}$ to $\mathbf{B}$.

Definition 19 (Transferring a Subregister). *Let $\mathbf{A}$ and $\mathbf{B}$ be non-overlapping qubit registers. A sub-register $\mathbf{R}$ is said to be transferred from $\mathbf{A}$ to $\mathbf{B}$, denoted $\mathbf{A} \xrightarrow{\mathbf{R}} \mathbf{B}$, if the elements of $\mathbf{R}$ are simultaneously removed from $\mathbf{A}$ and added to $\mathbf{B}$:*

$$\mathbf{A} \xrightarrow{\mathbf{R}} \mathbf{B} \iff \mathbf{A} \leftarrow \mathbf{A} \setminus \mathbf{R}, \quad \mathbf{B} \leftarrow \mathbf{B} \cup \mathbf{R} .$$

B.3 Measuring Registers

The final piece of the puzzle is defining the measurement operation $\text{Meas}(\mathbf{R})$.

Definition 20 (Measuring a Register). *Let $\mathbf{Q}$ be an n -qubit quantum memory in pure state $|\mathbf{Q}\rangle$, let the register $\mathbf{R}$ be an m -qubit subsystem of $\mathbf{Q}$, and let $\mathcal{M}$ be an m -qubit projective measurement that check the state for membership in Hilbert subspaces $\{\mathcal{S}_i\}$.*

Next, define $\mathcal{M}_{\mathbf{R}}$ to be the n -qubit projective measurement that has the effect of applying $\mathcal{M}$ on the qubits in $\mathbf{R}$ and the identity on the qubits in $\mathbf{Q} \setminus \mathbf{R}$.

Then, performing measurement $\mathcal{M}$ on the qubits in register $\mathbf{R}$ is captured in the (probabilistic) method $\text{Meas}_{\mathbf{Q}}(\mathcal{M} : \mathbf{R})$ which observes $|\mathbf{Q}\rangle$ under $\mathcal{M}_{\mathbf{R}}$. Observing outcome i , designated by the measurement procedure returning value m_i , is written

$$i \leftarrow \text{Meas}_{\mathbf{Q}}(\mathcal{M} : \mathbf{R}) ,$$

and the post-measurement will be

$$|\mathbf{Q}\rangle \xrightarrow{\mathcal{M}} |\mathbf{Q}\rangle_{\mathcal{S}_i} ,$$

where $|\mathbf{Q}\rangle_{\mathcal{S}_i}$ is the result of projecting the qubits of $\mathbf{R}$ onto subspace $\mathcal{S}_i$, and applying the identity to the rest.

Finally, observe that the projective measurement $\mathcal{M}_{\mathbf{R}}$ can be constructed similarly to $U_{\mathbf{R}}$ above.

C Thinking in Dual Alphabets

C.1 Dual Alphabets

We introduce the dual-alphabet notation, which allows us to reason about qubit states in both the computational and the Hadamard basis without having to explicitly apply the Hadamard transformation every time. Recall the the Hadamard basis states:

$$|+\rangle := \frac{|\mathbf{0}\rangle + |\mathbf{1}\rangle}{\sqrt{2}}, \quad |-\rangle := \frac{|\mathbf{0}\rangle - |\mathbf{1}\rangle}{\sqrt{2}}.$$

For $b \in \{\mathbf{0}, \mathbf{1}\}$, define the state

$$|\hat{b}\rangle := \frac{|\mathbf{0}\rangle + (-1)^b |\mathbf{1}\rangle}{\sqrt{2}},$$

i.e. $|\hat{b}\rangle = |+\rangle$ if $b = \mathbf{0}$, and $|\hat{b}\rangle = |-\rangle$ if $b = \mathbf{1}$. Notice how we are invited to think of the variable $\hat{b}$ as a bit defined over the alphabet $\{\hat{\mathbf{0}}, \hat{\mathbf{1}}\} := \{+, -\}$ in place of our usual alphabet $\{\mathbf{0}, \mathbf{1}\}$, since $|\hat{\mathbf{0}}\rangle = |+\rangle$ and $|\hat{\mathbf{1}}\rangle = |-\rangle$. We will refer to this as the *dual* alphabet. And just as $|\hat{b}\rangle$ can be expressed in the usual alphabet $\{\mathbf{0}, \mathbf{1}\}$ using the definition above, so can a state $|b\rangle$ be expressed in terms of the dual alphabet $\{\hat{\mathbf{0}}, \hat{\mathbf{1}}\}$:

$$|\hat{b}\rangle = \frac{|\mathbf{0}\rangle + (-1)^b |\mathbf{1}\rangle}{\sqrt{2}}, \quad |b\rangle = \frac{|\hat{\mathbf{0}}\rangle + (-1)^b |\hat{\mathbf{1}}\rangle}{\sqrt{2}}.$$

Going to multiple qubits, Hadamard-transforming an n -qubit computational-basis state $|x\rangle = |\mathbf{101} \dots \mathbf{01}\rangle$ just applies the Hadamard gate to each qubit individually:

$$|x\rangle = |\mathbf{101} \dots \mathbf{01}\rangle \xrightarrow{H^{\otimes n}} |-\mathbf{+} \dots \mathbf{+}\rangle = |\hat{\mathbf{1}}\hat{\mathbf{0}}\hat{\mathbf{1}} \dots \hat{\mathbf{0}}\hat{\mathbf{1}}\rangle =: |\hat{x}\rangle.$$

The result is a state labelled by a *bitstring* $\hat{x}$, again defined over the dual alphabet: $\hat{x} \in \{\hat{\mathbf{0}}, \hat{\mathbf{1}}\}^n = \{+, -\}^n$. Hence, our convention generalizes straightforwardly: Writing out the state $|\hat{x}\rangle$ in terms of the computational basis, we see that we can *translate* a state between alphabets using the usual rule for applying a Hadamard-transformation, expressed in the below definition.

Definition 21. Let $x \in \{\mathbf{0}, \mathbf{1}\}^n$. Then,

$$|\hat{x}\rangle = \frac{1}{\sqrt{2^n}} \sum_{z \in \{\mathbf{0}, \mathbf{1}\}^n} (-1)^{x \cdot z} |z\rangle, \quad |x\rangle = \frac{1}{\sqrt{2^n}} \sum_{z \in \{\mathbf{0}, \mathbf{1}\}^n} (-1)^{x \cdot z} |\hat{z}\rangle.$$

Table 7. Our dual-alphabet convention, and how to use it.

Convention	Rule/definition	Notes/examples
Hat = alphabet flag	$\hat{x}$ is the same value as x , expressed as a bitstring over the dual alphabet $\{+, -\} = \{\hat{\mathbf{0}}, \hat{\mathbf{1}}\}$.	Example: If $x = 5 = \mathbf{101}$, then $ \hat{x}\rangle = \hat{5}\rangle = \hat{\mathbf{1}}\hat{\mathbf{0}}\hat{\mathbf{1}}\rangle = -\mathbf{+}\mathbf{-}\rangle$.
Dual-domain states	$ \hat{b}\rangle := \frac{1}{\sqrt{2}}(\mathbf{0}\rangle + (-1)^b \mathbf{1}\rangle)$, $ \hat{x}\rangle = H^{\otimes n} x\rangle$.	Related to how the QKD literature writes $H^\theta x\rangle$ for a state expressed in the θ basis.
Hats are state labels	Hatted values never appear in arithmetic outside of kets.	Forbidden: $(-1)^{\hat{x} \cdot y}$, $f(\hat{x})$.
Wide hats go over arithmetic expressions.	Hats are applied <i>last</i> .	Examples: $ f(\widehat{x})\rangle$, $ \widehat{x \oplus y}\rangle$.
Xor of labels	$\hat{x} \oplus \hat{y} := \widehat{x \oplus y}$.	The one exception to the above “no arithmetic” rule; $(\widehat{\cdot})$ is a group homomorphism.
Fresh letters for dummy indices	Write $ x\rangle = 1/\sqrt{2^n} \sum_z (-1)^{x \cdot z} \hat{z}\rangle$.	$ x\rangle = 1/\sqrt{2^n} \sum_{\hat{x}} (-1)^{x \cdot \hat{x}} \hat{x}\rangle$ would not be well-defined under our convention.
Outcomes are plain	Measurement results always live in $\{\mathbf{0}, \mathbf{1}\}^n$.	Write $x \leftarrow \hat{\mathbf{X}}$, not $\hat{x} \leftarrow \hat{\mathbf{X}}$.

Note how hats only ever show up inside kets: they are to be used as *state labels*, never in the surrounding arithmetic. Relatedly, since the operation $\widehat{(\cdot)}$ takes any value expressed over $\{0, 1\}$ and translates it to that same value expressed over $\{\hat{0}, \hat{1}\}$, dummy variables must get a new letter z . Finally, classical values such as observed outcomes are always defined over $\{0, 1\}$ and may never carry hats. See Table 7 for a helpful overview of how to use our conventions.

To re-iterate the point: We are not *transforming* the states in the above identities – not a single qubit gate was applied – we are merely looking at the same state with a new pair of glasses! This is what allows us to express these as *identities*, rather than as an *update* (with arrows in place of equality signs).

So, measuring a state $|x\rangle$ in the computational basis yields outcome x with certainty, measuring a state $|\hat{x}\rangle$ in the computational basis yields a uniformly random outcome $x' \leftarrow \$\{0, 1\}^n$; and while measuring a state $|\hat{x}\rangle$ in the Hadamard basis yields outcome x with certainty, measuring a computational-basis state $|x\rangle$ in the Hadamard basis will *also* yield a uniformly random outcome $x' \in \{0, 1\}^n$.

Over the course of this section, we hope to convince you that the ability to translate between alphabets on the fly, expressing any given state in whichever basis it is the simplest (even at times using different alphabets for different registers), will turn out to be a very useful trick that simplifies many intermediate calculations, while bringing to the forefront many of the unique aspects that separate the quantum and classical worlds. Indeed, experience tells us that playing around with the dual alphabets can be very helpful in building that coveted quantum intuition!

C.2 Dual-Alphabet Pseudocode

We have introduced the notation $\mathbf{B} \leftarrow \oplus \mathbf{A}$ for the operation that, if register $\mathbf{A}$ is in state $|a\rangle$ and register $\mathbf{B}$ is in state $|b\rangle$, applies the operation

$$|a, b\rangle_{\mathbf{AB}} \rightarrow |a, b \oplus a\rangle_{\mathbf{AB}} .$$

If a and b are bits ($a, b \in \{0, 1\}$), then this is just a CNOT from qubit $\mathbf{A}$ to qubit $\mathbf{B}$. For any function f that we know how to compute, writing $\mathbf{B} \leftarrow \oplus f(\mathbf{A})$ denotes the standard reversible function evaluation:

$$|a, b\rangle_{\mathbf{AB}} \rightarrow |a, b \oplus f(a)\rangle_{\mathbf{AB}} .$$

What are the equivalent operations for Hadamard-basis states? Well, we know that the Z (phase-flip) gate plays the role of the X (not) gate in the Hadamard basis; in fact, with the bases being dual, Z and X switch roles exactly when moving between the computational and Hadamard bases:

$$\begin{aligned} |0\rangle &\xrightarrow{X} |1\rangle, & |1\rangle &\xrightarrow{X} |0\rangle & |0\rangle &\xrightarrow{Z} |0\rangle, & |1\rangle &\xrightarrow{Z} -|1\rangle \\ |+\rangle &\xrightarrow{Z} |-\rangle, & |-\rangle &\xrightarrow{Z} |+\rangle & |+\rangle &\xrightarrow{X} |+\rangle, & |-\rangle &\xrightarrow{X} -|-\rangle \end{aligned}$$

which in our notation may be written more succinctly as

$$\begin{aligned} |b\rangle &\xrightarrow{X} |b \oplus 1\rangle, & |b\rangle &\xrightarrow{Z} (-1)^b |b\rangle \\ |\hat{b}\rangle &\xrightarrow{Z} |\hat{b} \oplus \hat{1}\rangle, & |\hat{b}\rangle &\xrightarrow{X} (-1)^b |\hat{b}\rangle \end{aligned}$$

We want to identify the equivalent of the CNOT operation, a.k.a. CX, in the Hadamard domain, i.e. the operation

$$|\hat{a}, \hat{b}\rangle \rightarrow |\hat{a}, \hat{b} \oplus \hat{a}\rangle .$$

This *almost* looks like a controlled-Z, or CZ, except that we are controlling on $|\hat{a}\rangle = |\hat{1}\rangle$. In other words, the Z operation is only applied in the branches of the superposition where register $\mathbf{A}$ finds itself in state $|\hat{1}\rangle = |-\rangle$ state. Call this operation $\hat{CZ}$, defined as:

$$\hat{CZ}_{\mathbf{AB}} = |+\rangle\langle+|_{\mathbf{A}} \otimes \mathbf{I}_{\mathbf{B}} + |-\rangle\langle-|_{\mathbf{A}} \otimes \mathbf{Z}_{\mathbf{B}} ,$$

i.e. the unitary that picks out the $|-\rangle_{\mathbf{A}}$ branches and applies Z to register $\mathbf{B}$ there, and elsewhere applies the identity. To see that this is the correct operation, simply notice that since X and Z switch roles between domains, we might as well write $Z = \hat{X}$, in which case $\hat{CZ} = \hat{C}\hat{X}$.

Still, it is instructive to see explicitly how it all works out, so let us apply $\hat{C}Z$ to the state $|\hat{a}, \hat{b}\rangle$ and see what happens by relying on the standard phase-flipping effect of Z on computational-basis states:

$$\begin{aligned} |\hat{a}, \hat{b}\rangle &= |\hat{a}\rangle \left(\frac{|\mathbf{0}\rangle + (-1)^b |\mathbf{1}\rangle}{\sqrt{2}} \right) \\ &\xrightarrow{\hat{C}Z} |\hat{a}\rangle \left(\frac{|\mathbf{0}\rangle + (-1)^b (-1)^a |\mathbf{1}\rangle}{\sqrt{2}} \right) = |\hat{a}\rangle \left(\frac{|\mathbf{0}\rangle + (-1)^{b \oplus a} |\mathbf{1}\rangle}{\sqrt{2}} \right) = |\hat{a}, \widehat{b \oplus a}\rangle =: |\hat{a}, \hat{b} \oplus \hat{a}\rangle, \end{aligned}$$

where we have defined the xor between two dual-alphabet strings $\hat{a} \oplus \hat{b}$ to simply be the dual-alphabet expression of the xored value $\widehat{a \oplus b}$. (In other words, $\widehat{(\cdot)}$ is a group homomorphism over $(\mathbb{Z}_2, \oplus)$.) By translating back to the dual alphabet in the last step, we find that this is indeed the operation we sought, namely the dual-domain equivalent of the CNOT (or CX) operation.

For multi-qubit states, recall that the operation $\mathbf{Y} \leftarrow \oplus \mathbf{X}$ on two equally-sized registers $\mathbf{X}$ and $\mathbf{Y}$, denotes the operation:

$$|x, y\rangle \rightarrow |x, y \oplus x\rangle,$$

where now, $y \oplus x$ denotes the bitwise xor of x and y . Hence, this operation can be implemented by simply applying CX gates between corresponding qubits, so that $|x_i, y_i\rangle \rightarrow |x_i, y_i \oplus x_i\rangle$. It is clear that the corresponding Hadamard-domain operation must be to apply $\hat{C}Z$ between the corresponding qubits of $\mathbf{X}$ and $\mathbf{Y}$, so that $|\hat{x}_i, \hat{y}_i\rangle \rightarrow |\hat{x}_i, \widehat{y_i \oplus x_i}\rangle$, and

$$|\hat{x}, \hat{y}\rangle \rightarrow |\hat{x}, \widehat{x \oplus y}\rangle =: |\hat{x}, \hat{x} \oplus \hat{y}\rangle.$$

We will denote this operation $\mathbf{Y} \leftarrow \hat{\ominus} \mathbf{X}$, where the $\ominus$ indicates the Z operation (in reference to its association with phase-flipping), and the hat serves as a reminder that we are controlling on the states $|\hat{\mathbf{1}}\rangle$ rather than $|\mathbf{1}\rangle$.⁴⁸

C.3 Properties of $\leftarrow \oplus$ and $\leftarrow \hat{\ominus}$.

Let us explore how $\leftarrow \oplus$ and $\leftarrow \hat{\ominus}$ act on various initial states, starting with single qubits. By design, the $\leftarrow \oplus$ operator takes states $|a, b\rangle$ to $|a, b \oplus a\rangle$, while $\leftarrow \hat{\ominus}$ takes states $|\hat{a}, \hat{b}\rangle$ to $|\hat{a}, \hat{b} \oplus \hat{a}\rangle$. What about the other way around? Or for that matter, mixed-alphabet states like $|\hat{a}, b\rangle$ and $|a, \hat{b}\rangle$? In other words, we would like to fill out the missing cells in the following table:

AB initial state	B $\leftarrow \oplus$ A yields	B $\leftarrow \hat{\ominus}$ A yields
$ a, b\rangle$	$ a, b \oplus a\rangle$	
$ a, \hat{b}\rangle$		
$ \hat{a}, b\rangle$		
$ \hat{a}, \hat{b}\rangle$		$ \hat{a}, \hat{b} \oplus \hat{a}\rangle$

Case 1: $|a, b\rangle$, $\mathbf{B} \leftarrow \hat{\ominus} \mathbf{A}$. Let us begin by asking what is perhaps the most pressing question: How does $\leftarrow \hat{\ominus}$ affect computational-basis states like $|a, b\rangle$? Recall that $\leftarrow \hat{\ominus}$ does not control on $|a\rangle = |\mathbf{1}\rangle$, but rather on $|\hat{a}\rangle = |\hat{\mathbf{1}}\rangle$, so this does *not* act like a standard controlled- Z operation. To reveal the relevant controls, begin by translating $|a\rangle$ to the dual alphabet:

$$|a, b\rangle = \frac{|\hat{\mathbf{0}}\rangle + (-1)^a |\hat{\mathbf{1}}\rangle}{\sqrt{2}} |b\rangle.$$

For single-qubit registers, the operation $\mathbf{B} \leftarrow \hat{\ominus} \mathbf{A}$ just applies the $\hat{C}Z$ operation we defined earlier, which only fires in the $|\hat{\mathbf{1}}\rangle$ branch of the superposition. Meanwhile, Z applied to a computational-basis state $|b\rangle$ flips the phase iff $b = 1$. The result:

$$\frac{|\hat{\mathbf{0}}\rangle + (-1)^a |\hat{\mathbf{1}}\rangle}{\sqrt{2}} |b\rangle \xrightarrow{\hat{C}Z} \frac{|\hat{\mathbf{0}}\rangle + (-1)^a (-1)^b |\hat{\mathbf{1}}\rangle}{\sqrt{2}} |b\rangle = \frac{|\hat{\mathbf{0}}\rangle + (-1)^{a \oplus b} |\hat{\mathbf{1}}\rangle}{\sqrt{2}} |b\rangle = |a \oplus b, b\rangle,$$

⁴⁸ This suggests the additional operators $\leftarrow \ominus$ and $\leftarrow \hat{\oplus}$, which would indeed be perfectly well-defined, though we will not need them.

where we used the translation rule a second time to translate the state back to the computational-basis.

So, the operation $\mathbf{B} \leftarrow \hat{\ominus} \mathbf{A}$ takes a state $|a, b\rangle$ to the state $|a \oplus b, b\rangle$. In other words, it acts exactly like the operation $\mathbf{A} \leftarrow \oplus \mathbf{B}$, our previous operation with the registers flipped! Since operators $\leftarrow \hat{\ominus}$ and $\leftarrow \oplus$ switch roles when moving from the computational basis (standard alphabet) to the Hadamard basis (dual alphabet), we may conclude right away that the operation $\mathbf{B} \leftarrow \oplus \mathbf{A}$ must similarly take a state $|\hat{a}, \hat{b}\rangle$ to the state $|\hat{a} \oplus \hat{b}, \hat{b}\rangle$, i.e. the same as $\mathbf{A} \leftarrow \hat{\ominus} \mathbf{B}$. Let's add both to our table:

$\mathbf{AB}$ initial state	$\mathbf{B} \leftarrow \oplus \mathbf{A}$ yields	$\mathbf{B} \leftarrow \hat{\ominus} \mathbf{A}$ yields
$ a, b\rangle$	$ a, b \oplus a\rangle$	$ a \oplus b, b\rangle$
$ a, \hat{b}\rangle$		
$ \hat{a}, b\rangle$		
$ \hat{a}, \hat{b}\rangle$	$ \hat{a}, \hat{b} \oplus \hat{a}\rangle$	$ \hat{a}, \hat{b} \oplus \hat{a}\rangle$

The table surfaces two symmetries: First, the diagonal hat/non-hat symmetry from swapping $a, b, \leftarrow \oplus \leftrightarrow \hat{a}, \hat{b}, \leftarrow \hat{\ominus}$, which is simply saying that the computational basis and the Hadamard basis are dual, and that $\leftarrow \oplus$ and $\leftarrow \hat{\ominus}$ play the role of the other in the other domain.

The more interesting symmetry is the one we just discovered, that switching $\leftarrow \oplus$ for $\leftarrow \hat{\ominus}$ and reversing direction leaves the operation invariant. Since the symmetry holds for every state in the qubit basis $\{|0\rangle, |1\rangle\}$, the symmetry must hold for *every* qubit state; and since the $\leftarrow \oplus$ and $\leftarrow \hat{\ominus}$ operations are implemented bitwise for multi-qubit registers, the same must be true in general.

Lemma 15. *Let $\mathbf{X}$ and $\mathbf{Y}$ be n -qubit. Then applying $\leftarrow \oplus$ from $\mathbf{X}$ to $\mathbf{Y}$ is equivalent to applying $\leftarrow \hat{\ominus}$ from $\mathbf{Y}$ to $\mathbf{X}$, and vice versa:*

$$\mathbf{Y} \leftarrow \oplus \mathbf{X} \equiv \mathbf{X} \leftarrow \hat{\ominus} \mathbf{Y}, \quad \mathbf{X} \leftarrow \hat{\ominus} \mathbf{Y} \equiv \mathbf{Y} \leftarrow \oplus \mathbf{X}.$$

Lemma 15 may seem surprising, but it is actually just a restatement of the well-known fact that CNOT switches direction under the Hadamard transformation – the target becomes the control, and the control becomes the target: $H^{\otimes 2} CX_{\mathbf{AB}} H^{\otimes 2} = CX_{\mathbf{BA}}$.

Case 2: $|a, \hat{b}\rangle, \mathbf{B} \leftarrow \oplus \mathbf{A}$. Next up is our first multi-alphabet state, $|a, \hat{b}\rangle$. What is the effect of the operation $\mathbf{B} \leftarrow \oplus \mathbf{A}$ on this state, i.e. a CNOT from $|a\rangle$ to $|\hat{b}\rangle$? Well, recall that X and Z play opposite roles in the two domains, so applying an X gate to a state $|\hat{b}\rangle$ will have the same effect on it as if we had applied a Z gate to $|b\rangle$: If $\hat{b} = \hat{1}$, then the phase flips, and otherwise nothing happens. Since this operation is controlled on $a = 1$, we need that *both* a and b are 1 for the phase to flip; in other words, we need that their *product* is 1:

$$|a, \hat{b}\rangle \xrightarrow{CX} (-1)^{a \cdot b} |a, \hat{b}\rangle.$$

The only possible effect is a flip of the global phase. With global phases being unphysical, this means that in isolation, the operation acts on the state like the identity.⁴⁹ But of course, we will often *not* be applying these operations in isolation, and the moment the operation is controlled on other qubit states (such as when applied inside a “branch loop”), the phase is no longer global, and may certainly *not* be ignored.

Using the diagonal symmetry of our table, we may again immediately conclude that the operation $\mathbf{B} \leftarrow \hat{\ominus} \mathbf{A}$ with initial state $|\hat{a}, b\rangle$ will have the same overall effect:

$$|\hat{a}, b\rangle \xrightarrow{\hat{CZ}} (-1)^{a \cdot b} |\hat{a}, b\rangle.$$

Let's add both to the table:

⁴⁹ Another way to say this is that $|\hat{a}, b\rangle$ is an eigenstate of the $\hat{CZ}$ operator, which follows from the fact that $|b\rangle$ is an eigenstate of the Z operator.

AB initial state	B $\leftarrow \oplus$ A yields	B $\leftarrow \hat{\ominus}$ A yields
$ a, b\rangle$	$ a, b \oplus a\rangle$	$ a \oplus b, b\rangle$
$ a, \hat{b}\rangle$	$(-1)^{a \cdot b} a, \hat{b}\rangle$	
$ \hat{a}, b\rangle$		$(-1)^{a \cdot b} \hat{a}, b\rangle$
$ \hat{a}, \hat{b}\rangle$	$ \hat{a}, \hat{b} \oplus \hat{a}\rangle$	$ \hat{a}, \hat{b} \oplus \hat{a}\rangle$

Case 3: $|\hat{a}, b\rangle$, **B** $\leftarrow \oplus$ **A**. Finally, what is the effect of applying **B** $\leftarrow \oplus$ **A** on the initial state $|\hat{a}, b\rangle$? This one *should* look familiar to you: We are applying a CNOT from a qubit in superposition to a qubit in a computational basis state – which is exactly how entangled qubit pairs are usually prepared, just with $a = 0$, $b = 0$! In that case, we get the familiar maximally-entangled “Bell state” $1/\sqrt{2}(|00\rangle + |11\rangle)$. Let’s look at the general case: register **A** is initialized to $|\hat{a}\rangle$ and register **B** is initialized to $|b\rangle$. Since $\leftarrow \oplus$ controls on computational-basis values, begin by expressing $|\hat{a}\rangle$ in the standard alphabet. We get:

$$|\hat{a}, b\rangle = \frac{|\mathbf{0}\rangle + (-1)^a |\mathbf{1}\rangle}{\sqrt{2}} |b\rangle \xrightarrow{\text{CX}} \frac{|\mathbf{0}, b\rangle + (-1)^a |\mathbf{1}, b \oplus \mathbf{1}\rangle}{\sqrt{2}}.$$

The result is one of the four states of the *Bell basis*, depending on the values of a and b :

$$\begin{aligned} (a, b) = (0, 0) &\implies \frac{|\mathbf{00}\rangle + |\mathbf{11}\rangle}{\sqrt{2}} =: |\beta_{00}\rangle, & (a, b) = (1, 0) &\implies \frac{|\mathbf{00}\rangle - |\mathbf{11}\rangle}{\sqrt{2}} =: |\beta_{10}\rangle, \\ (a, b) = (0, 1) &\implies \frac{|\mathbf{01}\rangle + |\mathbf{10}\rangle}{\sqrt{2}} =: |\beta_{01}\rangle, & (a, b) = (1, 1) &\implies \frac{|\mathbf{01}\rangle - |\mathbf{10}\rangle}{\sqrt{2}} =: |\beta_{11}\rangle. \end{aligned}$$

So, applying $\leftarrow \oplus$ to $|\hat{a}, b\rangle$ gives the Bell state $|\beta_{ab}\rangle$. Diagonal table-symmetry then tells us that applying $\leftarrow \hat{\ominus}$ to the state $|a, \hat{b}\rangle$ must yield the same Bell state, but with hats:

$$|a, \hat{b}\rangle = \frac{|\hat{\mathbf{0}}\rangle + (-1)^a |\hat{\mathbf{1}}\rangle}{\sqrt{2}} |\hat{b}\rangle \xrightarrow{\hat{\text{CZ}}} \frac{|\hat{\mathbf{0}}, \hat{b}\rangle + (-1)^a |\hat{\mathbf{1}}, \hat{b} \oplus \hat{\mathbf{1}}\rangle}{\sqrt{2}} =: |\hat{\beta}_{ab}\rangle.$$

Interestingly, Lemma 15 tells us that the state $|\hat{\beta}_{ab}\rangle$ must *also* be what we’d get if we had applied the $\leftarrow \oplus$ operation in reverse, **A** $\leftarrow \oplus$ **B**, starting from the same initial state $|a, \hat{b}\rangle$. But clearly, this would just give us the same Bell state as before, except with the roles of a and b swapped:

$$|a, \hat{b}\rangle \xrightarrow{\text{CX}_{\mathbf{BA}}} \frac{|a, \mathbf{0}\rangle + (-1)^b |a \oplus \mathbf{1}, \mathbf{1}\rangle}{\sqrt{2}} = (-1)^{a \cdot b} |\beta_{ba}\rangle,$$

where the extra minus sign in the $a = b = 1$ case accounts for the reversed order. We are led to conclude that these two states must in fact be *the same state*:

$$|\hat{\beta}_{ab}\rangle = \frac{|\hat{\mathbf{0}}, \hat{b}\rangle + (-1)^a |\hat{\mathbf{1}}, \hat{b} \oplus \hat{\mathbf{1}}\rangle}{\sqrt{2}} = \frac{|a, \mathbf{0}\rangle + (-1)^b |a \oplus \mathbf{1}, \mathbf{1}\rangle}{\sqrt{2}} = (-1)^{a \cdot b} |\beta_{ba}\rangle.$$

What we have just discovered is another well-known fact, namely that the Hadamard transformation acts like a permutation on the set of Bell states – and that without ever touching a double sum!

With these additions, our Table 8 is finally complete.

Table 8. The effect of $\leftarrow \oplus$ and $\leftarrow \hat{\ominus}$ on the four single-qubit, dual-basis states.

AB initial state	B $\leftarrow \oplus$ A yields	B $\leftarrow \hat{\ominus}$ A yields
$ a, b\rangle$	$ a, b \oplus a\rangle$	$ a \oplus b, b\rangle$
$ a, \hat{b}\rangle$	$(-1)^{a \cdot b} a, \hat{b}\rangle$	$ \hat{\beta}_{ab}\rangle = (-1)^{a \cdot b} \beta_{ba}\rangle$
$ \hat{a}, b\rangle$	$ \beta_{ab}\rangle = (-1)^{a \cdot b} \hat{\beta}_{ba}\rangle$	$(-1)^{a \cdot b} \hat{a}, b\rangle$
$ \hat{a}, \hat{b}\rangle$	$ \hat{a} \oplus \hat{b}, \hat{b}\rangle$	$ \hat{a}, \hat{b} \oplus \hat{a}\rangle$

The Multi-Qubit Case. Since the operations are applied bitwise, this immediately generalizes to multi-qubit states:

$$\mathbf{Y} \leftarrow \oplus \mathbf{X} : |x, y\rangle \rightarrow |x, y \oplus x\rangle, \quad \mathbf{Y} \leftarrow \hat{\ominus} \mathbf{X} : |\hat{x}, \hat{y}\rangle \rightarrow |\hat{x}, \hat{y} \oplus \hat{x}\rangle.$$

Table 9 provides a summary.

Hans: Todo: Expand.

Table 9. The effect of applying the $\leftarrow \oplus$ and $\leftarrow \hat{\ominus}$ operator on the four possible dual-basis initial states. Here, $|\beta_{xy}\rangle$ denotes the generalized Bell states $|\beta_{xy}\rangle := 2^{-n/2} \sum_z (-1)^{x \cdot z} |z, y \oplus z\rangle$.

Initial state	$\mathbf{Y} \leftarrow \oplus \mathbf{X}$ yields	$\mathbf{Y} \leftarrow \hat{\ominus} \mathbf{X}$ yields
$ x, y\rangle$	$ x, y \oplus x\rangle$	$ x \oplus y, y\rangle$
$ x, \hat{y}\rangle$	$(-1)^{x \cdot y} x, \hat{y}\rangle$	$ \hat{\beta}_{xy}\rangle = (-1)^{x \cdot y} \beta_{yx}\rangle$
$ \hat{x}, y\rangle$	$ \beta_{xy}\rangle = (-1)^{x \cdot y} \hat{\beta}_{yx}\rangle$	$(-1)^{x \cdot y} \hat{x}, y\rangle$
$ \hat{x}, \hat{y}\rangle$	$ \hat{x} \oplus \hat{y}, \hat{y}\rangle$	$ \hat{x}, \hat{y} \oplus \hat{x}\rangle$

Function Evaluation. We originally introduced $\leftarrow \oplus$ to do reversible function evaluation, $\mathbf{Y} \leftarrow \oplus f(\mathbf{X})$, so it is natural to wonder how such an operation behaves for the various initial-state cases; see Table 10.

Hans: Todo: Expand.

Hans: make filling out Table 10 an exercise?

Table 10. Function evaluation table. Notice how setting f to the identity recovers Table 9.

Initial state	$\mathbf{Y} \leftarrow \oplus f(\mathbf{X})$ yields	$\mathbf{Y} \leftarrow \hat{\ominus} f(\mathbf{X})$ yields
$ x, y\rangle$	$ x, y \oplus f(x)\rangle$	$1/\sqrt{2^n} \sum_z (-1)^{x \cdot z \oplus f(z) \cdot y} \hat{z}, y\rangle$
$ x, \hat{y}\rangle$	$(-1)^{y \cdot f(x)} x, \hat{y}\rangle$	$1/\sqrt{2^n} \sum_z (-1)^{x \cdot z} \hat{z}, \hat{y} \oplus f(z)\rangle$
$ \hat{x}, y\rangle$	$1/\sqrt{2^n} \sum_z (-1)^{x \cdot z} z, y \oplus f(z)\rangle$	$(-1)^{y \cdot f(x)} \hat{x}, y\rangle$
$ \hat{x}, \hat{y}\rangle$	$1/\sqrt{2^n} \sum_z (-1)^{x \cdot z \oplus f(z) \cdot y} z, \hat{y}\rangle$	$ \hat{x}, \hat{y} \oplus f(x)\rangle$

C.4 Demonstrations

We next provide some examples of quantum algorithms and protocols expressed and analyzed using the conventions we have introduced, in order to demonstrate their expressiveness and helpfulness.

The Deutsch–Jozsa Algorithm. The Deutsch–Jozsa algorithm learns in a single function call whether a function $f : \{0, 1\}^n \rightarrow \{0, 1\}$ is constant or balanced (evaluates to 0 and 1 on exactly half of the inputs each), under the promise that it is either one or the other. The algorithm is a multi-qubit generalization of Deutsch’s algorithm, which is notable for being the first example of a task on which a quantum computer can provably outperform a classical computer, in the sense that any classical algorithm would need two function evaluations (or up to $2^{n-1} + 1$ in the general case) to arrive at the answer.

The procedure is given in pseudocode in Fig. 19, shown in two equivalent implementations depending on whether you start in the Hadamard basis and apply computational-basis function evaluations (which is how the algorithm is usually presented), or if you start from the computational basis and apply a Hadamard-basis function evaluation, which brings a slightly different perspective to the same procedure.

Let us analyze the algorithm via its computational-basis implementation shown on the right. The operation $\leftarrow \hat{\ominus}$ applies the $\hat{C}Z$ bitwise from the control to the target, conditioned on the control being in

Deutsch–Jozsa	Alt. implementation
01 $\mathbf{XB} \leftarrow \hat{0}^n, \hat{1}\rangle$	01 $\mathbf{XB} \leftarrow \mathbf{0}^n, \mathbf{1}\rangle$
02 $\mathbf{B} \leftarrow \oplus f(\mathbf{X})$	02 $\mathbf{B} \leftarrow \hat{\oplus} f(\mathbf{X})$
03 $x \leftarrow \hat{\leftarrow} \mathbf{X}$	03 $x \leftarrow \leftarrow \mathbf{X}$
04 return x	04 return x

Fig. 19. The Deutsch–Jozsa algorithm for determining whether a given function $f : \{0, 1\}^n \rightarrow \{0, 1\}$ is constant or balanced.

state $|\hat{1}\rangle$, so let us start by rewriting the initial state of the $\mathbf{X}$ in the Hadamard basis. Writing N for 2^n as usual, we have:

$$|\mathbf{0}^n, \mathbf{1}\rangle = \frac{1}{\sqrt{2^n}} \sum_{x \in \mathbb{Z}_N} |\hat{x}, \mathbf{1}\rangle .$$

As shown in Table 10 (third row, second column), this gives us:

$$\frac{1}{\sqrt{2^n}} \sum_{x \in \mathbb{Z}_N} |\hat{x}, \mathbf{1}\rangle \xrightarrow{\hat{C}Z^{\otimes n}} \frac{1}{\sqrt{2^n}} \sum_{x \in \mathbb{Z}_N} (-1)^{f(x)} |\hat{x}, \mathbf{1}\rangle .$$

(We could also have used the second column of the first row to arrive at this state directly.) If $f(x) = c$ for some constant c , then we have the state

$$\frac{1}{\sqrt{2^n}} \sum_{x \in \mathbb{Z}_N} (-1)^{f(x)} |\hat{x}, \mathbf{1}\rangle = \frac{1}{\sqrt{2^n}} \sum_{x \in \mathbb{Z}_N} (-1)^c |\hat{x}, \mathbf{1}\rangle \cong \frac{1}{\sqrt{2^n}} \sum_{x \in \mathbb{Z}_N} |\hat{x}, \mathbf{1}\rangle = |\mathbf{0}^n, \mathbf{1}\rangle ,$$

where we factored away the global phase. Hence, measuring the $\mathbf{X}$ register yields the outcome $\mathbf{0}^n$ with probability one.

To see what happens in the case that $f(x)$ is *balanced*, first translate the state back to the computational basis:

$$\frac{1}{\sqrt{2^n}} \sum_{x \in \mathbb{Z}_N} (-1)^{f(x)} |\hat{x}, \mathbf{1}\rangle = \frac{1}{\sqrt{2^n}} \sum_{z \in \mathbb{Z}_N} \frac{1}{\sqrt{2^n}} \sum_{x \in \mathbb{Z}_N} (-1)^{f(x) \oplus x \cdot z} |z, \mathbf{1}\rangle ,$$

flip the order of summation and define the amplitudes α_z for the outcomes of a computational-basis measurement,

$$= \frac{1}{\sqrt{2^n}} \sum_{z \in \mathbb{Z}_N} \frac{1}{\sqrt{2^n}} \sum_{x \in \mathbb{Z}_N} (-1)^{f(x) \oplus x \cdot z} |z, \mathbf{1}\rangle =: \frac{1}{\sqrt{2^n}} \sum_{z \in \mathbb{Z}_N} \alpha_z |z, \mathbf{1}\rangle .$$

To find the amplitude α_0 that the outcome $\mathbf{0}^n$ is observed, insert for $z = 0$:

$$\alpha_0 = \frac{1}{\sqrt{2^n}} \sum_{x \in \mathbb{Z}_N} (-1)^{f(x)} = 0 .$$

The sum evaluates to zero precisely because f is balanced *and* we are summing over an even number of x values, so every term cancels against a partner. Hence, we are guaranteed to *not* observe the outcome $\mathbf{0}^n$ in the balanced case. When combined with the above, this shows that the procedure returns $\mathbf{0}^n$ if *and only if* f is constant.

Bernstein–Vazirani. Given oracle access to a function $f : \{0, 1\}^n \rightarrow \{0, 1\}$ such that $f(x) = s \cdot x$ for some secret s , the Bernstein–Vazirani algorithm recovers s with a single call to the function, where any classical algorithm needs at least n calls. (The standard way to solve the problem classically would be to call $f(10 \dots 0)$ to recover the first bit of s , $f(010 \dots 0)$ to recover the second bit, and so on.)

The algorithm can be seen as a variant of the Deutsch–Jozsa algorithm, and it is notable for having been used to provide the first (relativized) separation between the classical complexity class BPP and the quantum complexity class BQP [BV97]. The algorithm is given in pseudocode in Fig. 20.

Let us analyze it, starting from the implementation on the left. The second cell of the first row of Table 10 tells us the effect of the $\leftarrow \hat{\oplus}$ operator on a computational basis state; inserting for $x = 0$, $y = 1$ gives us:

$$|\mathbf{0}^n, \mathbf{1}\rangle \xrightarrow{\mathbf{B} \leftarrow \hat{\oplus} f(\mathbf{X})} \frac{1}{\sqrt{2^n}} \sum_{z \in \{0, 1\}^n} (-1)^{f(z)} |\hat{z}, \mathbf{1}\rangle = \frac{1}{\sqrt{2^n}} \sum_{z \in \{0, 1\}^n} (-1)^{s \cdot z} |\hat{z}, \mathbf{1}\rangle ,$$

Bernstein–Vazirani	Alt. implementation
01 $\mathbf{XB} \leftarrow \mathbf{0}^n, \mathbf{1}\rangle$	01 $\mathbf{XB} \leftarrow \hat{\mathbf{0}}^n, \hat{\mathbf{1}}\rangle$
02 $\mathbf{B} \leftarrow \hat{\ominus} f(\mathbf{X})$	02 $\mathbf{B} \leftarrow \oplus f(\mathbf{X})$
03 $s \leftarrow \hat{\mathbf{X}} \mathbf{X}$	03 $s \leftarrow \hat{\mathbf{X}} \mathbf{X}$
04 return s	04 return s

Fig. 20. The Bernstein-Vazirani algorithm for recovering a secret s given oracle access to a function $f = s \cdot x$.

where we inserted the definition of f in the last step. But looking at the state on the right, we can recognize it as simply a Hadamard-translated state $|x\rangle$, except that $x = s$:

Hans: Add definition number that I can reference?

$$\frac{1}{\sqrt{2^n}} \sum_{z \in \{0,1\}^n} (-1)^{s \cdot z} |\hat{z}, \mathbf{1}\rangle = |s, \mathbf{1}\rangle .$$

Applying a computational-basis measurement to the $\mathbf{X}$ register, then, immediately yields the secret s .

Simon’s Algorithm. Simon’s algorithm is an early example of a quantum period-finding algorithm. Given a periodic two-to-one function f with a hidden period s , so that $f(x) = f(x \oplus s)$, it recovers s .

The main quantum step is in recovering a value x_i which is guaranteed to satisfy $x_i \cdot s = \mathbf{0}$. This subroutine is then repeated until enough such x_i have been recovered that the equation system is fully determined, at which point s may be recovered via basic linear algebra.

Simon’s subroutine	Alt. implementation
01 $\mathbf{XY} \leftarrow \hat{\mathbf{0}}^n, \mathbf{0}^n\rangle$	01 $\mathbf{XY} \leftarrow \mathbf{0}^n, \hat{\mathbf{0}}^n\rangle$
02 $\mathbf{Y} \leftarrow \oplus f(\mathbf{X})$	02 $\mathbf{Y} \leftarrow \hat{\ominus} f(\mathbf{X})$
03 $x \leftarrow \hat{\mathbf{X}} \mathbf{X}$	03 $x \leftarrow \hat{\mathbf{X}} \mathbf{X}$
04 return x	04 return x

Fig. 21. The quantum subroutine for Simon’s algorithm.

The quantum subroutine is shown in Fig. 21. Let us see how it gives us an x that is guaranteed to satisfy $x \cdot s = \mathbf{0}$, starting from the implementation shown on the left of Fig. 21.

$$|\hat{\mathbf{0}}^n, \mathbf{0}\rangle = \frac{1}{\sqrt{2^n}} \sum_{z \in \mathbb{Z}_N} |z, \mathbf{0}\rangle \xrightarrow{\mathbf{Y} \leftarrow \oplus f(\mathbf{X})} \frac{1}{\sqrt{2^n}} \sum_{z \in \mathbb{Z}_N} |z, f(z)\rangle .$$

After this, none of the following operations depend on the state of the $\mathbf{Y}$ register, so though the procedure in Fig. 21 is never shown to measure it, it makes our analysis a lot easier if we just measure it and discard the $\mathbf{Y}$ register. For each value y that we might observe, there are two input values z such that $f(z) = y$, so this is not enough to completely collapse the superposition:

$$\frac{1}{\sqrt{2^n}} \sum_{z \in \mathbb{Z}_N} |z, f(z)\rangle \xrightarrow{y \leftarrow \mathbf{Y}} \frac{|z\rangle + |z \oplus s\rangle}{\sqrt{2}} ,$$

where z satisfies that $f(z) = f(z \oplus s) = y$ for the value y that we measured. We are building towards a Hadamard-measurement, so translate this state to the dual domain so that we can inspect the amplitudes directly:

$$\frac{|z\rangle + |z \oplus s\rangle}{\sqrt{2}} = \frac{1}{\sqrt{2^n}} \sum_{x \in \mathbb{Z}_N} \frac{(-1)^{z \cdot x} |\hat{x}\rangle + (-1)^{(z \oplus s) \cdot x} |\hat{x}\rangle}{\sqrt{2}} = \sum_{x \in \mathbb{Z}_N} \frac{(-1)^{z \cdot x}}{\sqrt{2^{n+1}}} (1 + (-1)^{x \cdot s}) |\hat{x}\rangle =: \sum_{x \in \mathbb{Z}_N} \hat{\alpha}_x |\hat{x}\rangle .$$

Look at the amplitude $\hat{\alpha}_x$. Which values of x are we more likely to observe? Well, $\hat{\alpha}_x$ is proportional to the sum $1 + (-1)^{x \cdot s}$, where $x \cdot s$ denotes the bitstring inner product. There are only two cases: If $x \cdot s = 1$,

then $\hat{\alpha}_x = 1 - 1 = 0$, which means that the probability of observing such an outcome is also zero. Hence, for any x that we actually do observe, we must have that $x \cdot s = \mathbf{0}$. Indeed, restricting to $x \cdot s = \mathbf{0}$, we see that we have the state

$$\sum_{x : x \cdot s = \mathbf{0}} \frac{(-1)^{z \cdot x}}{\sqrt{2^{n+1}}} (1 + 1) |\hat{x}\rangle = \sum_{x : x \cdot s = \mathbf{0}} \frac{(-1)^{z \cdot x}}{\sqrt{2^{n-1}}} |\hat{x}\rangle ,$$

which as a sanity check we can see is normalized, since there are 2^{n-1} values of x that satisfy $x \cdot s = \mathbf{0}$. (Otherwise, that would have meant we had made an error somewhere.)

Applying a Hadamard-basis measurement to this state, then, is a procedure to sample uniformly from the set of x that satisfy $x \cdot s = \mathbf{0}$. Importantly, this shows that not only do we get a sample that satisfies our requirement, but we are also very unlikely to get the *same* x when we later repeat the subroutine, and that in all likelihood we will only need to repeat the procedure roughly n times to get n linearly independent values x_i that all satisfy $x_i \cdot s = \mathbf{0}$.

Grover's Algorithm.

Quantum Teleportation.

C.5 The Primer Paragraphs

Hans: copy-pasted while I decide what to keep where

The Hadamard Basis. Actually, we don't have to imagine explicitly applying the H gates to our qubits before measuring them, as Hadamard-basis measurements are perfectly well-defined on their own merits, as one choice among a continuous infinity of measurements we could choose to apply. Perhaps it is better, then, to view the Hadamard basis as a sort of “dual domain” under which we can view our qubit, with two possible questions we can ask our state: Either “Are you $|0\rangle$ or $|1\rangle$?”, or “Are you $|+\rangle$ or $|-\rangle$?” Either way, the qubit will provide a definite answer, after which its state will “collapse” to the answer you got.

This is a point worth internalizing, so let us repeat it: *The set of possible post-measurement states changes to match the question you chose to ask.* If you ask “Are you 0 or 1 ?”, then the post-measurement state will be $|0\rangle$ or $|1\rangle$; if you ask “Are you $+$ or $-$ ”, then the post-measurement state will be $|+\rangle$ or $|-\rangle$. So although at one level, the “collapse” of the quantum state is just the quantum version of how one would use Bayes' theorem to update a prior probability distribution to a marginal distribution conditioned on the outcome, at another level, it represents the deepest and strangest mystery in all of quantum physics. The grand mystery of quantum physics really comes down to this one fact, that the outcome statistics and the post-observation state both depend on *which measurement you chose to apply*, and it has led to a century of heated philosophical debate over how quantum theory should be interpreted. Ask five physicists for their opinion on the matter, and you are likely to get five different answers!⁵⁰

What about a question like “Are you $|0\rangle$ or are you $|+\rangle$?” Under our notion of measurement, this question is not well-defined, since $|0\rangle$ and $|+\rangle$ are not orthogonal states and therefore do not lead to mutually-exclusive outcomes. More on this in Sect. 5.7, but for now, think of a measurement as always being defined in terms of a set of mutually-exclusive outcomes.

Of course, any vector space has a continuous infinity of bases to choose from, each of which define a binary “question” we can pose to our qubit, and the Hadamard basis is just one particularly nice choice: geometrically, it is just the basis rotated by 45° relative to the original one, as illustrated in Fig. 22. To appreciate how this makes for a perspective “dual” to the computational basis, take a look at the following identities:

$$|+\rangle = \frac{|0\rangle + |1\rangle}{\sqrt{2}}, \quad |-\rangle = \frac{|0\rangle - |1\rangle}{\sqrt{2}} \quad \Longleftrightarrow \quad |0\rangle = \frac{|+\rangle + |-\rangle}{\sqrt{2}}, \quad |1\rangle = \frac{|+\rangle - |-\rangle}{\sqrt{2}}.$$

Notice the symmetry: The computational basis states $|0\rangle$ and $|1\rangle$ are to the Hadamard basis *exactly* what the plus state and minus states are to the computational basis; they really are *dual views*, which can be translated between via the map H .⁵¹

How does this duality generalize to multi-qubit states? We had that

$$|+^n\rangle = \frac{|0\dots 00\rangle + |0\dots 01\rangle + \dots + |1\dots 11\rangle}{\sqrt{2^n}} = \sum_{x \in \{0,1\}^n} \frac{1}{\sqrt{2^n}} |x\rangle$$

With $\{|0\rangle, |1\rangle\}$ and $\{|+\rangle, |-\rangle\}$ being exact duals of each other, then it must also be true that

$$|0^n\rangle = \frac{|+\dots +\rangle + |+\dots +\rangle + \dots + |-\dots -\rangle}{\sqrt{2^n}} = \sum_{\hat{x} \in \{+,-\}^n} \frac{1}{\sqrt{2^n}} |\hat{x}\rangle.$$

⁵⁰ The term “collapse” that we used above is typically associated with the rather conservative *Copenhagen interpretation*, championed by Niels Bohr, and commonly characterized as the “shut-up-and-calculate” approach, since it simply posits that “such questions are disallowed”. Another popular interpretation is the many-worlds interpretation due to Hugh Everett, favoured by father of quantum computing theory David Deutsch among others. Instead of the quantum state “collapsing” to a “classical” state, this interpretation says that both terms are ill-defined, and that the universe should instead be understood to be nothing but one big quantum superposition over all the worlds that could have possibly arisen from the initial conditions of the universe. (After all, we the observers are made of quantum particles too, so should we not then also be considered part of the quantum state?) If you're curious to learn more about the various interpretations of quantum mechanics, and the strengths and weaknesses of each attempt at a consistent resolution, Carlo Rovelli's book *Helgoland* provides an excellent overview.

⁵¹ The notion of a “dual domain” will sound familiar to anyone who has ever come across Fourier analysis, and not without reason: The Hadamard map is exactly the single-qubit Fourier transformation.

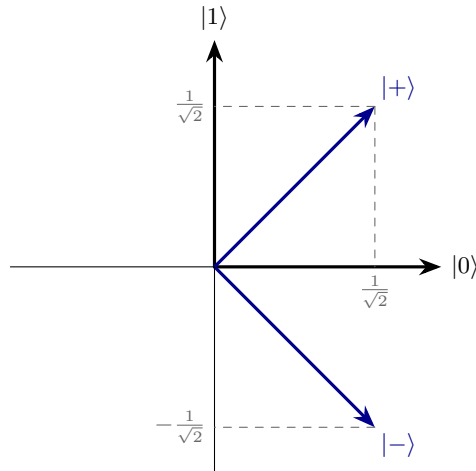


Fig. 22. The computational basis $\{|0\rangle, |1\rangle\}$ and the Hadamard basis $\{|+\rangle, |-\rangle\}$ describe the same qubit, seen along axes turned by 45° relative to each other.

Here, we have introduced a new kind of bitstring, one that is defined over the alphabet $\{+, -\}$ rather than the alphabet $\{0, 1\}$. We will use the convention that whenever a number or variable is represented with a hat, then it is understood to be a *Hadamard-bitstring* like above. For example, whereas we previously represented the decimal 21 by the bitstring $[21]_2 = 10101$, writing $\widehat{21}$ would imply that it is represented by the Hadamard-bitstring $[\widehat{21}]_2 = - + - + -$.

As we might have anticipated, just like the all-zero state $|\mathbf{0}^n\rangle$ maps to the decimal state $|0\rangle$, the uniform superposition $|+\rangle^n$ then maps to the Hadamard-zero state $|\hat{0}\rangle$. Thus, we recover our dual views on the quantum state space, which we can translate between as desired via the Hadamard transformation. Defining $\hat{\mathbb{Z}}_N := \{\hat{x} \mid x \in \mathbb{Z}_N\}$, and letting the dual-alphabet inner bitstring product $\hat{x} \cdot x$ simply treat $\{+, -\}$ as if it were $\{0, 1\}$, we can then express both directions of the Hadamard transformation in purely decimal notation:

$$|\hat{x}\rangle = \sum_{x \in \mathbb{Z}_N} \frac{(-1)^{\hat{x} \cdot x}}{\sqrt{2^n}} |x\rangle, \quad |x\rangle = \sum_{\hat{x} \in \hat{\mathbb{Z}}_N} \frac{(-1)^{x \cdot \hat{x}}}{\sqrt{2^n}} |\hat{x}\rangle.$$

The above notation will at times allow us to greatly simplify our notation, making arguments clearer. Consider for example the state

$$|\psi\rangle = -\frac{1}{\sqrt{2^n}} |0\rangle + \sum_{x \in \mathbb{Z}_N \setminus \{0\}} \frac{1}{\sqrt{2^n}} |x\rangle,$$

i.e. the uniform superposition state that singles out the $|0\rangle$ state (implicitly understood to be the $|\mathbf{0}^n\rangle$ state since we are using decimal notation) and applies a negative phase to it. Computing the Hadamard transformation of such a state can lead to a mess of double sums, but using the above notation, not only is applying the transformation as simple as adding hats everywhere,

$$-\frac{1}{\sqrt{2^n}} |0\rangle + \sum_{x \in \mathbb{Z}_N \setminus \{0\}} \frac{1}{\sqrt{2^n}} |x\rangle \xrightarrow{H^{\otimes n}} -\frac{1}{\sqrt{2^n}} |\hat{0}\rangle + \sum_{\hat{x} \in \hat{\mathbb{Z}}_N \setminus \{\hat{0}\}} \frac{1}{\sqrt{2^n}} |\hat{x}\rangle,$$

but we can then even choose to write out the state using a mix of notations, “unpacking” the complex-phase term for closer study while keeping the real-valued terms around in their Hadamard-bitstring form:

$$-\frac{1}{\sqrt{2^n}} |\hat{0}\rangle + \sum_{\hat{x} \in \hat{\mathbb{Z}}_N \setminus \{\hat{0}\}} \frac{1}{\sqrt{2^n}} |\hat{x}\rangle = \sum_{x \in \mathbb{Z}_N} -\frac{1}{2^n} |x\rangle + \sum_{\hat{x} \in \hat{\mathbb{Z}}_N \setminus \{\hat{0}\}} \frac{1}{\sqrt{2^n}} |\hat{x}\rangle.$$

D The **T**-transform

The **T**-transform is the first half of the Fujisaki–Okamoto transformation, which derandomizes the PKE by hashing the message, enabling a re-encryption consistency check to the decryption algorithm.

Definition 22 (T-transform). *To a PKE scheme $\text{PKE} = (\text{KeyGen}, \text{Enc}, \text{Dec})$ and a hash function $G : \mathcal{M} \rightarrow \mathcal{R}$, we associate the deterministic PKE scheme $\text{T}[\text{PKE}, G]$ defined in Figure 23.*

$\text{Enc}_{\text{pk}}^{\text{T}}(m) :$	$\text{Dec}_{\text{sk}}^{\text{T}}(c) :$
01 $c \leftarrow \text{Enc}_{\text{pk}}(m; G(m))$	03 $m' \leftarrow \text{Dec}_{\text{sk}}(c)$
02 return c	04 if $m' = \perp$ or $c \neq \text{Enc}_{\text{pk}}(m; G(m))$
	05 return $\perp$
	06 else return m'

Fig. 23. Algorithms of $\text{T}[\text{PKE}, G] =: (\text{KeyGen}, \text{Enc}^{\text{T}}, \text{Dec}^{\text{T}})$.

E A Brief Quantum Information Theory Primer

F Deriving the Tools

The previous sections were concerned with how cryptographers can use the tools developed for the QROM to give post-quantum proofs of security. There, I did my best to leave the quantum analyses to surface-level only, taking the various lemmas as given, and focusing instead on developing an intuition for how to reason about the QROM.

However, proving results about the QROM itself can serve as rich playground for the more powerful tools found in *Quantum Information Theory*. So if you, like many of us before you, have gotten a taste for all things quantum, and would like to dive deeper, then this is the section for you.

In the following, I will introduce the powerful *density matrix* formulation of quantum mechanics, introduced by legendary physicist and computer scientist John von Neumann, and show how, once understood, it can provide elegant (and often simple!) proofs of the lemmas that we employed.

Hans: In which I present all the tools needed to prove all the lemmas contained herein, some of which are then proven in-text and some of which are left as exercises (but with solutions).

F.1 The Density Matrix Formalism

Hans: A primer.

F.2 Tools of Quantum Information Theory

Hans: Distance measures, distinguishability, gentle measurements, almost-commuting; essentially a summary of Wilde chapter 9.

F.3 Equivalent Oracles

Superposition Oracles

Hans: A “warmup” proof showing that superposition oracles are equivalent to monolithic oracles.

Superposition Oracles

Hans: The more involved proof of CROs being the same.

F.4 One-Way to Hiding

The Original O2H Lemma

Hans: Include as warmup?

The Semi-Classical O2H Lemma

Hans: Prove the most general version; essentially just repeat the proof of the paper in your own notation.

Next, let’s re-derive the Semi-Classical O2H bound by introducing $\mathcal{M}_{\text{ext}}$ directly into the random oracle. If $\mathcal{E}$ has access to a qPCA oracle, then they may implement this oracle directly, which is shown in Fig. 24.

We again consider the setting that $\mathcal{D}$ is given some input z that may depend arbitrarily on $\mathcal{H}_0$, and that $\mathcal{E}$ provides a simulation for $\mathcal{D}$, except now it is implemented as a Semi-Classical CRO.

Hans: The CRO is probably unnecessary; however, if we want $\mathcal{H}_1$ to be related to $\mathcal{H}_0$ in the way specified in the SC-O2H, then we could probably benefit from formulating it in terms of an independent oracle, and using the CSWAP technique. (Of course, the independent oracle *could* then be implemented as a CRO.)

Or wait . . . the measurement projects the state onto the subspace independent of $\mathcal{S}$. . . so we *can* really just call $\mathcal{H}_0$ instead?

```

Oracle  $\mathcal{H}_1(\mathbf{X}, \mathbf{Y} : \mathbf{B})$ 


---


// 1. Check input
foreach  $|x\rangle$  in  $\mathbf{X}$  :
    if  $x \in \mathcal{S}$  : Apply( $X \rightarrow \mathbf{B}$ )
b  $\leftarrow$   $\$$  Meas( $\mathbf{B}$ )
if  $b = 1$  : abort
// 2. Process query
 $(\mathbf{X}, \mathbf{Y}) \leftarrow \mathcal{H}_0(\mathbf{X}, \mathbf{Y})$ 
return  $(\mathbf{X}, \mathbf{Y})$ 

```

Fig. 24. The simulated semi-classical oracle $\mathcal{H}_1$, implemented as a compressed oracle (recalled from Def. 17). Here, $\mathbf{B}$ is a single-qubit register initialized to $|0\rangle$, and $\mathbf{D}$ is the database, initialized to the list of q identical tuples $(\varepsilon, 0^m)$, (i.e. at the outset, the oracle runs the method $\mathbf{D} \leftarrow \text{Init}(q \times |\varepsilon, 0^m\rangle)$).

The Resampling Lemma

Hans: Include, or just rely on Joseph's permutations instead?

The Adaptive Reprogramming Framework

Hans: Include, or just rely on Joseph's permutations instead?

Joseph's Counterexample

Hans: Showing how the proof, which we saw took a lot of calculation to derive the state directly, becomes like two lines with the tools that we now have.

G Select Exercise Solutions

Solution to Exercise 5 Let U be such that, for all states $|\psi\rangle$, $\|U|\psi\rangle\| = \||\psi\rangle\|$. Using the definition of the norm, this gives us that $|(U|\psi\rangle)^\dagger U|\psi\rangle| = |\langle\psi|U^\dagger U|\psi\rangle| = |\langle\psi|\psi\rangle|$. Since this should hold for any $|\psi\rangle$, we must have that $U^\dagger U = \mathbf{1} = UU^\dagger$; the latter equality follows from the fact that, for all $|\varphi\rangle$ and $|\xi\rangle$, $|\langle\varphi|\xi\rangle| = |\langle\xi|\varphi\rangle|$. Thus, U satisfies the definition of an invertible matrix, with the inverse $U^\dagger$. Further, it is easy to see that the opposite also holds: any unitary matrix must be norm-preserving. Therefore, the set of norm-preserving operations on N -dimensional Hilbert space equal to the set of $N \times N$ unitary matrices.

Solution to ?? Let P_φ be the projector that projects a state $|\psi\rangle$ onto the one-dimensional subspace spanned by the state $|\varphi\rangle$. Then,

$$P_\varphi^\dagger = (|\varphi\rangle\langle\varphi|)^\dagger = |\varphi\rangle\langle\varphi| = P_\varphi.$$

Since $|\varphi\rangle$ is arbitrary, and any projector can be written as a sum of such one-dimensional projectors, *and* a sum of Hermitian matrices must itself be Hermitian:

$$P^\dagger = \sum_i P_i^\dagger = \sum_i P_i = P,$$

this suffices to show that any projector must be Hermitian.